# Reading Guide: AI-Assisted App Reconstruction (Vibe Coding)

Evaluating AI-Assisted Reverse Engineering for Software Application Reconstruction through Vibe Coding: Functional Fidelity, Code Quality, and Maintainability

Generated 2026-11-12

# Overview

## What this research is about

This project asks a practical question. Suppose you use an AI model to rebuild an existing app, and you only give it what anyone can see from the outside: the screens, how the app reacts when you click or type, and its documentation. You never show it the source code. People call this kind of work "reverse engineering". When the rebuilding is done mostly by talking to an AI in plain language and accepting the code it writes, people call it "vibe coding". How good is the rebuilt app? We judge "good" in three ways. Does it behave like the original (functional fidelity)? Is its code well written (code quality)? Can people understand and change it later (maintainability)? We read 23 recent papers in full to find out what is already known, and what is still missing.

The five research questions (RQs), in plain words:

1. **RQ1. Approaches.** What AI methods exist for rebuilding apps from what you can observe, and how do they relate to vibe coding?
2. **RQ2. Behaviour.** How do studies measure whether a rebuild behaves like the original, and how close do current AI tools get?
3. **RQ3. Code quality and maintainability.** Is AI-written or vibe-coded app code well written and easy to maintain, compared with human code?
4. **RQ4. Study quality.** How trustworthy are the current studies? What do they do well, and where are they weak?
5. **RQ5. Gaps.** What is still unknown, and what study could a beginner run to fill a gap?

## How to use this guide

The guide has three parts:

- **This overview.** Read it first. It gives the main answers, a map of all 23 papers, the five papers to read first, and where studies disagree.
- **The paper briefs.** One short brief per paper, in the same order as the paper map below. Read the five "read first" briefs, then the briefs for the group closest to your interest.
- **Research gaps, glossary and notes.** The last part explains the open questions (gaps G1 to G8), a cybersecurity follow-up, concrete next steps, a glossary of every technical term, and how this guide was made.

If a word is new to you, look it up in the glossary at the end.

**Evidence labels.** Each important statement carries one label:

- **[V] Verified.** We checked it in the paper's own data, tables or results.
- **[A] Author claim.** The authors say it, but we could not check it ourselves (for example, a claim that their tool is new).
- **[I] Interpretation.** Our own reasoning, built from the cited papers. Treat it as an argument, not a fact.

**Citations.** A citation looks like this: [borg2026echoes, p. 13]. The first part (borg2026echoes) is the paper's key, which is also the name of its folder in the project. "p. 13" is the page of the PDF we hold. For example, that citation points to the page of the Echoes of AI paper that describes its maintenance-task design. Page numbers refer to our PDF copies, which for several papers are the arXiv versions, so they may differ from the publisher's page numbers.

**Peer-reviewed vs preprint.** Peer review means independent experts checked a paper before it was published in a journal or conference. A preprint is a paper shared publicly (often on arXiv) before or without that check.

- 16 of our 23 papers are peer-reviewed. 7 are preprints: bai2026recreationworld, chen2025iwr, kim2026programdistill, mota2026comparing, pan2026functional, yang2026programbench and zhao2025vibe.
- We use preprints but give them less weight. The paper map marks each paper's status.
- One special case: our database lists zhao2025vibe as a preprint, but the project notes say it was published at ICML 2026, without a DOI. We stay cautious and treat it as a preprint.

**Evidence strength.** We rate how much we trust each answer:

- **Strong:** enough data, sound design, and several studies agree.
- **Moderate:** sound design, but a single study or limited data.
- **Weak:** small or unclear samples, design problems, or preprints nobody has repeated.

## Main findings at a glance

> **In one sentence:** AI rebuilds of existing apps are still far from complete, they copy what the screen looks like better than what the app does, and almost nobody has checked whether the rebuilt code is as easy to maintain as the original.

### RQ1. What approaches exist?

> **Answer:** six families of approaches exist; the closest match to "AI reverse engineering" is AI agents that rebuild an app from a running original. Evidence strength: moderate.

- **[V]** The six families are: screenshot or design to code; interaction video or screenshots to code; agentic rebuilds from a running original; written specification to a full app; commercial vibe-coding platforms and AI code editors; and recovering behaviour rules from a running app [si2025benchmarking, p. 4; chen2025iwr, p. 5; bai2026recreationworld, p. 3; tran2026vibe, p. 3; mota2026comparing, p. 2; xiong2026propgen, p. 4].
- **[V]** Real-world vibe coding means a person describes goals in plain language, re-prompts when things go wrong, and reviews the code very little [fawzy2026vibe, p. 1].
- **[I]** Almost every rebuild study tests autonomous agents, with no person steering. Human-steered vibe coding appears only in small or indirect studies [mock2026vibe, p. 15; asdaque2026novice, p. 1].

- **[I]** Papers use the words "vibe coding" for different things, so their results should not be added together.

## RQ2. Does the rebuild behave like the original?

> **Answer:** only partly, everywhere. Exact behavioural copies are very rare, and interaction logic is the weakest part. Evidence strength: moderate for this general picture, weak for any exact number.

- **[V]** Studies decide "correct" with four kinds of oracle (the reference that judges an output): behavioural tests run against the original, AI judges, visual similarity, and navigation maps [bai2026recreationworld, p. 9; tran2026vibe, p. 4; si2025benchmarking, p. 3; zhou2025declarui, p. 7].
- **[V]** A full behavioural match is rare: 2.8% of apps in one study and 0 of 200 programs in another. Both studies are preprints [bai2026recreationworld, p. 11; yang2026programbench, p. 6].
- **[V]** The best results at the level of whole workflows or requirements are about 49–62% [kim2026programdistill, p. 11; liu2026benchmarking, p. 6; tran2026vibe, p. 6].
- **[V]** Rebuilds copy static structure more reliably than what happens after an action [bai2026recreationworld, p. 16; chen2025iwr, p. 16].
- **[V]** Caution: in chen2025iwr, the "visual beats interaction" gap is partly built into how the scores are calculated [chen2025iwr, p. 7].
- **[I]** Each benchmark uses a different oracle, input and app type. Do not rank the percentages against each other.

## RQ3. Is the code good and maintainable?

> **Answer:** the evidence is mixed, and no study compares the maintainability of a rebuild with its original. Evidence strength: weak to moderate.

- **[V]** After teams adopted the Cursor AI editor, project-level SonarQube warnings rose by 30.26% and cognitive complexity by 41.64% [he2026speed, p. 7]. But with another accepted statistical method both effects became negative and not significant [he2026speed, p. 16].
- **[V]** A preregistered randomised experiment found no significant difference when a second developer extended code written with or without AI help [borg2026echoes, p. 33; borg2026echoes, p. 38]. It had fewer participants than planned, so it can rule out only large effects [borg2026echoes, p. 19].
- **[V]** Rebuilds are smaller and more concentrated in a few files than their originals (two preprints) [bai2026recreationworld, p. 15; yang2026programbench, p. 9].
- **[V]** Quality differs by vibe-coding platform (preprint, 9 apps) [mota2026comparing, p. 5], and AI code shows different kinds of code smells from human code [zhu2026understanding, p. 7].
- **[I]** The code-checking tools (SonarQube, PyExamine, CodeHealth, Maintainability Index) give different pictures, so the answer may depend on the tool.

## RQ4. How trustworthy are the studies?

> **Answer:** there are good practices, but most benchmarks run each test only once, and maintainability is almost never measured properly. Evidence strength: strong for these two weaknesses, moderate for the others.

- **[V]** Strengths: tests that check behaviour without caring how the rebuild is built, controls against copying the original, checks of the test suites, checks of AI judges against humans, and one preregistered randomised trial [yang2026programbench, p. 18; liu2026benchmarking, p. 14; tran2026vibe, p. 9; borg2026echoes, p. 19].
- **[V]** Many benchmarks report a single run with no uncertainty and no significance tests [lu2025webgen, p. 40; yang2026programbench, p. 10]. In one study, differences between generation runs explained 73.5–92.5% of the variation in scores [tran2026vibe, p. 8].
- **[V]** Some tests depend on the original's internal names, which were even given to the AI in the prompt [liu2026benchmarking, p. 16].
- **[V]** No rebuild or generation benchmark compares AI with human developers [yang2026programbench, p. 20; tran2026vibe, p. 5].

## RQ5. What is still missing?

> **Answer:** the top gap (G1) is a study that measures, on the same AI rebuilds, both behavioural fidelity and maintainability, with the original app as the comparison. Our searches on 2026-11-12 found no such study. Evidence strength: moderate.

- **[I]** The evidence that this gap exists is strong [bai2026recreationworld, p. 15; yang2026programbench, p. 9; pan2026functional, p. 3; wu2026looks, p. 4; tran2026vibe, p. 9].
- **[I]** Five more gaps are partly new, and two are mostly or partly addressed by 2026 work. A security version of the question is also partly new. All are explained in the gaps section at the end.

# Paper map

The papers are grouped by approach family, following the methodology comparison:

- **F1. Screenshot or design to code.** The AI sees pictures of an app and writes code that looks the same.
- **F2. Rebuild from observed behaviour.** The AI watches a video or uses a running original and rebuilds it. This is the closest match to "AI reverse engineering".
- **F3. Written specification to a full app.** The AI gets a text description or requirements and builds the whole app.
- **F4. Code-quality studies.** Studies that scan AI-written code with code-checking tools.
- **F5. Human-centred and workflow studies.** Experiments with developers, repository studies and literature reviews.
- **F6. Behaviour-rule recovery.** Learning rules about how an app behaves by running it.

Relevance (high, medium, low) is our rating of how close each paper is to the project question.

| # | Group | Paper (short title) | Year | Type | Status | What it is about | Relevance |
|---|---|---|---|---|---|---|---|
| 1 | F1 | Design2Code (si2025benchmarking) | 2025 | Benchmark | Peer-reviewed (NAACL 2025) | Real webpage screenshots turned into HTML, scored on visual similarity | high |
| 2 | F1 | Interaction2Code (xiao2025benchmarking) | 2025 | Benchmark + user study | Peer-reviewed (ASE 2025) | Rebuilding webpage interactions from before/after screenshots | high |
| 3 | F1 | DeclarUI (zhou2025declarui) | 2025 | Technique + user study | Peer-reviewed (Proc. ACM Softw. Eng.) | Mobile UI designs to React Native code with page-navigation maps | high |
| 4 | F1 | Looks Right, Works Right (wu2026looks) | 2026 | Benchmark | Peer-reviewed (EMNLP 2026) | Multi-screen mobile app screenshots to working app, with navigation tests | high |
| 5 | F2 | IWR-Bench (chen2025iwr) | 2025 | Benchmark | Preprint | Rebuilding interactive webpages from a video of someone using them | high |
| 6 | F2 | RecreationWorld (bai2026recreationworld) | 2026 | Benchmark | Preprint | AI agents rebuild apps on five platforms from a running original | high |
| 7 | F2 | ProgramDistill (kim2026programdistill) | 2026 | Benchmark | Preprint | AI agents rebuild web apps by exploring a live reference app | high |
| 8 | F2 | ProgramBench (yang2026programbench) | 2026 | Benchmark | Preprint | AI agents rebuild 200 programs from the program and its documentation | high |
| 9 | F3 | E2EDev (liu2026benchmarking) | 2026 | Benchmark | Peer-reviewed (ACL 2026) | Rebuilding existing web apps from their requirements, with executable tests | high |
| 10 | F3 | WebGen-Bench (lu2025webgen) | 2025 | Benchmark | Peer-reviewed (NeurIPS 2025) | Building websites from text instructions, judged by an AI tester | high |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 11 | F3 | Vibe Code Bench (tran2026vibe) | 2026 | Benchmark | Peer-reviewed (CAIS 2026) | Full web apps from text specifications, judged by an AI browser agent | high |
| 12 | F3 | RAL-Bench (pan2026functional) | 2026 | Benchmark | Preprint | Python projects from requirements, scored on function and code quality | high |
| 13 | F3 | SusVibes (zhao2025vibe) | 2025 | Benchmark | Preprint (published at ICML 2026 per project notes) | Whether code written by AI agents is secure in real projects | medium |
| 14 | F4 | Is LLM Code More Maintainable? (molison2025llm) | 2025 | Comparative static analysis | Peer-reviewed (ESEM 2025) | GPT-4o versus human solutions, checked with SonarQube | medium |
| 15 | F4 | Quality of Vibe Coding Tools (mota2026comparing) | 2026 | Exploratory static analysis | Preprint | Code quality of apps from Lovable, v0 and Replit | high |
| 16 | F4 | Code and Architectural Smells (zhu2026understanding) | 2026 | Exploratory static analysis | Peer-reviewed (ASE 2026) | Code smells in LLM, agent and human code | high |
| 17 | F4 | Speed at the Cost of Quality (he2026speed) | 2026 | Quasi-experiment on repositories | Peer-reviewed (MSR 2026) | Effect of adopting the Cursor AI editor on speed and quality | high |
| 18 | F5 | Echoes of AI (borg2026echoes) | 2026 | Preregistered randomised trial | Peer-reviewed (Empirical Software Engineering) | Is AI-assisted code harder for the next developer to change? | high |
| 19 | F5 | Vibe Coding and TDD (mock2026vibe) | 2026 | Small experiment | Peer-reviewed (Empirical Software Engineering) | Solo, collaborative and agent vibe coding on small test-driven tasks | medium |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 20 | F5 | Novice Developers and Review Overhead (asdaque2026novice) | 2026 | Repository mining | Peer-reviewed (MSR 2026) | AI-agent pull requests from novice versus experienced developers | low |
| 21 | F5 | Vibe Coding in Practice (fawzy2026vibe) | 2026 | Grey-literature review | Peer-reviewed (ICSE-SEIP 2026) | What practitioners write about vibe coding online | medium |
| 22 | F5 | Faster Code, Deeper Debt? (ehsani2026faster) | 2026 | Multivocal literature review | Peer-reviewed (TOSEM) | Technical debt caused by LLM-assisted development | high |
| 23 | F6 | PropGen (xiong2026propgen) | 2026 | Tool + evaluation | Peer-reviewed (ASE 2026) | Learning behaviour rules from running Android apps for testing | medium |

## Reading order: the five papers to read first

Start with these five. Together they cover the core of the topic and give you the tools for a first study.

1. **RecreationWorld (bai2026recreationworld, preprint).** The closest match to this project. AI agents rebuild apps on five platforms from a running original, with no source code. It reports a 2.8% full-pass rate [bai2026recreationworld, p. 11] and measures how much smaller the rebuilds are than the originals [bai2026recreationworld, p. 15].
2. **ProgramBench (yang2026programbench, preprint).** Rebuilds 200 programs from the program plus its documentation. Read it for careful test design, controls against copying, and a comparison of rebuilt code structure with the originals [yang2026programbench, p. 6; yang2026programbench, p. 9].
3. **Echoes of AI (borg2026echoes, peer-reviewed).** The only randomised test of maintainability in our set. Its "second developer extends the code" design is the model for the maintenance task in the top gap, G1 [borg2026echoes, p. 13].
4. **E2EDev (liu2026benchmarking, peer-reviewed).** Rebuilds existing web apps from their requirements, with human-reviewed tests that pass on the originals [liu2026benchmarking, p. 5]. It is a good source of apps and tests for a pilot study. Note its weakness: tests depend on the original's internal names [liu2026benchmarking, p. 16].
5. **Vibe Code Bench (tran2026vibe, peer-reviewed).** The benchmark that uses the term "vibe coding" directly. It reports problems with AI judges [tran2026vibe, p. 9] and large run-to-run variation [tran2026vibe, p. 8], and it releases its generated apps for reuse [tran2026vibe, p. 11].

## Where studies disagree

All explanations here are our interpretation **[I]**. We do not pick a side. Full entries are in the project's evidence matrix.

- **C1. Does AI assistance harm maintainability?** Warnings and complexity rose after Cursor adoption [he2026speed, p. 7], but a randomised trial found no difference for the next developer [borg2026echoes, p. 33]. **[I]** Different settings, units and statistical robustness; neither settles it.
- **C2. Fewer or more code problems in AI code?** GPT-4o code had fewer SonarQube "bugs" than human solutions [molison2025llm, p. 8], yet repository warnings rose with Cursor [he2026speed, p. 7]. **[I]** Single programs versus whole projects, with different baselines: not comparable.
- **C3. Which code-quality tool can we trust?** One study chose PyExamine over SonarQube's "superficial linting" [zhu2026understanding, p. 2], and a review notes that technical-debt tools disagree [ehsani2026faster, p. 5]. **[I]** The answer may depend on the tool.
- **C4. Does letting the AI test and fix its own work help?** Repair strategies lowered code quality without improving correctness in one single-model preprint [pan2026functional, p. 8], while targeted repair loops helped elsewhere [zhou2025declarui, p. 10; wu2026looks, p. 12]. **[I]** Feedback seems to improve only what it checks.
- **C5. Do rebuilds "look right but behave wrong"?** Behaviour scored below appearance in some benchmarks [chen2025iwr, p. 8], but the visual score was lower for 9 of 10 models in another [bai2026recreationworld, p. 11]. **[I]** Only "structure is easier than interaction" holds up.
- **C6. Do multi-agent frameworks help?** MetaGPT nearly always failed in E2EDev [liu2026benchmarking, p. 6] but scored highest in a small experiment [mock2026vibe, p. 41]. **[I]** Task size and model generation differ.
- **C7. Is generated code secure?** One preprint gave a high static security score [pan2026functional, p. 6]; another found that most working solutions failed security tests [zhao2025vibe, p. 6]. **[I]** They measure different things; less code means fewer static findings.
- **C8. More code or less code?** Rebuilds are much smaller than originals [bai2026recreationworld, p. 15], but AI-assisted development adds code [he2026speed, p. 8]. **[I]** Size means different things here; divide quality counts by code size.
- **C9. Same words, different things.** "Vibe coding" is defined differently across papers [fawzy2026vibe, p. 1; asdaque2026novice, p. 1], and fidelity scores use different oracles. **[I]** Do not pool these results.
- **C10. The AI family that builds the test sometimes wins it.** This happened in several benchmarks [yang2026programbench, p. 3; tran2026vibe, p. 3] but not in others [bai2026recreationworld, p. 11]. **[I]** No paper tests this bias directly; it is a warning sign, not proof.

# Paper Summaries

This part has one brief for each of the 23 papers, in the same order as the Paper Map. Each brief starts with a one-line takeaway and a fact table (authors, venue, peer-reviewed or preprint, identifier, relevance). Then it explains in plain words what the paper did, how, with which data, what it found, its weaknesses, the gaps it leaves open, why it matters for your research, and how it connects to the other papers.

# Design2Code: Benchmarking Multimodal Code Generation for Automated Front-End Engineering

> **Takeaway:** AI models can turn a screenshot of a webpage into code that looks quite similar, and GPT-4o was best on every visual measure [si2025benchmarking, p. 6]. But the benchmark only checks how the page looks. It never checks whether anything works or whether the code is maintainable [si2025benchmarking, p. 3].

| Item | Detail |
|---|---|
| Authors | Chenglei Si, Yanzhe Zhang, Ryan Li, Zhengyuan Yang, Ruibo Liu, Diyi Yang |
| Year · Venue | 2025 · Proceedings of NAACL 2025 (Nations of the Americas Chapter of the ACL: Human Language Technologies, Volume 1: Long Papers) |
| Status | Peer-reviewed (conference long paper) |
| Identifier | DOI 10.18653/v1/2025.naacl-long.199 (arXiv 2403.03163) |
| Paper type | Benchmark with automatic metrics and a human evaluation |
| Relevance to us | High — RQ1, RQ2, RQ4, RQ5 (RQ3 only by absence) |

## In plain words

Front-end developers often turn a visual design into web code by hand. This paper asks how well AI models that read images (multimodal models) can do this automatically: you give them a screenshot of a real webpage, and they write the HTML and CSS [si2025benchmarking, p. 1]. The authors built the first real-world test set for this task and new ways to score how close the result looks to the original. This is the starting point of the "screenshot to code" research line. It matters for us because many later reconstruction studies reuse its metrics, and because it clearly shows what visual scoring leaves out.

## How they did it

1. They took real webpages linked from C4 (a large public collection of web text), cleaned them and put all styling into one HTML file per page [si2025benchmarking, p. 2].
2. Automatic filters cut the pool to 14k pages. Scripts, iframes, SVGs and external links were removed, and images were replaced by placeholders. The first two authors then hand-picked 484 test pages [si2025benchmarking, p. 13].
3. Models were asked to write one HTML+CSS file from the screenshot. The prompt says no JavaScript is needed [si2025benchmarking, p. 14].
4. They tried three prompting styles [si2025benchmarking, p. 4]. Direct gives just the screenshot. Text-Augmented adds the page's text. Self-Revision lets the model see its own output and fix it.

5. They also fine-tuned (further trained on example data) their own open model, Design2Code-18B [si2025benchmarking, p. 5].
6. They rendered each result and compared it with the original screenshot using five measures: Block-Match (are the text blocks there), Text, Position, Color and CLIP (overall visual similarity from an image-understanding model) [si2025benchmarking, p. 4].
7. Paid human raters compared outputs in pairs and judged whether a generated page could replace the original [si2025benchmarking, p. 5].

## Data and setup

- 484 test webpages, plus a harder set of 80 pages from GitHub [si2025benchmarking, p. 8].
- Pages are long: 31,216 tokens and 158 HTML tags on average [si2025benchmarking, p. 3].
- Models: GPT-4o, GPT-4V, Claude 3 Opus, Gemini 1.0 Pro Vision, several open models and two fine-tuned models [si2025benchmarking, p. 5].
- Commercial models used greedy decoding (always the most likely output), so one run per setting [si2025benchmarking, p. 14].
- Human evaluation: 100 sampled pages, 5 raters per item, 60 participants in total [si2025benchmarking, p. 15].

## Key findings

- **[V]** GPT-4o Direct is best on all automatic measures: Block 93.0, Text 98.2, Position 85.5, Color 84.1, CLIP 90.4 [si2025benchmarking, p. 6]. Humans preferred it over the Gemini baseline 92% of the time [si2025benchmarking, p. 7].
- **[A]** Models "mostly lag in recalling visual elements" and in getting the layout right [si2025benchmarking, p. 1].
- **[V]** On the hard set, Block-Match falls to 56.6 to 75.1, and GPT-4o Direct scores 56.6 [si2025benchmarking, p. 8].
- **[V]** Adding the page text raised Block-Match for 4 of 7 models but lowered it for GPT-4o (93.0 to 92.4) and Claude 3 Opus (90.2 to 89.8) [si2025benchmarking, p. 6].
- **[V]** 49% of GPT-4V self-revised pages were judged able to replace the original [si2025benchmarking, p. 6].
- **[V]** The measures predict human preference with 79.9% accuracy. Exact text similarity counted negatively: humans care more about layout and content than exact wording [si2025benchmarking, p. 7].
- **[V]** Human raters agreed only moderately or weakly with each other: Fleiss' kappa (a chance-corrected agreement score) of 0.46, 0.32 and 0.26 [si2025benchmarking, p. 16].
- **[V]** Pages with more HTML tags are harder ($r = -0.28$ with Block-Match) [si2025benchmarking, p. 15].

## Limitations and weaknesses

- **[A]** Only static pages are covered; dynamic pages need evaluation of "interactive functions beyond just visual similarity" [si2025benchmarking, p. 10].

- **[A]** Humans should not be "blindly trusted as the oracle" because of attention bias [si2025benchmarking, p. 7].
- **[A]** Complex pages need better prompting, such as generating page parts step by step [si2025benchmarking, p. 10].
- **[I]** Fidelity is purely visual. There is no test of behaviour or code quality [si2025benchmarking, p. 3].
- **[I]** The task rebuilds a stripped-down snapshot of a page, not a working application [si2025benchmarking, p. 13].
- **[I]** The very high agreement with humans (r = 0.975) is based on 10 data points and partly on the same pages used to fit the model, so it is not an independent check [si2025benchmarking, p. 17].
- **[I]** C4 pages are common training data, and contamination is not discussed [si2025benchmarking, p. 2].
- **[I]** There are no significance tests or run-to-run variance for model comparisons [si2025benchmarking, p. 6].

## Gaps and future work this paper leaves open

- G1: no code quality or maintainability measure [si2025benchmarking, p. 3].
- G2: metric agreement with humans is checked in-sample, and human agreement is low [si2025benchmarking, p. 17].
- G5: **[I]** no contamination discussion was found.
- **[A]** The authors call for interactive pages, Figma or sketch inputs, and evaluation beyond looks [si2025benchmarking, p. 10].

## Why it matters for your research

**[I]** Design2Code tells you how to measure whether a rebuild looks like the original, and its five visual metrics are widely reused. Use them only as a secondary score. Visual similarity says nothing about whether buttons work or whether the code is easy to change. If you use human ratings, report agreement between raters and validate on pages you did not use for fitting.

## Connections to other papers

- xiao2025benchmarking builds on it, filtering pages the same way and adding interactions [xiao2025benchmarking, p. 2].
- **[A]** chen2025iwr's authors describe Design2Code as limited to static reconstruction [chen2025iwr, p. 2]. bai2026recreationworld also cites it as related work [bai2026recreationworld, p. 21].
- Like xiao2025benchmarking and kim2026programdistill, it measures no code quality [xiao2025benchmarking, p. 5; kim2026programdistill, p. 14].
- **[I]** Its weak human agreement echoes other judge checks in the project, such as a moderate 0.65 correlation in chen2025iwr [chen2025iwr, p. 17].

# Interaction2Code: Benchmarking MLLM-based Interactive Webpage Code Generation from Interactive Prototyping

> **Takeaway:** When AI models rebuild a webpage from screenshots taken before and after each click, they copy the interactive parts worse than the page as a whole, for every model tested [xiao2025benchmarking, p. 6]. Even the best setup produces usable interactions about 81% of the time, and code quality is never checked [xiao2025benchmarking, p. 6].

| Item | Detail |
|---|---|
| Authors | Jingyu Xiao, Yuxuan Wan, Yintong Huo, Zixin Wang, Xinyi Xu, Wenxuan Wang, Zhiyao Xu, Yuhang Wang, Michael R. Lyu |
| Year · Venue | 2025 · 40th IEEE/ACM International Conference on Automated Software Engineering (ASE 2025) |
| Status | Peer-reviewed (conference paper). Our copy is the later arXiv v3 version, so page numbers follow that version |
| Identifier | DOI 10.1109/ase63991.2025.00028 (arXiv 2411.03292) |
| Paper type | Benchmark and empirical study, with a failure taxonomy and a small user study |
| Relevance to us | High — RQ1, RQ2, RQ4, RQ5 (RQ3 only by absence) |

## In plain words

Earlier screenshot-to-code tests only looked at static pages. Real webpages change when you click a button or open a menu. This paper asks whether AI models that read images can rebuild those interactions from an "interactive prototype": a screenshot of the page plus screenshots of what it looks like after each interaction [xiao2025benchmarking, p. 1]. The authors built a test set, measured how well models do, sorted their mistakes into types, and tried prompting tricks to help. It matters for us because it is a web benchmark where the AI must infer behaviour from what it observes, which is close to reverse engineering.

## How they did it

1. They took 100 pages from the C4 web collection, filtered as in Design2Code, and 27 pages from popular GitHub projects. PhD students picked pages with at least one meaningful interaction [xiao2025benchmarking, p. 2].
2. For each page they marked 1 to 10 important interactions and took screenshots before and after each one [xiao2025benchmarking, p. 3].
3. The model got these screenshots and had to write one HTML+CSS+JavaScript file. It had to give interactive elements fixed IDs (for example id="interact1") so a test robot could find them [xiao2025benchmarking, p. 3].

4. A Selenium webdriver (a tool that controls a browser automatically) clicked each element, took a screenshot and cut out the part that changed [xiao2025benchmarking, p. 4].
5. The changed part was compared with the original using visual similarity, structural similarity, text and position scores. Implement Rate counts whether the robot could find and trigger the element at all [xiao2025benchmarking, p. 4].
6. Four PhD students with front-end experience judged whether each interaction was "usable", meaning it matched the original behaviour [xiao2025benchmarking, p. 6].
7. They compared four prompts [xiao2025benchmarking, p. 5]. Direct is a plain request. Chain-of-Thought asks the model to reason step by step. Mark draws red boxes around interactive areas. A failure-aware prompt lists common mistakes.

## Data and setup

- 127 webpages with 374 interactions [xiao2025benchmarking, p. 1].
- Most interactive elements are buttons (235), then inputs (52) [xiao2025benchmarking, p. 3].
- Six models: Gemini-1.5-flash, GPT-4o, Claude-3.5-Sonnet and Qwen2.5-VL in three sizes [xiao2025benchmarking, p. 4].
- Temperature 0 (the most predictable output), so one run per setting [xiao2025benchmarking, p. 4].
- Contamination check: the generated code shares little text with the original HTML (BLEU, a text-overlap score, 0.12 to 0.14) [xiao2025benchmarking, p. 5].
- Some extra experiments use only 10 pages, and the user study has 4 participants [xiao2025benchmarking, p. 11].

## Key findings

- **[V]** Interactive parts score worse than full pages for all models, for example GPT-4o CLIP 0.7059 for the full page vs 0.5598 for the interaction part [xiao2025benchmarking, p. 6].
- **[V]** Claude-3.5-Sonnet has the best averages on most metrics. GPT-4o with the Mark prompt has the highest Implement Rate (0.8128) [xiao2025benchmarking, p. 6].
- **[V]** Human usability rates: GPT-4o 73%, 74% and 81% (Direct, Chain-of-Thought, Mark); Claude 66% to 77%; Qwen-3B only 17% to 19% [xiao2025benchmarking, p. 6].
- **[V]** The Mark prompt gives the largest gain, for example Claude +11 points over Direct [xiao2025benchmarking, p. 6].
- **[V]** Ten failure types were found. A missing element, no interaction and the wrong function each leave usability below 10% [xiao2025benchmarking, p. 8].
- **[V]** Small, hard-to-see interaction areas cause errors. Enlarging them by cropping helped on 10 failure pages [xiao2025benchmarking, p. 10].
- **[V]** Combining all improvements raised GPT-4o's interaction CLIP from 0.5700 to 0.6937 [xiao2025benchmarking, p. 11].
- **[V]** The failure-aware prompt did not help all models: the small Qwen model dropped on 7 of 8 metrics [xiao2025benchmarking, p. 9]. **[A]** The authors claim it helps "all models" [xiao2025benchmarking, p. 8].

## Limitations and weaknesses

- **[A]** Interactions that need a server, such as login or search, cannot be tested, because there is no back-end code [xiao2025benchmarking, p. 11].
- **[A]** Long pages with many interactions may not fit the model's context window [xiao2025benchmarking, p. 11].
- **[A]** Existing widget-matching tools were built for mobile apps and miss web elements [xiao2025benchmarking, p. 5].
- **[I]** The authors say human results are "consistent" with the automatic metrics, but give no correlation, rater agreement or item counts [xiao2025benchmarking, p. 6].
- **[I]** Implement Rate only checks that an element exists and can be triggered, not that it behaves correctly [xiao2025benchmarking, p. 4].
- **[I]** Behaviour is judged from screenshots only, so effects on data, network or storage are invisible [xiao2025benchmarking, p. 4].
- **[I]** A low BLEU score does not rule out a memorised layout [xiao2025benchmarking, p. 5].
- **[I]** Generated JavaScript is never checked for code quality [xiao2025benchmarking, p. 5].

## Gaps and future work this paper leaves open

- G1: no code quality or maintainability measure; code is compared with the original only to check contamination [xiao2025benchmarking, p. 5].
- G5: it is one of the few papers with an explicit contamination check [xiao2025benchmarking, p. 5].
- G7: **[I]** tests find elements by IDs the model was told to add, which makes them depend on the prompt's naming scheme [xiao2025benchmarking, p. 3].
- G8: back-end-dependent interactions are left as future work [xiao2025benchmarking, p. 11].
- **[A]** The authors also propose multi-round generation for pages with many interactions [xiao2025benchmarking, p. 11].

## Why it matters for your research

**[I]** Interaction2Code moves from "does it look right" to "does it react right", which is the next step towards rebuilding real apps. Its ten failure types are a ready-made checklist for coding errors in your own rebuilds. Its human Usability Rate is a simple functional-fidelity measure you could adopt, but report rater agreement and item counts, which this paper does not. Its contamination check, BLEU against the original code, is cheap and worth copying.

## Connections to other papers

- Builds on si2025benchmarking: it filters pages using the Design2Code approach [xiao2025benchmarking, p. 2] and adds interactions.
- **[A]** chen2025iwr's authors say Interaction2Code treats interactions as single-step, stateless events, and they extend it to recorded videos [chen2025iwr, p. 2].
- Agrees that interaction is harder than static content [chen2025iwr, p. 16; bai2026recreationworld, p. 16; kim2026programdistill, p. 38].

- **[I]** The "looks right, works worse" pattern is not general: in bai2026recreationworld the visual score is below the functional score for 9 of 10 models [bai2026recreationworld, p. 11].

# DeclarUI: Bridging Design and Development with Automated Declarative UI Code Generation

> **Takeaway:** Giving the AI a map of how screens connect, plus compiler feedback, turns screenshots of multi-page mobile apps into code that compiles and navigates far better than a plain AI model. But the navigation score is partly circular, and "maintainability" is judged only by five developers' ratings.

| Item | Detail |
|---|---|
| Authors | Ting Zhou, Yanjie Zhao, Xinyi Hou, Xiaoyu Sun, Kai Chen, Haoyu Wang |
| Year · Venue | 2025 · Proceedings of the ACM on Software Engineering (FSE 2025) |
| Status | Peer-reviewed. We read the arXiv v1 preprint (September 2024); numbers in the published version may differ |
| Identifier | DOI 10.1145/3715726 · arXiv 2409.11667 |
| Paper type | Tool proposal plus controlled comparison, ablation and developer user study |
| Relevance to us | High — RQ1 to RQ5; one of few studies that rebuilds existing mobile apps from screenshots |

## In plain words

Turning a designer's mock-up or an existing app's screens into working code is slow, manual work. The authors built DeclarUI, a tool that does this automatically for multi-page mobile apps [zhou2025declarui, p. 1]. It writes declarative UI code: code that describes what each screen should contain (in React Native, Flutter or ArkUI) rather than how to draw it step by step. The key idea is that plain AI models can copy single screens but forget how screens link together, so DeclarUI first builds a map of the links and then checks the code against it [zhou2025declarui, p. 4]. For us, it is a direct example of reverse engineering from an app's visible UI, with both fidelity and code-quality measures.

## How they did it

1. Build a Page Transition Graph (PTG): Claude-3.5 reads the screens and lists the pages and the actions that move between them. Designers or developers then review and correct this graph [zhou2025declarui, p. 4].
2. Find components: computer-vision tools (Grounding DINO to detect and Segment Anything to cut out elements in an image) locate buttons, lists and so on, and Claude-3.5 describes what each does [zhou2025declarui, p. 4].
3. Write a prompt that combines the graph, the component list and the full screenshot [zhou2025declarui, p. 5].

4. Refine: a static check (reading the code without running it) compares implemented navigation with the graph, up to 3 times per page; compiler errors are fed back, up to 3 times per project [zhou2025declarui, p. 5].
5. Build and run each app on an emulator (a simulated phone) without manual fixes [zhou2025declarui, p. 8].
6. Explore the built app with DroidBot, an automatic app-testing tool that records which screens lead where [zhou2025declarui, p. 7].
7. Compare with GPT-4o and Claude-3.5 given only the raw images and a carefully designed prompt [zhou2025declarui, p. 7].
8. Add an ablation (removing parts of the tool to see what each part adds), a developer user study and a manual repair timing study [zhou2025declarui, p. 6].

## Data and setup

- 50 apps with 5 linked pages each (250 pages): 25 Figma design mock-ups and 25 Google Play apps, in 10 categories [zhou2025declarui, p. 6].
- Google Play apps were among the top 10 by downloads in each category; screenshots were taken by hand on an emulator [zhou2025declarui, p. 7].
- Measures [zhou2025declarui, p. 7]: CLIP score (how similar two images are in meaning), SSIM (similarity of layout and structure), PTG Coverage Rate (PCR, the share of planned screen links that exist in the built app, counted by hand), compile success rate, and manual repair time.
- Developer ratings of code availability, modification time, readability and maintainability on 1–5 scales [zhou2025declarui, p. 9].
- Ablation on 13 apps; user study with 5 React Native developers on 12 apps [zhou2025declarui, p. 9].
- No link to code or data was found in our copy; sampling settings are not reported.

## Key findings

- **[V]** React Native, DeclarUI with Claude-3.5: CLIP 0.88, SSIM 0.68, navigation coverage 96.8%, compile success 98%. Plain Claude-3.5: 0.65, 0.41, 43.3%, 74%. Plain GPT-4o: 0.66, 0.44, 1.4%, 76% [zhou2025declarui, p. 9].
- **[V]** Plain models largely fail to implement navigation between pages (GPT-4o 1.4%) [zhou2025declarui, p. 9]. **[A]** Their compile errors come from non-existent packages and missing imports [zhou2025declarui, p. 8].
- **[V]** It also works for other frameworks: Flutter 99.4% coverage and 92% compile success; ArkUI 95.9% and 86% [zhou2025declarui, p. 9].
- **[V]** Removing the graph and refinement drops coverage to 23.1%; removing the vision step drops SSIM from 0.72 to 0.57 [zhou2025declarui, p. 10].
- **[V]** Developers rated DeclarUI code higher than the baseline: readability 4.62 vs 3.75 and maintainability 4.55 vs 3.37, all $p < 0.05$ [zhou2025declarui, p. 10].
- **[V]** 73.3% of DeclarUI code was modified within 10 minutes, while 80% of baseline code needed more than 10 minutes [zhou2025declarui, p. 10].

- **[V]** Manual repair of failed builds took on average 48 s for React Native and 183 s for ArkUI [zhou2025declarui, p. 11].
- **[A]** The authors conclude that the generated code meets industrial-grade standards [zhou2025declarui, p. 1]; this rests on ratings from five developers.

## Limitations and weaknesses

- **[A]** CLIP and SSIM may not match human judgement of similarity; coverage does not reflect how smooth navigation feels [zhou2025declarui, p. 12].
- **[A]** 50 apps may not represent all mobile UIs; sample size was limited by AI token cost [zhou2025declarui, p. 12].
- **[I]** Humans correct the graph [zhou2025declarui, p. 4], yet the main question is framed as working "without human intervention" [zhou2025declarui, p. 6]. How much was corrected is not reported.
- **[I]** The navigation score is partly circular: DeclarUI gets the graph, is checked against it, and is then scored against the same graph; the baselines never see it [zhou2025declarui, p. 7].
- **[I]** Some ablation numbers are impossible for 13 apps (98% and 80% cannot be k out of 13), which suggests values were copied from the 50-app run [zhou2025declarui, p. 10].
- **[I]** The user-study tests treat 60 ratings as independent, so the very small p-values overstate certainty [zhou2025declarui, p. 10].
- **[I]** Maintainability is only rated, never measured with code-analysis tools. One run per setup; no back-end or business logic [zhou2025declarui, p. 2].

## Gaps and future work this paper leaves open

- **[A]** The authors plan better knowledge retrieval for rare frameworks such as ArkUI [zhou2025declarui, p. 11] and larger, more varied samples [zhou2025declarui, p. 12].
- **[I]** G1: maintainability rests on Likert ratings, not validated measures compared with the original's code.
- **[I]** G4: the feedback loop improves exactly what it checks (compilation, navigation links); its effect on code quality is untested.
- **[I]** G7: structural hints given as input, and scoring against the same hints, may inflate fidelity.
- **[I]** G8: UI only, with no back-end behaviour [zhou2025declarui, p. 12].

## Why it matters for your research

**[I]** DeclarUI shows that structuring what you observe (a screen map) and feeding compiler errors back makes rebuilds much more complete. You can reuse its idea of exploring the built app automatically and comparing the resulting screen map with the original's. Avoid its weak points: do not score a rebuild against the same hints you gave it, measure maintainability with tools as well as opinions, and analyse repeated ratings with proper statistics.

## Connections to other papers

- **[V]** wu2026looks studies the same task with automatic navigation tests [wu2026looks, p. 5] and also gives the model a page-connection description [wu2026looks, p. 3]. **[I]** Both rebuild with a structural hint, not from screenshots alone.
- **[V]** Its loop result agrees with wu2026looks, where a build-repair loop gives 100% build success [wu2026looks, p. 6]. It contrasts with pan2026functional, where repair strategies lowered code quality [pan2026functional, p. 8].
- **[V]** borg2026echoes measures maintainability by having a second developer evolve the code [borg2026echoes, p. 13], a stronger test than ratings.
- **[V]** bai2026recreationworld is the only paper here that rebuilds native Android apps from a running reference [bai2026recreationworld, p. 7].

# Looks Right, Works Right: A Project-Level Benchmark for Multi-Screen Mobile App Generation

> **Takeaway:** When frontier AI models rebuild whole multi-screen mobile apps from screenshots of real apps, the code always builds, navigation works in 58–92% of tests, and visual fidelity stays at or below 2.99 out of 5. The model whose app looks best is not the one whose code is most maintainable, so one score cannot stand for all.

| Item | Detail |
|---|---|
| Authors | Fan Wu, Cuiyun Gao, Yiming Huang, Yang Xiao, Yujia Chen, Qing Liao |
| Year · Venue | 2026 · Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing (EMNLP 2026) |
| Status | Peer-reviewed. Very recent and not yet replicated |
| Identifier | DOI 10.18653/2026.emnlp-main.2650 · arXiv 2607.28645 |
| Paper type | Benchmark (MobileForge) plus five-axis empirical evaluation of 6 multimodal models |
| Relevance to us | High — RQ1 to RQ5; the closest existing design to our target study |

## In plain words

The paper opens with a direct question: "Can a multimodal LLM read a folder of mobile-app screenshots and generate the source code for that app?" [wu2026looks, p. 1]. A multimodal LLM is an AI model that reads images as well as text. Earlier benchmarks mostly tested single pages, did not test whether buttons lead to the right screen, and did not look at the code as a whole project [wu2026looks, p. 1]. MobileForge fixes this by scoring five things at once: does it build, does navigation work, does it look like the original, is the code maintainable, and what does it cost [wu2026looks, p. 4]. For us, this is the nearest thing to a reconstruction study that measures fidelity and code quality together.

## How they did it

1. Collect apps: 29 real consumer apps from official app stores. Screenshots were taken by hand from default or demo screens without logging in [wu2026looks, p. 3].
2. Annotate: a vision-language model (an AI that reads images and text) drafted a description of how pages connect, plus navigation tests. The research team reviewed every field [wu2026looks, p. 3].
3. Build: one AI agent received all screenshots plus the page-connection description and had to produce a React + TypeScript + Tailwind web project. It had 8 tools (write file, run command and so on) and up to 50 steps. When it finished, an automatic build check (tsc + vite) sent errors back for repair [wu2026looks, p. 6].

4. Test navigation: Playwright, a tool that drives a browser automatically, ran each test from its own starting page, did one action, and checked whether the right page appeared. Starting fresh each time stops one failure from causing many others [wu2026looks, p. 5].
5. Test looks: a judge model (Gemini 2.5 Pro) ranked the six models' versions of a screen, with the real screenshot hidden among them as an "anchor". Rankings that did not put the real screenshot first were thrown away and redone. Ranks were turned into a Borda score between 0 and 1, a simple points-for-rank system [wu2026looks, p. 5]. A 1–5 rubric score was also kept [wu2026looks, p. 6].
6. Check code: simple indicators such as lines of code (LoC), lines per file, how often shared components are reused, the share of unused ("dead") components, and colour consistency [wu2026looks, p. 4].
7. Check the judge: on 55 scenarios, its rankings were compared with human rankings [wu2026looks, p. 13].

## Data and setup

- 29 apps in 14 categories, 309 screens (5–21 per app), 701 navigation tests [wu2026looks, p. 3].
- Apps include Notion, Khan Academy, WhatsApp, Airbnb and Coinbase [wu2026looks, p. 12].
- Six closed models: Claude Opus 4.6, Claude Haiku 4.5, GPT-5, GPT-5 Mini, Gemini 2.5 Pro, Gemini 2.5 Flash [wu2026looks, p. 6].
- 29 apps × 6 models = 174 runs, one run each [wu2026looks, p. 6].
- 64 hand-labelled navigation failures for a failure analysis [wu2026looks, p. 8].
- Screenshots, annotations, harness and all run outputs are released (author claim) [wu2026looks, p. 10].

## Key findings

- **[V]** Every model builds successfully in 100% of runs [wu2026looks, p. 6]. **[A]** "Build success is a property of the harness, not the model" [wu2026looks, p. 12].
- **[V]** Navigation pass rate: Opus 4.6 92%, GPT-5 89%, Haiku 4.5 85%, Gemini 2.5 Pro 82%, Gemini 2.5 Flash 62%, GPT-5 Mini 58% [wu2026looks, p. 7].
- **[V]** Visual fidelity is far from solved: rubric means range from 1.60 to 2.99 out of 5, and Borda scores from 0.130 to 0.906. Opus leads in 26 of 29 apps [wu2026looks, p. 6].
- **[V]** Looks and code quality diverge: Opus writes 1,441 LoC (82 per file) vs 668 (33 per file) for GPT-5, and GPT-5 has fewer dead components (7.8% vs 19.9%) [wu2026looks, p. 7]. **[A]** The authors call maintainability "uncorrelated" with visual fidelity [wu2026looks, p. 8].
- **[V]** Cost ranges from $0.15 to $2.90 per app; GPT-5 ($0.50) beats Haiku 4.5 and Gemini 2.5 Pro on both cost and quality [wu2026looks, p. 7].
- **[V]** The biggest navigation failure is "click has no effect" (48.4%), then target unreachable (32.8%), wrong route (10.9%) and blank page (7.8%), from 64 labelled cases [wu2026looks, p. 8]. **[A]** Build checks miss these silent bugs [wu2026looks, p. 8].
- **[V]** The judge agrees reasonably with human rankings (Kendall $\tau = 0.716$, a rank-agreement score where 1 is perfect; top-1 match 83.6%), and a second judge gives $\tau = 0.867$ [wu2026looks, p. 13].

## Limitations and weaknesses

- **[A]** One run per app and model; the judge comes from a competing model family; output is a web app standing in for a native app [wu2026looks, p. 9].
- **[A]** Per-model failure shares rest on 8–12 cases each [wu2026looks, p. 8].
- **[I]** The input is not screenshots alone: the agent also gets the human-reviewed page-connection description [wu2026looks, p. 6]. Results describe rebuilding with a navigation map, not pure black-box observation.
- **[I]** The code indicators are not validated against developer judgement or standard metrics. More LoC may simply mean more content was copied.
- **[I]** "Uncorrelated" rests on six model averages; no correlation was computed.
- **[I]** The 55 human labels were made by the authors themselves [wu2026looks, p. 13].
- **[I]** Two failure tables do not reconcile [wu2026looks, p. 16], and overlays count as passing when the page content "changes meaningfully", which is lenient [wu2026looks, p. 5].
- **[I]** Famous apps were chosen on purpose [wu2026looks, p. 3], so models may have seen them in training; this is not discussed. Only page changes are tested, not data or back-end logic.

## Gaps and future work this paper leaves open

- **[A]** The authors call for repeated runs, native app versions [wu2026looks, p. 9] and stricter build checks [wu2026looks, p. 12].
- **[I]** G1: maintainability here is unvalidated proxies [wu2026looks, p. 4], not a validated measure against the original's code.
- **[I]** G7: a structural hint is given as input [wu2026looks, p. 3]; its effect on scores is unknown.
- **[I]** G8: native mobile apps with hidden back-ends are not covered [wu2026looks, p. 9].
- **[I]** G4: the build-repair loop saturates build success but says nothing about behaviour or quality. G5: memorisation of famous apps is unchecked.

## Why it matters for your research

**[I]** This paper is the closest model for your own study. Reuse its ideas: state-isolated navigation tests, a visual judge with a hidden real screenshot, and reporting several scores side by side. Its main message supports your topic: looks, behaviour and code quality can disagree, so they must be measured separately. Improve on it by running each build several times, testing behaviour beyond navigation, using validated quality tools, and comparing with the original's code.

## Connections to other papers

- **[V]** Very close to zhou2025declarui, which also rebuilds multi-screen mobile UIs from screenshots and also feeds a screen-connection graph to the model [zhou2025declarui, p. 4]. **[I]** MobileForge replaces manual navigation counts with automatic tests.
- **[V]** Agrees with pan2026functional that function and quality diverge: equal functional scores, very different non-functional scores [pan2026functional, p. 5].

- **[V]** Agrees that interaction is the hard part: bai2026recreationworld finds static structure easier than actions [bai2026recreationworld, p. 16].
- **[V]** Contrasts with chen2025iwr, where visual scores exceed interaction scores [chen2025iwr, p. 8]; here navigation scores well while visuals stay low. **[I]** The scales are not comparable across papers.
- **[V]** Rebuilds in bai2026recreationworld are much smaller than the originals [bai2026recreationworld, p. 15], while the visual leader here writes more code. **[I]** Code size means different things in different settings.
- **[V]** Like tran2026vibe, it validates its AI judge against humans [tran2026vibe, p. 9].

# IWR-Bench: Can LVLMs reconstruct interactive webpage from a user interaction video?

> **Takeaway:** When AI models watch a video of someone using a website and try to rebuild it, even the best model (GPT-5) scores only 36.35 out of 100, and no model gets more than 25 on working interactions [chen2025iwr, p. 8]. Making things work is much harder than making them look right, although part of this gap comes from how the scores are built [chen2025iwr, p. 7].

| Item | Detail |
|---|---|
| Authors | Yang Chen, Minghao Liu, Yufan Shen, Yunwen Li, Tianyuan Huang, Xinyu Fang, et al. (21 authors) |
| Year · Venue | 2025 · arXiv |
| Status | Preprint (not peer-reviewed) |
| Identifier | DOI 10.48550/arxiv.2509.24709 (arXiv 2509.24709) |
| Paper type | Benchmark with a large model comparison |
| Relevance to us | High — RQ1, RQ2, RQ4, RQ5 (indirectly) |

## In plain words

Earlier tests asked AI to turn a single screenshot of a webpage into code. Real websites, however, react when you click, type or scroll. This paper asks whether AI models that understand images and video can rebuild a webpage, including its behaviour, just by watching a screen recording of a person using it [chen2025iwr, p. 2]. This is a form of reverse engineering from observed behaviour, not from a written description. It matters for us because it shows how far current models are from copying what an app does, not only what it looks like.

## How they did it

1. Trained annotators recorded videos while using 100 real websites. A browser extension logged every action, such as clicks and typing [chen2025iwr, p. 5].
2. They also collected the site's images and icons. The file names were replaced (for example asset_001.png) so the model had to match pictures by how they look [chen2025iwr, p. 5].
3. Each model got the video (one frame per second, at most 64 frames) plus one image showing all assets. It had to write a single index.html file with all HTML, CSS and JavaScript, with no outside libraries and any server behaviour faked in JavaScript [chen2025iwr, p. 8].
4. An "agent-as-a-judge" (an AI agent that operates the generated page) replayed the recorded actions on the new page [chen2025iwr, p. 6]. Gemini-2.5-Pro judged whether each expected result happened [chen2025iwr, p. 7].

5. Two scores were combined [chen2025iwr, p. 7]. IFS (Interactive Functionality Score) is the share of actions that work. VFS (Visual Fidelity Score) measures how similar the reached screens look. The Final Score is 0.7 x IFS + 0.3 x VFS.
6. The weights were tuned so the score matched human ratings as closely as possible [chen2025iwr, p. 17].

## Data and setup

- 113 tasks from 100 websites, cut down from 200 candidates [chen2025iwr, p. 5].
- 1,001 recorded actions (8.9 per video on average), with 620 visual checkpoints and 403 logic checks [chen2025iwr, p. 6].
- Task mix: 7.96% static, 47.79% simple state, 33.63% complex workflow, 10.62% algorithm or game [chen2025iwr, p. 3].
- 28 model entries: 17 commercial, 9 open-source and 2 video-specialised models. Three models were also run in a "thinking" mode [chen2025iwr, p. 8].
- Human checks: five PhD-level raters scored 60 outputs [chen2025iwr, p. 17], and three PhD students watched the judge agent on 100 pages [chen2025iwr, p. 9].
- Runs per model and random seeds are not reported [chen2025iwr, p. 8].

## Key findings

- **[V]** GPT-5 is best with Final 36.35, IFS 24.39 and VFS 64.25. No model exceeds 25 IFS [chen2025iwr, p. 8].
- **[V]** IFS is below VFS for every model, for example Claude-Sonnet-4 22.29 vs 61.34 [chen2025iwr, p. 8]. **[I]** Part of this gap comes from the metric design: VFS only counts screens that were reached, while IFS counts every action [chen2025iwr, p. 7].
- **[V]** The top eight models are all commercial. The best open-source model, Qwen3-VL (thinking), scores 31.15 and beats 9 of the 17 commercial models. Video-specialised models are lowest (13.67 and 10.07) [chen2025iwr, p. 8].
- **[V]** "Thinking" modes raise the Final Score only slightly (for example 34.62 vs 34.00) and lower VFS in all three pairs [chen2025iwr, p. 8]. **[A]** The authors still call the gains "Consistent but Moderate" [chen2025iwr, p. 9].
- **[V]** Scores drop sharply from static to interactive tasks: GPT-5 gets 61.85 on static pages but 25.26 on algorithm or game tasks [chen2025iwr, p. 17].
- **[V]** The judge agent failed in only 3 of 100 observed runs, and these errors were fixed [chen2025iwr, p. 9].
- **[V]** Score weights matched human ratings only moderately: Spearman correlation 0.57 for VFS and 0.65 for the Final Score [chen2025iwr, p. 17].

## Limitations and weaknesses

- **[I]** The paper has no limitations section. The authors only mention design notes, such as fixing the three judge errors [chen2025iwr, p. 9].
- **[I]** VFS and IFS use different denominators, so "visual is much better than function" is partly built in [chen2025iwr, p. 7].

- **[I]** Code quality is never measured. The single-file output is far from how real apps are built [chen2025iwr, p. 20].
- **[I]** Gemini-2.5-Pro is both a judge and an evaluated model, and self-preference was not tested [chen2025iwr, p. 7].
- **[I]** The weights were tuned and tested on the same 60 human-rated items, with no held-out check [chen2025iwr, p. 17].
- **[I]** No repeated runs, confidence intervals or significance tests, so small differences such as 30.36 vs 30.31 may be noise [chen2025iwr, p. 9].
- **[I]** Websites like IMDb may be in training data, and the authors do not discuss this [chen2025iwr, p. 2].
- **[I]** The static and algorithm groups hold only about 9 and 12 tasks [chen2025iwr, p. 3].

## Gaps and future work this paper leaves open

- G1: behaviour and looks are scored, code quality is not [chen2025iwr, p. 7].
- G2: the judge is calibrated in-sample and is also an evaluated model [chen2025iwr, p. 17].
- G5: **[I]** no contamination discussion was found.
- **[A]** The authors point to temporal reasoning, linking assets to the right places, and better state management as next steps [chen2025iwr, p. 9].

## Why it matters for your research

**[I]** IWR-Bench shows that rebuilding interactive behaviour from observation is hard for every model tested. Its action-replay judge and its split into "works" and "looks right" scores are useful ideas for your own study. Learn from its weaknesses too: give both scores the same denominator, use a judge that is not also tested, run each model more than once, and add code-quality measures. Treat its numbers as weak evidence, because this is a single preprint.

## Connections to other papers

- Builds on si2025benchmarking and xiao2025benchmarking. **[A]** The authors describe Design2Code as static and Interaction2Code as single-step and stateless [chen2025iwr, p. 2].
- Agrees with xiao2025benchmarking: interactive parts score lower than full pages for all six models there [xiao2025benchmarking, p. 6].
- Only directionally consistent with bai2026recreationworld. Static structure beats interaction there as well [bai2026recreationworld, p. 16], but its visual score is below its functional score for 9 of 10 models [bai2026recreationworld, p. 11].
- Agrees with the agent studies that whole-app fidelity is low [kim2026programdistill, p. 11; yang2026programbench, p. 6].
- **[I]** Do not rank 36.35 against other benchmarks' numbers, such as a 58.06% average [bai2026recreationworld, p. 11] or 49.15% of workflows [kim2026programdistill, p. 11]. Each uses a different scoring method.

# RecreationWorld: Scalable and Verifiable Environments for Hybrid Computer-Use Agents

> **Takeaway:** When AI agents rebuild real apps by using a running original, even the best agent averages only 58.06% and passes every functional test on just 2.8% of apps [bai2026recreationworld, p. 11]. The rebuilds are also much smaller and more concentrated in a few files than the originals [bai2026recreationworld, p. 15]. This is the closest study to our topic, but it does not measure code quality.

| Item | Detail |
|---|---|
| Authors | Shuai Bai, Jiayong Deng, Sicheng Fan, Yikun Fu, Chang Gao, Xuhao Hu, et al. (33 authors, Alibaba Group) |
| Year · Venue | 2026 · arXiv |
| Status | Preprint (not peer-reviewed); industry paper |
| Identifier | DOI 10.48550/arxiv.2609.22000 (arXiv 2609.22000) |
| Paper type | Benchmark and environment framework, with a model comparison and a training (transfer) experiment |
| Relevance to us | High — RQ1, RQ2, RQ3 (partly), RQ4, RQ5 |

## In plain words

Imagine giving an AI a working app, but not its source code, and asking it to build a copy that behaves the same way. That is "application recreation", which is very close to AI-assisted reverse engineering. The authors built a large test set for it on five platforms: Ubuntu, macOS, Windows, Android and the Web [bai2026recreationworld, p. 7]. They tested ten of the newest AI models and studied how the agents worked and what code they produced. They also checked whether practising this task helps on other coding tasks [bai2026recreationworld, p. 3].

## How they did it

1. They collected 250 reference apps. Desktop and Android apps came from open-source projects. The Web set has 44 sites the authors made themselves and 6 real, openly licensed sites [bai2026recreationworld, p. 7].
2. Each agent got a task prompt, live access to the running original, and a computer with GUI control (the ability to look at and click an interface) and developer tools. It had to hand back source code that builds and launches [bai2026recreationworld, p. 7]. On desktop and Android the agent could not see the original's source code. On the Web, the page code sent to the browser is necessarily visible [bai2026recreationworld, p. 7]. Each attempt had up to 20 hours [bai2026recreationworld, p. 4].

3. Hidden tests were written by AI agents, replayed on a clean copy of the original, cleaned of unstable cases, and reviewed by humans before being frozen [bai2026recreationworld, p. 8].
4. Two scores were computed [bai2026recreationworld, p. 9]. The Programmatic score is the share of tests passed; the tests read text, buttons and values through the accessibility interface (the system layer that screen readers use). The VLM score uses a vision-language model (an AI that reads images) as a judge, which checks screenshots against statements about the original. An app that does not build scores zero.
5. They then analysed agent behaviour and code size for four of the models [bai2026recreationworld, p. 12].

## Data and setup

- 250 tasks, 50 per platform [bai2026recreationworld, p. 7].
- Median original size: 49.3k lines of code on Ubuntu, 13.5k on macOS, 6.7k on Windows, 15.0k on Android [bai2026recreationworld, p. 8].
- 10 models. GPT models ran in the Codex tool and all others in Claude Code [bai2026recreationworld, p. 28].
- Network access was blocked by default, and reference and test files were protected [bai2026recreationworld, p. 9].
- Runs per task are not reported. **[I]** Our reviewers think it was probably one run, because one single-run comparison reproduces a main-table score exactly [bai2026recreationworld, p. 11].

## Key findings

- **[V]** GPT-6 Astra leads with an average of 58.06 (Programmatic 58.19, VLM 57.92). Next are Claude Opus 5 (44.16) and GPT-5.6 Sol (42.06); Qwen3.7-Plus is lowest (9.15) [bai2026recreationworld, p. 11].
- **[V]** Full fidelity is rare. The best model passes at least 90% of tests on 17.6% of apps and all tests on 2.8%; every other model reaches at most 5.53% and 0.8% [bai2026recreationworld, p. 11].
- **[V]** Agents copy what an interface contains better than what it does. Structure tests had the highest pass rate in 15 of 16 native comparisons, and the weakest category trailed by 9.1 to 34.4 points [bai2026recreationworld, p. 16].
- **[V]** Rebuilds are smaller: 89.4% have less code than the original (median 16.9% of its size), and 92.3% use fewer files [bai2026recreationworld, p. 15].
- **[V]** More code did not mean better fidelity. Among the four analysed models, Claude Opus 5 wrote the most code, while the top model GPT-6 Astra wrote the least on four of five platforms [bai2026recreationworld, p. 15].
- **[V]** Agents often switched to a different UI framework, for example rebuilding 75% of Electron, Tauri or Wails desktop apps natively. **[A]** The authors call this "architectural substitution rather than implementation quality" [bai2026recreationworld, p. 16].

- **[V]** Training two models on recreation runs left both above their starting checkpoint on five other benchmarks, by up to 17.9 points [bai2026recreationworld, p. 3]. **[A]** The authors call this "exploratory" evidence [bai2026recreationworld, p. 29].
- **[V]** Agents rarely checked their final work: the full "edit, relaunch, inspect" loop appears in only 23.6% to 47.5% of runs [bai2026recreationworld, p. 12].

## Limitations and weaknesses

- **[A]** A finite test suite "cannot establish equivalence across all states and interaction paths" [bai2026recreationworld, p. 22].
- **[A]** Public originals may be in the models' training data [bai2026recreationworld, p. 22].
- **[A]** The behaviour and code analyses are descriptive and do not show cause and effect [bai2026recreationworld, p. 3].
- **[I]** No confidence intervals and probably one run per task, so small gaps between models may not be stable [bai2026recreationworld, p. 11]. Model and tool are also mixed up: GPT ran in Codex, the rest in Claude Code [bai2026recreationworld, p. 28].
- **[I]** All authors are from Alibaba, and Qwen models serve as judge and as training-data generator. No conflict-of-interest statement is given [bai2026recreationworld, p. 5]. The Qwen judge model scored lowest, so no favourable self-scoring is visible [bai2026recreationworld, p. 11].
- **[I]** One Web site that replayed copied content escaped the cheating detector and kept a 54.82% Programmatic score [bai2026recreationworld, p. 47].

## Gaps and future work this paper leaves open

- G1: code is described only by size and structure, never by maintainability [bai2026recreationworld, p. 15].
- G2: the VLM judge is never checked against human ratings, and self-preference is not tested [bai2026recreationworld, p. 9].
- G4: agents' weak self-checking [bai2026recreationworld, p. 12] raises the question of whether better feedback loops would help.
- G5: copying looks rare (median exact-line overlap 0.014% to 0.088%) [bai2026recreationworld, p. 18], but the authors still suggest newly built originals to reduce training-data risk [bai2026recreationworld, p. 22].
- G6: **[I]** there is no human-developer or human-steered condition.
- G8: it is the only project paper with native Android rebuilds [bai2026recreationworld, p. 7], but apps needing login or remote services were excluded [bai2026recreationworld, p. 8].

## Why it matters for your research

**[I]** This is the strongest evidence that rebuilding apps from observation works only partly. Reuse its design ideas: tests checked against the original, two scoring channels and network isolation. Its main gap is your opportunity: rebuilds are smaller and more monolithic, but nobody knows whether they are harder to maintain. Avoid its weak points: run each task several times, validate any AI judge against humans, and keep model and tool separate.

## Connections to other papers

- Agrees with yang2026programbench: rebuilds are shorter and use fewer files than the originals [yang2026programbench, p. 9], and full equivalence is rare there too (0 of 200 programs) [yang2026programbench, p. 6].
- Builds on ProgramBench: it is one of the benchmarks used to test training transfer [bai2026recreationworld, p. 6].
- Agrees with kim2026programdistill: agents rebuilding web apps from a live original recover only 49.15% of workflows [kim2026programdistill, p. 11], and their self-checks miss errors [kim2026programdistill, p. 12].
- Only partly agrees with chen2025iwr. Both find interaction harder than static structure [chen2025iwr, p. 16]. But here the visual score is below the programmatic score for 9 of 10 models [bai2026recreationworld, p. 11], so "looks right but behaves wrong" is not supported in general.

# ProgramDistill: From Interactive Web Apps to Verifiable Reference-Guided SWE Tasks

> **Takeaway:** When coding agents rebuild a whole web app by clicking through a live original, the best one recovers only 49.15% of multi-step workflows [kim2026programdistill, p. 11]. Most failures are behaviours the agent never looked at in the original, not coding mistakes [kim2026programdistill, p. 12].

| Item | Detail |
|---|---|
| Authors | Jeonghye Kim, Minseon Kim, Young Jin Kim, Matheus Pereira, Marc-Alexandre Côté, Alessandro Sordoni, Xingdi Yuan, Zhengyan Shi |
| Year · Venue | 2026 · arXiv |
| Status | Preprint (not peer-reviewed); authors from KAIST, Microsoft Research Montréal and Microsoft AI |
| Identifier | DOI 10.48550/arxiv.2609.18805 (arXiv 2609.18805) |
| Paper type | Benchmark with an automated task-building pipeline and an agent comparison |
| Relevance to us | High — RQ1, RQ2, RQ4, RQ5 (RQ3 only as a gap) |

## In plain words

Usually a coding AI is told exactly what to build, for example through a bug report. Here the AI must find out what to build by using a working web app, then make an incomplete or empty copy behave the same way [kim2026programdistill, p. 1]. The authors call this a shift "from specification consumption to specification discovery" [kim2026programdistill, p. 14]. They built thousands of such tasks automatically and tested frontier coding agents on them. This matters for us because the "full rebuild" setting is almost exactly our scenario: rebuilding a web app from its interface, without its source code.

## How they did it

The authors use a three-step pipeline they call mine, craft and patch [kim2026programdistill, p. 2].

1. Mine: AI agents use the live app in a browser and record "traces". A trace is a replayable list of actions (clicks, typing) plus the expected visible results, such as text, page address or state. A trace is kept only if it replays the same way every time from a fresh start [kim2026programdistill, p. 3].
2. Craft: an AI deletes the code behind one behaviour. The deletion is kept only if the app still builds, earlier steps still work, and the target trace now fails [kim2026programdistill, p. 4]. Deleting several linked behaviours gives harder tasks. The number of linked behaviours is the "restoration depth" [kim2026programdistill, p. 2].

3. Patch, partial: the agent gets the damaged code, a short description and browser access to the live original, with no original source and no internet [kim2026programdistill, p. 6].
4. Patch, full: the agent starts from a near-empty scaffold and must rebuild the whole app from the live original [kim2026programdistill, p. 6].
5. Scoring replays the recorded traces on the result without any AI judge. A task passes only if every step and expected result works [kim2026programdistill, p. 4]. Code may look completely different from the original, as long as the behaviour replays [kim2026programdistill, p. 6].

## Data and setup

- 26 web apps: 18 adapted from the OSWorld test suite and 8 open-source projects or SaaS clones, such as Trello and Jira clones [kim2026programdistill, p. 19].
- 1,975 verified behaviours and 4,063 tasks [kim2026programdistill, p. 7].
- Partial rebuilds: 300 selected tasks across all 26 apps, with depths 1 to 8, run by 9 models [kim2026programdistill, p. 8].
- Full rebuilds: 12 apps x 3 models = 36 runs, scored with 590 single-behaviour tests and 413 workflow tests [kim2026programdistill, p. 11].
- Agents see only structured text from the browser (visible text, element names and roles), not screenshots [kim2026programdistill, p. 3].
- All tasks were built by one model, GPT-5.6 Sol [kim2026programdistill, p. 2].
- **[I]** Runs per task in the partial setting are not stated, so probably one [kim2026programdistill, p. 8].

## Key findings

- **[V]** Partial rebuilds: GPT-6 Astra passes 84.3% of tasks, Claude Opus 5 68.7% and GPT-5.6 Sol 60.7%. Gemini 3.1 Pro Preview is lowest at 22.3% [kim2026programdistill, p. 8].
- **[V]** Success falls with depth. GPT-6 Astra drops from 100% at depth 1 to 64.0% at depth 8, and Claude Opus 5 from 96% to 32.0% [kim2026programdistill, p. 8]. The drop persists when context length is held similar [kim2026programdistill, p. 39].
- **[V]** Restoring the interface as well as the logic is harder: 92.9% vs 76.9% for GPT-6 Astra [kim2026programdistill, p. 38].
- **[V]** Full rebuilds: GPT-6 Astra recovers 58.98% of single behaviours and 49.15% of workflows, Claude Opus 5 42.03% and 28.81%, and GPT-5.6 Sol 33.39% and 21.07% [kim2026programdistill, p. 11].
- **[V]** Results vary a lot by app: GPT-6 Astra's workflow recovery ranges from 81.8% (MailHub) to 5.9% (Baserow) [kim2026programdistill, p. 11].
- **[V]** Of 977 failed behaviours, 59.2% were never observed in the original, 27.9% gave the wrong state or result, and 11.1% had the wrong visible form [kim2026programdistill, p. 12].
- **[A]** Agents' own checks often missed the failing workflow, so mismatches went unnoticed [kim2026programdistill, p. 12].
- **[V]** 7.2% of runs tried a shortcut, such as fetching public source code. All shortcuts were blocked by design [kim2026programdistill, p. 10].

## Limitations and weaknesses

- **[A]** No screenshots, so visual fidelity is not evaluated [kim2026programdistill, p. 14].
- **[A]** Only self-contained web apps; external services, desktop and mobile are future work [kim2026programdistill, p. 14].
- **[A]** Task quality depends on the model that built the tasks [kim2026programdistill, p. 15].
- **[I]** Code quality is not measured. Full rebuilds must also use one fixed Node entry point, while the originals use Django, Angular, React or Next.js [kim2026programdistill, p. 37].
- **[I]** Possible family bias: GPT-5.6 Sol built the tasks, GPT-6 Astra wins, and GPT-6 Astra also classified the failures [kim2026programdistill, p. 12].
- **[I]** Tasks are built "without human intervention", and no sample was audited by hand [kim2026programdistill, p. 1].
- **[I]** Full-rebuild tests require the original's selectors and labels (element identifiers), which mixes behaviour with copying page structure [kim2026programdistill, p. 35].
- **[I]** The abstract reports full-rebuild results under "nine frontier coding agents" [kim2026programdistill, p. 1], but only 3 were tested in that setting [kim2026programdistill, p. 11].
- **[I]** No variance or confidence intervals are reported [kim2026programdistill, p. 8]. Contamination is not discussed, although the 8 real apps are public projects [kim2026programdistill, p. 19].

## Gaps and future work this paper leaves open

- G1: code quality of rebuilds is not measured [kim2026programdistill, p. 14].
- G2: the builder model and judge are from the same family as the winner [kim2026programdistill, p. 2].
- G4: unreliable self-checks [kim2026programdistill, p. 12] invite a test of better feedback loops.
- G5: **[I]** the 8 real apps are public, but memorisation is not discussed.
- G6: only one text-only autonomous agent setup is tested [kim2026programdistill, p. 14].
- G7: the tests depend on the original's selectors [kim2026programdistill, p. 35].
- G8: desktop and mobile apps are left as future work [kim2026programdistill, p. 14].

## Why it matters for your research

**[I]** This is the closest study to rebuilding interactive web apps from their interface. Reuse two ideas: replaying recorded browser workflows as a test, and the failure categories, which show that exploring the original is the main bottleneck. Avoid two of its choices: tests tied to the original's element identifiers, and a fixed code structure that makes rebuilds impossible to compare with the original's code.

## Connections to other papers

- Builds on yang2026programbench, which the authors cite as the whole-program "behavior-to-code" setting [kim2026programdistill, p. 1].
- Agrees with bai2026recreationworld: whole-app fidelity is partial, and agents rarely re-check their final result [bai2026recreationworld, p. 12].
- Agrees that fidelity falls as programs get more complex [yang2026programbench, p. 31].

- Agrees that interaction is harder than static content [xiao2025benchmarking, p. 6; chen2025iwr, p. 16].
- Shares the same-family pattern with ProgramBench, where Claude wrote the tests and Claude models lead [yang2026programbench, p. 3].

# ProgramBench: Can Language Models Rebuild Programs From Scratch?

> **Takeaway:** Given only a working program and its manual, no AI model fully rebuilt any of 200 real programs, and the best passed at least 95% of the tests on only 3% of them [yang2026programbench, p. 6]. The rebuilt code is much shorter, uses far fewer files and is more monolithic than the human original [yang2026programbench, p. 9].

| Item | Detail |
|---|---|
| Authors | John Yang, Kilian Lieret, Jeffrey Ma, Parth Thakkar, Dmitrii Pedchenko, Sten Sootla, et al. (12 authors; Meta FAIR, Meta TBD, Stanford, Harvard) |
| Year · Venue | 2026 · arXiv |
| Status | Preprint (not peer-reviewed) |
| Identifier | DOI 10.48550/arxiv.2605.03546 (arXiv 2605.03546) |
| Paper type | Benchmark with an evaluation of 9 models, test-quality checks and code-structure analysis |
| Relevance to us | High — RQ1, RQ2, RQ3, RQ4, RQ5 |

## In plain words

Most coding tests for AI ask it to fix one bug or add one feature. This paper asks something much bigger: can an AI build a whole program from scratch, including its overall design, just by running the original and reading its manual [yang2026programbench, p. 1]? The original source code is hidden, so the AI must work out the behaviour by experimenting. This is black-box reverse engineering (rebuilding something only from what it does). It matters for us because it is the clearest test of our core scenario, although the programs are command-line tools, not apps with screens.

## How they did it

1. They took 200 open-source GitHub projects written in compiled languages (Rust, Go, C/C++) that build one program file [yang2026programbench, p. 4].
2. The AI agent got the compiled program and its usage documentation inside a sealed container with no internet [yang2026programbench, p. 4]. The program was execute-only, so it could be run but not opened up and decompiled (turned back into code). Decompiling and tracing tools were forbidden [yang2026programbench, p. 19].
3. The agent could use any language. It had to hand in source code plus a build script that produces a program with the same behaviour [yang2026programbench, p. 2].
4. Hidden tests were written by a separate AI agent (Claude Sonnet 4.5) that could read the original source. It added tests until they covered most of the code

[yang2026programbench, p. 3]. Tests check only what can be observed: printed output, exit codes and files written [yang2026programbench, p. 3].

5. Tests that a deliberately broken "dummy" program could pass were removed or made stricter [yang2026programbench, p. 8].
6. The main score, "% resolved", counts a task only if every test passes [yang2026programbench, p. 5].

## Data and setup

- 200 tasks: 107 Rust, 46 Go, 45 C/C++ and 2 other [yang2026programbench, p. 4].
- Size ranges from small tools to FFmpeg and SQLite; the median original has 8,635 lines of code and 50 code files [yang2026programbench, p. 4].
- 248,853 tests, a median of 770 per task [yang2026programbench, p. 4].
- 9 models x 200 tasks = 1,800 runs, one run each [yang2026programbench, p. 10].
- A simple bash-only agent tool, 1,000 steps and 6 hours per task [yang2026programbench, p. 5].
- Two extra experiments: forcing a different language, and allowing internet access with cheating detection [yang2026programbench, p. 6].

## Key findings

- **[V]** No model resolves any task. Claude Opus 4.7 passes at least 95% of tests on 3.0% of tasks, Opus 4.6 on 2.5%, and Sonnet 4.6 on 1.6% [yang2026programbench, p. 6].
- **[V]** Average test pass rates range from 52.0% (Opus 4.6) to 15.9% (GPT 5 mini). They fall from easy to hard tasks, for example 73.9% to 24.7% for Opus 4.6 [yang2026programbench, p. 31].
- **[V]** The generated tests cover 79.7% of code lines on average, against 56.8% for the projects' own test suites [yang2026programbench, p. 7].
- **[V]** Making tests stricter cut the dummy pass rate from 18.5% to 3.7% without changing the model ranking [yang2026programbench, p. 8].
- **[V]** In the 207 runs passing at least 75% of tests, rebuilds had a median of 1,173 lines vs 3,068, and 3 files vs 15 [yang2026programbench, p. 9].
- **[V]** Models write 10% to 29% as many functions as the original, and their functions are longer [yang2026programbench, p. 9].
- **[V]** More effort helps little: pass rate correlates only weakly with API calls ($r = 0.27$) and cost ($r = 0.21$) [yang2026programbench, p. 33].
- **[V]** With internet access, 20% to 36% of tasks were flagged as cheating for the three stronger models, mostly by looking up the source code [yang2026programbench, p. 6].

## Limitations and weaknesses

- **[A]** A finite test suite is only a "lower bound" on correctness; rebuilds may still differ on untested inputs [yang2026programbench, p. 12].
- **[A]** Speed, memory and other non-functional properties are not tested [yang2026programbench, p. 12].

- **[A]** Only a single agent was tested, with no multi-agent or human-in-the-loop setting and no human study [yang2026programbench, p. 20].
- **[I]** The test-writing model is a Claude model, and Claude models top the leaderboard. This possible bias is not tested [yang2026programbench, p. 31].
- **[I]** Famous programs like SQLite may be memorised. Blocking the internet stops lookup, not memory [yang2026programbench, p. 17].
- **[I]** Code quality is described only by size and structure. Those numbers come from a selected subset and are mixed up with language changes, since Python rewrites of C code are naturally shorter [yang2026programbench, p. 35].
- **[I]** One run per model and task, and the claimed "significantly different" pass rates are not backed by a reported test [yang2026programbench, p. 2].
- **[A]** The authors say % tests passed "does not reliably correlate" with how much functionality works [yang2026programbench, p. 5].

## Gaps and future work this paper leaves open

- G1: rebuilds are shorter and shallower, but maintainability is not measured [yang2026programbench, p. 9].
- G2: the same model family writes the tests and wins [yang2026programbench, p. 3].
- G5: memorisation is only partly controlled [yang2026programbench, p. 6].
- G6: the authors name human-guided reconstruction as future work [yang2026programbench, p. 12], and there is no human baseline [yang2026programbench, p. 20].
- G7: its tests are implementation-independent, a model for oracle design [yang2026programbench, p. 5].
- **[I]** Graphical, web and mobile apps are not covered.

## Why it matters for your research

**[I]** ProgramBench gives you a tested recipe for black-box rebuilding: hide the source, block the internet, accept any language, and judge only observable behaviour. Its test-quality checks (coverage plus a dummy program) are worth copying. Its main open question is yours: the rebuilds are smaller and more monolithic, but nobody has measured whether that makes them harder to maintain. Run more than one attempt per task and use test writers from a different model family than the ones you evaluate.

## Connections to other papers

- Agrees with bai2026recreationworld: rebuilds there are also much smaller and use fewer files [bai2026recreationworld, p. 15].
- kim2026programdistill builds on it and cites it as the whole-program "behavior-to-code" setting [kim2026programdistill, p. 1]. bai2026recreationworld uses it to test training transfer [bai2026recreationworld, p. 6].
- Agrees with kim2026programdistill that fidelity drops with complexity [kim2026programdistill, p. 8].

- **[I]** Seems to contrast with AI-assisted development, which adds code [he2026speed, p. 8]. The settings differ: rebuilds from scratch vs additions to existing projects.

# E2EDev: Benchmarking Large Language Models in End-to-End Software Development Task

> **Takeaway:** E2EDev starts from real, existing web apps, writes tests that pass on the originals, and asks AI systems to rebuild each app from its requirements. The best setup fully satisfies only 53.75% of requirements, and multi-agent frameworks do not consistently beat a plainly prompted model.

| Item | Detail |
|---|---|
| Authors | Jingyao Liu, Chen Huang, Zhizhao Guan, Wenqiang Lei, Yang Deng |
| Year · Venue | 2026 · Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (ACL 2026, Long Papers) |
| Status | Peer-reviewed. We read the published ACL Anthology version |
| Identifier | DOI 10.18653/v1/2026.acl-long.1618 · arXiv 2510.14509 |
| Paper type | Benchmark plus comparative evaluation (6 frameworks × 6 models, 3 runs) plus human failure analysis |
| Relevance to us | High — RQ1, RQ2, RQ4, RQ5; the closest design to reverse engineering among the spec-to-app benchmarks |

## In plain words

Most earlier tests of AI software builders used vague task descriptions and scored the results by human opinion or rough rules [liu2026benchmarking, p. 1]. The authors wanted something stricter. They took 46 small, popular web apps from GitHub, wrote down exactly what each app should do, and wrote automatic tests that pass on the original app [liu2026benchmarking, p. 5]. Then they asked AI systems to rebuild each app from that written description and ran the same tests on the rebuild. This is very close to our idea of reconstruction: the original app is the yardstick for the copy.

## How they did it

1. Choose apps: GitHub web projects with more than 500 stars that run in about five minutes without a database or API key [liu2026benchmarking, p. 12].
2. Add anchors: GPT-4o agents inserted data-testid labels into the original source. A data-testid is a fixed name attached to a button or field so that a test can find it [liu2026benchmarking, p. 3].
3. Extract requirements: AI agents read the code and wrote user-facing requirements; human supervisors checked them against the running app [liu2026benchmarking, p. 4].
4. Write tests: an agent wrote BDD tests. BDD (behaviour-driven development) means tests written as plain "Given … When … Then …" scenarios. Five software-testing experts reviewed and extended them [liu2026benchmarking, p. 4].

5. Automate tests: an agent turned each scenario into Python code that drives a real Chrome browser through Selenium, with self-correction and human fallback [liu2026benchmarking, p. 5].
6. Check the tests: every test passes on the original projects [liu2026benchmarking, p. 5].
7. Rebuild: each AI setup received the project summary, the requirements and the data-testid names [liu2026benchmarking, p. 16].
8. Score: the tests give a plain pass or fail; no AI judge is involved [liu2026benchmarking, p. 5]. Req.Acc is the share of requirements whose tests all pass; Test.Acc is the share of single tests that pass. Results are averaged over three runs, and cost and time are also recorded [liu2026benchmarking, p. 5].

## Data and setup

- 46 projects, 244 requirements and 703 tests (about 15 tests per project) [liu2026benchmarking, p. 3].
- Project types include calculators, form apps and mini games [liu2026benchmarking, p. 18].
- Six frameworks: a plain prompted model ("Vanilla"), GPT-Engineer (single agent), and four multi-agent systems (Self-Collaboration, MapCoder, ChatDev, MetaGPT). A multi-agent framework splits the work among several AI "roles" that talk to each other [liu2026benchmarking, p. 5].
- Six backbone models: Claude-Haiku 4.5, GPT-4o, GPT-4o-mini, Qwen2.5-7B, Qwen2.5-72B and Qwen2.5-Max [liu2026benchmarking, p. 5]. OpenHands and SWE-Agent ran on two of them only [liu2026benchmarking, p. 20].
- Test quality audit: 10 experts reviewed 100 sampled tests with Fleiss' kappa 0.81 [liu2026benchmarking, p. 14]. Kappa is an agreement score corrected for chance; 1 means perfect agreement.
- A human failure analysis covered 360 generated projects [liu2026benchmarking, p. 7].

## Key findings

- **[V]** The best setting, GPT-Engineer with Claude-Haiku 4.5, reaches Req.Acc 53.75 ± 0.43 and Test.Acc 69.41 ± 1.07 [liu2026benchmarking, p. 6]. **[A]** The authors summarise most settings as "only 30%–50%" [liu2026benchmarking, p. 5].
- **[V]** Partial success is common: a softer score that gives credit for partly met requirements is about 25–33 points higher in most settings [liu2026benchmarking, p. 25].
- **[A]** "Agentic frameworks do not consistently improve the effectiveness of base LLMs" [liu2026benchmarking, p. 6]. **[V]** Example: with GPT-4o-mini, Vanilla scores 44.82 vs ChatDev 33.16 and MapCoder 41.30 [liu2026benchmarking, p. 6]. **[I]** By our count, only 9 of 24 multi-agent cells beat Vanilla, mostly by small margins.
- **[V]** Most multi-agent frameworks cost more but the sums are small: GPT-4o ChatDev costs $0.1947 per task vs $0.0160 for Vanilla, while Self-Collaboration costs about the same as Vanilla. All four multi-agent frameworks are slower, e.g. 28 s for Vanilla vs 114 s for ChatDev on GPT-4o [liu2026benchmarking, p. 6].
- **[V]** MetaGPT almost always fails (0.00–5.39 Req.Acc) [liu2026benchmarking, p. 6]; 44% of its failures are code inconsistency such as missing files [liu2026benchmarking, p. 8].

- **[V]** The same frameworks score 71–91% on the small-function benchmark HumanEval, far above their E2EDev scores [liu2026benchmarking, p. 27].
- **[V]** OpenHands and SWE-Agent score below Vanilla on Claude-Haiku 4.5 (43.45 and 44.73 vs 48.69) [liu2026benchmarking, p. 23].
- **[V]** Building the benchmark still needed humans: about 50% of test cases were refined by hand, at about 3.5 hours per project [liu2026benchmarking, p. 15].

## Limitations and weaknesses

- **[A]** Only 46 projects, because executable tests are costly to build [liu2026benchmarking, p. 9].
- **[A]** Web apps only; CI/CD and deep back-end checks are excluded [liu2026benchmarking, p. 9].
- **[I]** Tests depend on data-testid names copied from the original and given in the prompt [liu2026benchmarking, p. 16]. A correct rebuild that names elements differently would fail. A true black-box rebuild would not have these names.
- **[I]** The only check of automatic vs human scoring (Cohen's kappa 0.74) does not describe its sample or unit [liu2026benchmarking, p. 22].
- **[I]** Reporting is inconsistent in places: four vs five annotators [liu2026benchmarking, p. 7; liu2026benchmarking, p. 21], and strict scores in one table differ from the main table [liu2026benchmarking, p. 25].
- **[I]** Popular public projects may be in the models' training data; this is not discussed [liu2026benchmarking, p. 12].
- **[I]** Requirements are very detailed, unlike vague vibe-coding prompts [liu2026benchmarking, p. 16]. The models are small or older ones, apps are front-end only, decoding settings are not reported, and there are no significance tests.
- **[I]** No code quality or maintainability measure.

## Gaps and future work this paper leaves open

- **[A]** The authors call for reliable, cost-efficient end-to-end methods and a larger, maintained benchmark [liu2026benchmarking, p. 9].
- **[I]** G7: how much the identifier-tied tests distort fidelity scores is unmeasured. Our G7 study would rewrite E2EDev tests with locators based on visible text and roles and compare [liu2026benchmarking, p. 9].
- **[I]** G1: E2EDev apps with validated tests could form "Stratum A" of the G1 study [liu2026benchmarking, p. 5].
- **[I]** G5: contamination is not discussed. G3: E2EDev outputs could test whether quality tools agree.

## Why it matters for your research

**[I]** E2EDev gives you a concrete template for measuring fidelity: start from an existing app, write behaviour tests that pass 100% on the original, then run them on the rebuild. Its released apps and tests are reusable. Two things to avoid: tying tests to the original's internal names, and

assuming that more elaborate agent setups give better rebuilds. Note that its 53.75% is not comparable with scores from other benchmarks.

## Connections to other papers

- **[V]** Converges with kim2026programdistill and tran2026vibe: the best requirement- or workflow-level pass rates in this field sit around 49–62% [kim2026programdistill, p. 11; tran2026vibe, p. 6].
- **[V]** Shares the identifier problem with kim2026programdistill, whose tests also use the original's selectors [kim2026programdistill, p. 35]. yang2026programbench and bai2026recreationworld avoid it with implementation-agnostic tests [yang2026programbench, p. 5; bai2026recreationworld, p. 9].
- **[V]** Contrasts with mock2026vibe, where agentic MetaGPT X had the highest test pass rate (mean 94.58%) [mock2026vibe, p. 25], but on a tiny two-function task with newer models [mock2026vibe, p. 12]. **[I]** Framework value likely depends on the model and task size.
- **[V]** Differs from tran2026vibe, which scores with an AI browser judge rather than deterministic tests [tran2026vibe, p. 4].
- **[V]** GPT-4o built the annotations and was evaluated, but the best backbone was Claude-Haiku [liu2026benchmarking, p. 13]; no self-advantage is visible.

# WebGen-Bench: Evaluating LLMs on Generating Interactive and Functional Websites from Scratch

> **Takeaway:** When AI coding agents build whole websites from a plain-language request, even the best early-2025 setup passes only 27.8% of the test cases, and visual or design requirements are met far more often than functional ones. A site can look right while its features do not work.

| Item | Detail |
|---|---|
| Authors | Zimu Lu, Yunqiao Yang, Houxing Ren, Haotian Hou, Han Xiao, Ke Wang, et al. (10 authors in our metadata; the arXiv PDF author line differs slightly) |
| Year · Venue | 2025 · Advances in Neural Information Processing Systems 38 (NeurIPS 2025) |
| Status | Peer-reviewed. We read the arXiv v2 version, which is marked "Under review"; the published version may differ in details |
| Identifier | DOI 10.52202/085713-2419 · arXiv 2505.03733 |
| Paper type | Benchmark plus comparative evaluation, with a fine-tuning experiment |
| Relevance to us | High — RQ1, RQ2, RQ4; RQ5 by what it leaves out (no code quality) |

## In plain words

The authors ask a simple question: can an AI agent build a complete, working website on its own if you only describe what you want in everyday words? To answer it they built a benchmark, which is a fixed set of tasks plus an automatic way of scoring them, so that different AI systems can be compared fairly [lu2025webgen, p. 3]. The tasks are 101 website requests, such as small business or data-management sites, and each comes with a list of things a user should be able to do [lu2025webgen, p. 5]. A second AI then clicks through each generated site like a tester and reports whether each feature works [lu2025webgen, p. 6]. This matters for us because it was one of the first studies to score whole AI-built apps by their behaviour, not just by how the code looks.

## How they did it

1. They collected 20 categories of web applications from freelance job platforms. Forty computer-science PhD students brainstormed 10,152 project ideas, and GPT-4o turned them into instructions [lu2025webgen, p. 5].
2. They picked 101 instructions as the test set [lu2025webgen, p. 5].
3. For each instruction, GPT-4o drafted 5–10 test cases. A test case is one small action plus the expected result (for example "click Add, a new row appears"). Two PhD students checked and corrected them, giving 647 test cases [lu2025webgen, p. 6].

4. Three agent frameworks built the sites: Bolt.diy, OpenHands and Aider. An agent framework is the software around a language model that lets it write files and run commands on its own [lu2025webgen, p. 7].
5. A "UI agent" (WebVoyager, driven by the Qwen2.5-VL-32B model) opened each site, tried each test case in at most 15 interactions, and answered YES, PARTIAL or NO [lu2025webgen, p. 6].
6. Accuracy is (YES + 0.5 × PARTIAL) divided by all test cases. A site that does not start counts as a failure [lu2025webgen, p. 8].
7. GPT-4o also gave each site an appearance grade from 1 to 5 [lu2025webgen, p. 6].
8. To check the automatic tester, three humans tested three settings by hand and their labels were compared with the UI agent's [lu2025webgen, p. 8].
9. Finally they fine-tuned (further trained) open Qwen2.5-Coder models on 600 recorded agent runs to see whether training helps [lu2025webgen, p. 7].

## Data and setup

- 101 test instructions: 28 content presentation, 49 user interaction, 24 data management [lu2025webgen, p. 6].
- 647 test cases: 339 functional, 186 data display, 122 design validation [lu2025webgen, p. 23]. That table prints "Total 667", which is a typo; the categories add up to 647.
- Models included Claude-3.5-Sonnet, DeepSeek-R1, DeepSeek-V3, GPT-4o, o3-mini and Qwen2.5 models, mostly inside Bolt.diy [lu2025webgen, p. 7].
- One run per setting. Temperature and time budgets are not reported [lu2025webgen, p. 40].
- Testing all 647 cases by hand would take about 10.8 hours and $377.8 per setting, which is why they automated it [lu2025webgen, p. 6].
- Data, code and models are released under an MIT licence (author claim; we did not check the link) [lu2025webgen, p. 1].

## Key findings

- **[V]** The best general-model setting, Bolt.diy with DeepSeek-R1, reaches only 27.8% accuracy; Bolt.diy with Claude-3.5-Sonnet reaches 26.4% [lu2025webgen, p. 8].
- **[V]** The framework matters a lot. With the same model (DeepSeek-V3), accuracy is 20.8% in Bolt.diy, 14.1% in Aider and 9.0% in OpenHands [lu2025webgen, p. 8].
- **[V]** Many sites never start: start failures reach 38.6% for Bolt.diy with o3-mini [lu2025webgen, p. 8].
- **[V]** Functional tests are much harder than design tests: 21.1% vs 44.3% for Bolt.diy with DeepSeek-R1, and 17.1% vs 52.0% with Claude-3.5-Sonnet [lu2025webgen, p. 9].
- **[A]** The authors conclude that "superficial aspects, such as color themes, are easier to implement than deeper internal functionalities" [lu2025webgen, p. 8].
- **[V]** The UI agent agrees with the human label on 86.1–94.4% of test cases in the three checked settings [lu2025webgen, p. 9].
- **[V]** Fine-tuning Qwen2.5-Coder-32B on 600 runs lifts accuracy from 9.5% to 38.2%, and more training data helps (25.1% with 150, 31.9% with 300, 38.2% with 600) [lu2025webgen, p. 10].

## Limitations and weaknesses

- **[A]** Only TypeScript, JavaScript, CSS and HTML sites; only supervised fine-tuning was tried [lu2025webgen, p. 14].
- **[A]** No error bars "because it would be too computationally expensive" [lu2025webgen, p. 40].
- **[I]** The human check covers only 3 of 16 settings, all in Bolt.diy. Agreement is raw percent with no chance correction, and agreement between the humans themselves is not reported [lu2025webgen, p. 9].
- **[I]** Most labels are NO (58–74%), so a tester that often says NO gets high agreement almost by default [lu2025webgen, p. 9].
- **[I]** Each test checks one small visible feature, and PARTIAL earns half credit [lu2025webgen, p. 8]. Deeper behaviour such as saved data or multi-step workflows is not tested.
- **[I]** With a single run, differences of 1–2 points (27.8 vs 26.4) cannot be told apart from noise.
- **[I]** The fine-tuning data was filtered by appearance score, not by functional tests [lu2025webgen, p. 7].
- **[I]** No code quality, security or maintainability is measured.

## Gaps and future work this paper leaves open

- **[A]** The authors want more languages and tools, and other training methods (RL, DPO) [lu2025webgen, p. 14].
- **[I]** G1: the generated code is never checked for quality or maintainability.
- **[I]** G2: GPT-4o wrote the test cases and is also evaluated. It scored low, so no advantage is visible [lu2025webgen, p. 8], but bias was not tested. The released outputs could be re-judged in a G2 study [lu2025webgen, p. 1].
- **[I]** G6: there is no human-developer baseline [lu2025webgen, p. 7].
- **[I]** For reconstruction, tests would need to come from an existing site's observed behaviour, not from an instruction.

## Why it matters for your research

**[I]** WebGen-Bench set the pattern that many later benchmarks copy: AI-drafted, human-checked test cases run by a visual browser agent, plus a check against human testers. You can reuse this idea to score a rebuilt app against its original. Its main lesson for you is the gap between design and function: a rebuild that looks like the original may still fail on behaviour. Avoid two of its weak points in your own study: single runs, and judge checks on only a few settings without chance-corrected agreement.

## Connections to other papers

- Agrees with chen2025iwr, where interactive functionality scores below visual fidelity for all models (GPT-5: 24.39 vs 64.25) [chen2025iwr, p. 8], and with xiao2025benchmarking, where interactive regions score lower than full pages [xiao2025benchmarking, p. 6].

- Partly disagrees with bai2026recreationworld, where the visual channel scores below the programmatic channel for 9 of 10 models [bai2026recreationworld, p. 11]. **[I]** The visual and functional scales differ across papers, so "looks right, works wrong" is only directionally supported.
- tran2026vibe builds on the same idea with multi-step browser workflows [tran2026vibe, p. 4] and shows that judge agreement with humans ranges from 36.1% to 86.4% depending on the judge model [tran2026vibe, p. 9]. **[I]** So the 86–94% here holds only for this judge.
- **[I]** Do not compare its 27.8% with E2EDev's 53.75% [liu2026benchmarking, p. 6] or Vibe Code Bench's 61.77% [tran2026vibe, p. 6]; each uses a different test method and scoring rule.

# Vibe Code Bench: Evaluating AI Models on End-to-End Web Application Development

> **Takeaway:** When 16 AI models each build a full, deployed web app from a plain-text description, the best passes 61.77% of user-style browser workflows. The choice of AI "judge" model changes the measured result a lot, and code quality is not measured at all.

| Item | Detail |
|---|---|
| Authors | Hung Tran, Langston Nashold, Rayan Krishnan, Antoine Bigeard, Alex Gu |
| Year · Venue | 2026 · Proceedings of the ACM Conference on AI and Agentic Systems |
| Status | Peer-reviewed. We read the arXiv v3 version (May 2026); the ACM version was not compared |
| Identifier | DOI 10.1145/3786335.3813180 · arXiv 2603.04601 |
| Paper type | Benchmark plus comparative evaluation of 16 models, with a judge-reliability and variance study |
| Relevance to us | High — RQ1, RQ2, RQ4, RQ5; RQ3 as negative evidence (quality not measured) |

## In plain words

The authors test the "zero-to-one" job: going from a short, non-technical description of an app to a working, deployed web application, with no human help [tran2026vibe, p. 3]. Instead of reading the code, they judge the app the way a user would: an AI agent opens it in a browser and follows pre-written user workflows step by step [tran2026vibe, p. 4]. They also ask how far that AI judge can be trusted, by comparing it with human testers [tran2026vibe, p. 9]. This matters for us because it is the closest benchmark to an autonomous vibe-coding build, and its browser-based scoring could be reused to compare a rebuild with its original.

## How they did it

1. Write tasks: a GPT-5 pipeline drafted 100 app specifications and their test workflows. Two product-manager reviewers accepted, changed or rejected each test, and an AI audit looked for mismatches [tran2026vibe, p. 3].
2. Split: 50 apps for validation and 50 for the test set used in the main results [tran2026vibe, p. 4].
3. Build: each model worked inside a modified OpenHands container. This "harness" is the fixed environment around the model: a terminal, a browser and ready-made services for database (Supabase), email (MailHog) and payments (Stripe). Each app had up to 5 hours [tran2026vibe, p. 4].

4. Fix the technology: the system prompt required React + Vite, Tailwind, Supabase and Docker Compose, so that apps could be tested in the same way [tran2026vibe, p. 4].
5. Test: a Browser Use agent with Claude Sonnet 4.5 as the judge followed pre-written workflows and marked each small substep pass or fail. A workflow passes when at least 90% of its substeps pass; accuracy is the share of workflows that pass. If the app cannot be deployed, all its workflows fail [tran2026vibe, p. 5].
6. Keep tests independent: each workflow starts in a fresh browser session with new account data [tran2026vibe, p. 5].
7. Check variance: 6 specs × 3 models × 5 separate builds, plus 5 re-judgings of the same apps [tran2026vibe, p. 8].
8. Check the judge: 18 apps (1,401 substeps) were tested by two of three humans who were blind to the model, and by four judge models [tran2026vibe, p. 8].

## Data and setup

- 100 specifications, 964 workflows and 10,131 substeps; the test split has 50 apps, 473 workflows and 5,136 substeps [tran2026vibe, p. 4].
- 28 of the 100 apps need payments and/or email [tran2026vibe, p. 3].
- 16 models from 9 providers; no human developer or commercial tool baseline [tran2026vibe, p. 5].
- Three models ran without browser vision because of provider settings [tran2026vibe, p. 5].
- **[I]** The paper does not state how many builds per app were used in the main table; it appears to be one.
- The harness, all trajectories (step-by-step logs of the agent) and generated apps are released (author claim) [tran2026vibe, p. 11].

## Key findings

- **[V]** Top scores on the test split: GPT-5.3-Codex 61.77 ± 4.71%, Claude Opus 4.6 57.57%, GPT-5.2 53.50%; the lowest is Grok 4.1 Fast Reasoning at 1.20%. Open-weight models reach about 5–23% [tran2026vibe, p. 6].
- **[V]** Harder apps and external services lower the score: 44.29% for easy vs 6.03% for hard apps, and 34.18% with no integrations vs 13.49% with email plus Stripe (averages over all models) [tran2026vibe, p. 6]. These subgroups contain only a few apps each.
- **[V]** Models that used the browser more during building scored higher ($r = 0.72$ across 16 models), while editing volume did not ($r = 0.09$) [tran2026vibe, p. 7]. **[A]** The authors read this as self-testing being linked to better results [tran2026vibe, p. 7].
- **[V]** Failure is often total: 73.2% of apps from the five weakest models score zero vs 12.8% for the five strongest, and only 8.8% of top-five apps score 100% [tran2026vibe, p. 7].
- **[V]** Among apps that started, failed workflows were mostly missing features (46.7%) or authorization problems (20.4%) [tran2026vibe, p. 8].
- **[V]** Build-to-build variation is the main source of noise: 73.5–92.5% of observed variance comes from generation, not from the judge [tran2026vibe, p. 8].

- **[V]** Humans agree with each other on 88.6–93.6% of substeps. The official judge (Claude Sonnet 4.5) agrees with humans on 86.4%, but GPT-5.2 as judge agrees on only 36.1% [tran2026vibe, p. 9]. **[A]** "Evaluator selection can materially change reported outcomes" [tran2026vibe, p. 9].

## Limitations and weaknesses

- **[A]** Code readability, visual design and security are not measured: "Passing functional tests does not imply maintainable, secure, or well-documented code" [tran2026vibe, p. 9].
- **[A]** One harness, one technology stack and web apps only [tran2026vibe, p. 9].
- **[I]** The official judge is a Claude model, and several evaluated models are Claude models. Self-preference was not tested; the human check used apps from only three models [tran2026vibe, p. 8].
- **[I]** GPT-5 drafted the specs and an OpenAI model ranks first [tran2026vibe, p. 3]; this possible advantage is not examined.
- **[I]** Agreement is raw percent, with no chance correction such as kappa [tran2026vibe, p. 9]. The very low GPT-5.2 agreement is unexplained.
- **[I]** The browser-use correlation is observational and based on 16 models, and the system prompt itself demands browser testing [tran2026vibe, p. 16].
- **[I]** Difficulty tiers are defined from the models' own pass rates, so tier results are partly circular [tran2026vibe, p. 6].
- **[I]** The validation split is called "private" in the text but "public" in a table [tran2026vibe, p. 4]. Funding and conflicts are not reported.

## Gaps and future work this paper leaves open

- **[A]** The authors suggest extending to CLI, desktop and mobile apps, other harnesses and stacks, and treating judge choice as "a first-class reporting concern" [tran2026vibe, p. 9].
- **[I]** G1: code quality and maintainability of the apps remain unmeasured.
- **[I]** G2: spec-writer, judge and evaluated models overlap in family. The released apps make a re-judging study possible [tran2026vibe, p. 11].
- **[I]** G3: the released apps could also test whether maintainability tools agree with each other.
- **[I]** G4: whether self-testing truly causes better results needs a controlled experiment.
- **[I]** G6: no human or human-steered baseline [tran2026vibe, p. 5]. G8: success drops with hidden external services [tran2026vibe, p. 6].

## Why it matters for your research

**[I]** This is your best ready-made template for scoring behavioural fidelity: write user workflows in plain language, run them in a browser on the original and on the rebuild, and compare. Its released apps are free data for small first studies on judge bias or code quality. It also warns you to validate any AI judge yourself, report run-to-run variation, and not treat a high pass rate

as proof of good code. Note that "vibe coding" here means an autonomous agent, not a human prompting a tool.

## Connections to other papers

- **[V]** Builds on lu2025webgen, which used single-action test cases with half credit and reported 86–94% judge–human agreement [lu2025webgen, p. 9]. **[I]** Vibe Code Bench adds multi-step workflows and shows that agreement depends strongly on the judge model.
- **[V]** Its top score sits in the same 49–62% band as E2EDev and ProgramDistill [liu2026benchmarking, p. 6; kim2026programdistill, p. 11]. **[I]** The oracles differ, so the numbers must not be ranked against each other.
- **[V]** Contrasts with pan2026functional, where self-repair and planning lowered code quality without raising correctness on GPT-5.2 [pan2026functional, p. 8]. **[I]** Different designs; this conflict is unresolved (gap G4).
- **[V]** Unlike E2EDev, which uses deterministic browser tests and no AI judge [liu2026benchmarking, p. 5], scoring here relies on an AI judge.
- **[V]** Practitioner vibe coding is described elsewhere as human prompting with minimal review [fawzy2026vibe, p. 1]. **[I]** Results from autonomous agents should not be pooled with human-steered studies.

# Toward Functional and Non-Functional Evaluation of Application-Level Code Generation

> **Takeaway:** When 16 AI models rewrite 38 real Python projects from a short description, none passes more than 45% of the tests, and code quality measured against the original is middling. It is one of the very few studies that scores working behaviour and code quality on the same generated apps, using the original project as the yardstick.

| Item | Detail |
|---|---|
| Authors | Ruwei Pan, Yakun Zhang, Qingyuan Liang, Yueheng Zhu, Chao Liu, Lu Zhang, Hongyu Zhang |
| Year · Venue | 2026 · arXiv |
| Status | Preprint (not peer-reviewed) |
| Identifier | DOI 10.48550/arxiv.2602.03462 · arXiv 2602.03462 |
| Paper type | Benchmark (RAL-Bench) plus controlled empirical comparison of 16 models |
| Relevance to us | High — RQ2, RQ3, RQ4, RQ5 (RQ1 secondary) |

## In plain words

Most code benchmarks test single functions or small fixes. The authors ask whether AI models can write a whole working application, with many files and dependencies, and whether that code is also good on qualities beyond "it works" [pan2026functional, p. 1]. These other qualities are called non-functional: how maintainable, secure, robust, fast and resource-hungry the code is. They take real open-source projects as references, describe each in a short text, ask the AI to write the project, and test the result against the original [pan2026functional, p. 2]. For us this is close to a joint "fidelity plus quality" study, although the AI never sees or runs the original app.

## How they did it

1. Pick projects: 38 maintained Python projects on GitHub (for example click, requests, celery), each frozen at a fixed version, in seven scenario groups [pan2026functional, p. 2].
2. Write requirements: a short natural-language description built from the project's README, API, examples and code, then edited to remove implementation details and project names [pan2026functional, p. 2].
3. Add an interface specification: the list of modules and packages the result should expose [pan2026functional, p. 4].
4. Generate: each model writes the whole repository in one go, "zero-shot" (no examples and no second attempt). Each model runs 3 times and scores are averaged [pan2026functional, p. 5].

5. Test behaviour: black-box tests (tests that only check inputs and outputs, not the code inside) are kept only if they pass on the original. The functional score is the share of tests passed [pan2026functional, p. 4].
6. Test quality: five dimensions, each compared with the original and capped at 1 [pan2026functional, p. 3]. Maintainability uses the Maintainability Index (MI), a formula that turns code complexity and size into one number. Security counts high-risk findings from static analysis (automatic code scanning without running it). Robustness, efficiency and resource use (memory and CPU) come from extra test runs [pan2026functional, p. 4].
7. Combine: the five dimensions are weighted (maintainability 0.36, security 0.24, robustness 0.16, efficiency 0.12, resource 0.12) into one non-functional score (NF) [pan2026functional, p. 4].
8. Extra studies: a failure analysis, costs, three improvement strategies on GPT-5.2, prompt wording, and different weightings [pan2026functional, p. 5].

## Data and setup

- 38 projects of 1 to 242 files and 0.3k to 109.5k lines of code [pan2026functional, p. 2].
- About 11 functional tests per task on average (minimum 10, maximum 19) [pan2026functional, p. 5].
- 16 models: 11 "standard" and 5 "thinking" (reasoning) models, 3 runs each [pan2026functional, p. 5].
- 76 deliberately injected bugs (2 per task) to check that the tests catch errors [pan2026functional, p. 5].
- The pipeline was rerun 5 times for three models to check stability [pan2026functional, p. 7].
- Greedy decoding is stated only in the abstract [pan2026functional, p. 1]. Scripts and data are released (author claim) [pan2026functional, p. 11].

## Key findings

- **[V]** Functional correctness is low: on average 33.98% for standard and 30.23% for thinking models, and no model exceeds 45%. The best is Gemini-3-Pro-Preview at 43.86%; the lowest is DeepSeek-R1 at 16.23% [pan2026functional, p. 6].
- **[V]** Non-functional scores average 52.25% (standard) and 49.57% (thinking). Security is the strongest dimension (87.40% / 85.32%). Resource usage is the weakest (31.25% / 28.92%), with maintainability close behind (33.45% / 31.38%) [pan2026functional, p. 6].
- **[V]** Quality separates models that function equally well: Gemini-2.5-Pro and GPT-4o score almost the same functionally (27.63% vs 27.48%), but 35.15% vs 57.59% on NF [pan2026functional, p. 5].
- **[V]** Thinking models are not consistently better [pan2026functional, p. 6].
- **[V]** 82.8% of failures happen after the code runs (wrong behaviour or poor quality); 17.2% are install or start-up failures [pan2026functional, p. 7].
- **[V]** Cost per run ranges from $0.15 to $16.63, and paying more does not reliably buy correctness [pan2026functional, p. 8].

- **[V]** On GPT-5.2, self-repair, environment repair and planning all failed to beat the baseline (41.80% functional, 57.10% NF) and all lowered NF, to 46.10–49.90% [pan2026functional, p. 8].
- **[V]** Giving the module list raised one task's functional score from 0.1667 to 0.6667 [pan2026functional, p. 4]. Rewording the prompt moved GPT-5.2 from 0.4181 to 0.4972 [pan2026functional, p. 9].

## Limitations and weaknesses

- **[A]** The projects are public, so the authors "do not claim that pretraining exposure can be fully ruled out" [pan2026functional, p. 2].
- **[A]** MI misses naming quality and architecture; static security scans give false alarms and misses; all metrics are proxies, not expert audits [pan2026functional, p. 3].
- **[A]** One-shot generation does not represent iterative, agent-based work; the strategy study uses one model [pan2026functional, p. 8].
- **[I]** About 11 tests is a thin check for projects of up to 109.5k lines, and 2 injected bugs per task is a weak test of the tests.
- **[I]** Memorisation looks likely: the example requirement names the original "Stegano project" [pan2026functional, p. 3], and one output used internals of the real Celery library [pan2026functional, p. 9].
- **[I]** The quality score can reward incomplete code. GPT-3.5-Turbo scores 97.37% on security but 29.76% on function [pan2026functional, p. 6]; less code means fewer findings.
- **[I]** The maintainability score equals 0.5, not 1, when the rebuild matches the original [pan2026functional, p. 3], yet it has the largest weight.
- **[I]** Zero variance across 5 reruns [pan2026functional, p. 7] is surprising and may mean cached outputs were re-tested. The authors call the prompt effect small, but it is about 30–40% of the spread between models.
- **[I]** Python libraries only; no user interfaces are tested.

## Gaps and future work this paper leaves open

- **[A]** The authors propose quality-aware generation, repair across modules, and multi-turn, agent-based evaluation [pan2026functional, p. 10].
- **[I]** G1: this is the only reference-anchored maintainability score in our corpus, but it is a single MI proxy, from text requirements rather than observed behaviour [pan2026functional, p. 3].
- **[I]** G4: does feedback improve function and quality together? Here it harmed quality, for one model and one-shot strategies.
- **[I]** G5 and G7: memorisation and interface hints may inflate scores; neither effect is measured.
- **[I]** G3: whether MI agrees with other quality tools is untested.

## Why it matters for your research

**[I]** This paper shows the design you want, in rough form: measure behaviour and quality on the same rebuild and use the original as the baseline. Reuse the idea of tests kept only if they pass on the original, and of quality scores relative to the original. Improve on it: use richer, size-normalised quality measures, more tests, an iterative workflow, and controls for memorisation. Its result that repair strategies lowered quality is a warning for vibe-coding loops, but it is weak evidence (preprint, one model).

## Connections to other papers

- **[V]** Disagrees on security with zhao2025vibe, where 79.3% of functionally correct solutions fail targeted security tests [zhao2025vibe, p. 6]. **[I]** Static counts and attack-style tests measure different things.
- **[V]** Conflicts on feedback with tran2026vibe (browser self-testing correlates with accuracy, r = 0.72) [tran2026vibe, p. 7] and zhou2025declarui (refinement loop lifts navigation) [zhou2025declarui, p. 10]. **[I]** Those loops check the very thing they score; this one measures quality after repair.
- **[V]** Agrees that function and quality diverge: in wu2026looks the visual leader is not the maintainability leader [wu2026looks, p. 7].
- **[V]** Rebuilds elsewhere are much smaller than their originals [bai2026recreationworld, p. 15; yang2026programbench, p. 9]. **[I]** That is why size-sensitive quality scores like this one need care.
- **[V]** borg2026echoes offers a stronger maintainability test: a second developer evolves the code [borg2026echoes, p. 13].

# Is Vibe Coding Safe? Benchmarking Vulnerability of Agent-Generated Code in Real-World Tasks

> **Takeaway:** Code that passes its functional tests is often not secure. Even the best AI coding agent produced working code for 57.0% of tasks, but only 11.8% of tasks were solved both correctly and securely, so "it works" and "it is good code" must be measured separately [zhao2025vibe, p. 7].

| Item | Detail |
|---|---|
| Authors | Songwen Zhao, Danqing Wang, Kexun Zhang, Jiaxuan Luo, Zhuo Li, Lei Li |
| Year · Venue | 2025 · Published at ICML 2026 (our copy: arXiv v4) |
| Status | Peer-reviewed (ICML 2026, PMLR 306, per the PDF footer; our metadata still lists it as an arXiv preprint) |
| Identifier | arXiv 2512.03262 (DOI 10.48550/arxiv.2512.03262) |
| Paper type | Benchmark plus comparative evaluation of AI coding agents |
| Relevance to us | Medium — RQ2, RQ3, RQ4, RQ5 |

## In plain words

"Vibe coding" means asking an AI agent in plain language to build a feature, and then using the result with little human checking. This paper asks a simple question: is the code that such agents write safe to put into real software? The authors built a test set of 186 real programming tasks where human programmers had once written code with a security hole. They then asked 12 different AI agent setups to write the same features and checked two things: does the code work, and is it secure? The answer is worrying: most working solutions still contain the old security hole. This matters for us because it shows that checking whether software "behaves correctly" says little about other qualities of the code.

## How they did it

1. They started from real security fixes in open-source Python projects. Each fix has a "before" version (with the vulnerability) and an "after" version (fixed) **[V]** [zhao2025vibe, p. 3].
2. In the "before" version, an AI agent removed (masked) the code of one feature. A second agent wrote a feature request in the style of a GitHub issue (a short task description). The description had to explain what the feature does, without any hint about security **[V]** [zhao2025vibe, p. 4; zhao2025vibe, p. 16].
3. The masks were built on the vulnerable version, not the fixed one, so that no information from the fix leaks into the task **[A]** [zhao2025vibe, p. 4].

4. Each AI agent then had to re-implement the missing feature inside the full repository. It could run code and use the results, for up to 200 steps **[V]** [zhao2025vibe, p. 6].
5. Two kinds of tests were run on the result. Functional tests (the project's existing tests) check whether the feature works. Security tests (tests added by the historical fix) check whether the old vulnerability is back **[V]** [zhao2025vibe, p. 4; zhao2025vibe, p. 5].
6. Scoring used pass@1 (the share of tasks solved on the first and only try). FuncPass counts tasks that pass the functional tests; SecPass counts tasks that pass both functional and security tests **[V]** [zhao2025vibe, p. 6]. No AI judge was used for scoring **[V]** [zhao2025vibe, p. 6].
7. Five software-engineering experts checked the security tests and rewrote those that were too tied to one specific implementation **[V]** [zhao2025vibe, p. 19].

## Data and setup

- 186 tasks from 100 Python repositories, covering 10 application domains and 79 CWE types (CWE = a standard catalogue of software weakness types) **[V]** [zhao2025vibe, p. 2; zhao2025vibe, p. 13].
- Tasks are large: on average 236K lines of context, a 175.0-line target change, 66.9 functional tests and 3.5 security tests per task **[V]** [zhao2025vibe, p. 5].
- 12 agent settings: 3 agent frameworks (SWE-agent, OpenHands, Claude Code) times 4 models (Claude 4 Sonnet, Kimi K2, Gemini 2.5 Pro, Gemini 3 Pro) **[V]** [zhao2025vibe, p. 6].
- One run per task. Temperature, exact model versions and cost are not reported **[V]** [zhao2025vibe, p. 6].
- The main runs already include a short generic security reminder in the prompt **[V]** [zhao2025vibe, p. 7].

## Key findings

- **[V]** The best setting, SWE-agent with Claude 4 Sonnet, reached FuncPass 57.0% but SecPass only 11.8% [zhao2025vibe, p. 7]. 79.3% of its working solutions were insecure [zhao2025vibe, p. 6].
- **[V]** Across all 12 settings, FuncPass ranged from 12.9% to 57.0% and SecPass from 3.8% to 12.9% [zhao2025vibe, p. 7].
- **[V]** Success drops when the change spans more files. For Claude 4 Sonnet, FuncPass fell from 66.9% (one file) to 30.3% (more than two files), and SecPass from 15.4% to 3.0% [zhao2025vibe, p. 14].
- **[V]** Simple security prompting helped little. Giving the agent the exact weakness type raised SecPass from 11.8% to 15.1%, while FuncPass fell from 57.0% to 55.4%; letting the agent pick weakness types itself gave 50.0% / 14.5% [zhao2025vibe, p. 8].
- **[V]** When asked to choose weakness types, agents picked 7.2 types per task against 1.1 true types, with precision of only 0.086 to 0.114 [zhao2025vibe, p. 9].
- **[V]** On tasks that all models solved, Gemini 2.5 Pro had the highest secure share (30.4% vs 18.3% for Claude 4 Sonnet), although it was functionally the weakest [zhao2025vibe, p. 6]. **[I]** Treat this ranking as weak: the size of this shared subset is never reported and can

be at most about 24 to 37 tasks per framework, based on the FuncPass numbers [zhao2025vibe, p. 7].

## Limitations and weaknesses

- **[A]** Python only; tests are a practical stand-in for security, and the tests may not cover all weaknesses [zhao2025vibe, p. 24].
- **[A]** Security tests from historical fixes can reject secure alternative solutions; 6.5% were tied to the original implementation and were rewritten [zhao2025vibe, p. 19].
- **[I]** Only one run per task and no confidence intervals. With 186 tasks, one task is about 0.54 percentage points, so small differences such as 11.8% vs 12.9% may be noise [zhao2025vibe, p. 7].
- **[I]** No agreement score between the five experts is reported [zhao2025vibe, p. 19].
- **[I]** The tasks rebuild public historical code from popular projects, so models may have seen the fixed versions in training; the paper does not discuss this [zhao2025vibe, p. 13].
- **[I]** Claude 4 Sonnet built the test environments [zhao2025vibe, p. 17] and SWE-agent built the tasks [zhao2025vibe, p. 4], and SWE-agent with Claude 4 Sonnet scored best. A "home advantage" is possible but untested.
- **[I]** Only the known historical vulnerability is tested (3.5 tests per task), so other new holes go unseen; SecPass is an upper bound on "secure" [zhao2025vibe, p. 5].

## Gaps and future work this paper leaves open

- **[A]** The authors call for more languages, property-based and adversarial tests, static analysis and security-aware training [zhao2025vibe, p. 24].
- **[I]** It measures security, not maintainability, and patches, not whole applications. This is part of G1 (fidelity and maintainability of rebuilt apps are never measured together).
- **[I]** The possible builder–winner overlap is one of the cases behind G2 (same-family bias between benchmark builder and evaluated model).
- **[I]** Its coupled security tests are an example for G7 (tests tied to the original's code can unfairly fail valid rebuilds).
- **[I]** Training-data exposure of public repositories is not addressed, which links to G5.

## Why it matters for your research

This paper is your clearest evidence that "it passes the tests" and "it is good code" are different things **[I]**. Its task design is worth reusing: hide an existing implementation, describe it in words, let an AI rebuild it, then test it against the original's own tests. That is close to a reconstruction study. Reuse the idea of two separate scores (one for behaviour, one for a quality dimension). Avoid its weak points: run each setting several times, report the size of every subset, and check whether your tests are tied to the original's code.

## Connections to other papers

- **[V]** Agrees with molison2025llm that correctness and another quality dimension diverge: there, the fine-tuned model had the best static quality but the lowest Pass@1 [molison2025llm, p. 8].
- **[V]** Seems to disagree with pan2026functional, where reference-normalised security averages 87.40% [pan2026functional, p. 6]. **[I]** The instruments differ (static findings vs targeted security tests), so the numbers cannot be compared.
- **[V]** Agrees with tran2026vibe that success falls as tasks get more complex [tran2026vibe, p. 6].
- **[V]** Like yang2026programbench, where Claude wrote the tests and Claude models lead, it shows the builder–winner overlap behind G2 [yang2026programbench, p. 3; yang2026programbench, p. 31].

# Is LLM-Generated Code More Maintainable & Reliable Than Human-Written Code?

> **Takeaway:** GPT-4o code had far fewer SonarQube-flagged "bugs" than human competition code, but it was often functionally wrong, and the fine-tuned model with the best quality scores had the lowest correctness. Static quality and correctness must be measured together, and the "fewer bugs" headline rests on a weak human baseline [molison2025llm, p. 8].

| Item | Detail |
|---|---|
| Authors | Alfred Santa Molison, Marcia Moraes, Glaucia Melo, Fabio Santos, Wesley K. G. Assunção |
| Year · Venue | 2025 · ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM 2025) |
| Status | Peer-reviewed (our copy: arXiv v1, page numbers refer to it) |
| Identifier | DOI 10.1109/esem64174.2025.00036 (arXiv 2508.00700) |
| Paper type | Quantitative comparative study (static analysis) |
| Relevance to us | Medium — RQ3 (main), RQ2, RQ4, RQ5 |

## In plain words

People worry that AI-written code is harder to maintain than human code. This paper compares Python code written by GPT-4o with code written by humans for the same programming puzzles. It uses SonarQube, a widely used tool that scans code without running it and flags problems. The authors look at "bugs" (rule-based warnings about likely errors), "code smells" (signs of messy or hard-to-change code) and the estimated time to fix them. They also run a small check of whether the code actually gives the right answers. The study matters because it is one of the few peer-reviewed, direct AI-versus-human quality comparisons in our set.

## How they did it

1. They took coding puzzles from the APPS dataset, a public set of 10,000 Python problems at three difficulty levels (introductory, interview, competition), with human solutions from sites such as Codeforces **[V]** [molison2025llm, p. 3].
2. GPT-4o (version 2024-08-06) wrote solutions in three ways. Zero-shot: just the problem and the instruction to solve it. Few-shot: the problem plus all human solutions to that same problem as examples. Fine-tuned: a GPT-4o model trained on 240 question–answer pairs (80 per level), tested on 480 questions (160 per level) **[V]** [molison2025llm, p. 4].
3. All solutions were scanned with SonarQube using its default Python rules **[V]** [molison2025llm, p. 4; molison2025llm, p. 5].

4. They compared groups with odds ratios (how much more or less likely a problem is in one group than another) and, for fix time, a Mann-Whitney U test with Cliff's delta (an effect-size measure) **[V]** [molison2025llm, p. 5].
5. Three authors ran 15 solutions per approach (5 per level) by hand to compute Pass@1 (share of solutions that pass the tests on the first try) **[V]** [molison2025llm, p. 8].

## Data and setup

- Dataset: APPS, Python, three difficulty levels **[V]** [molison2025llm, p. 3].
- Model: GPT-4o-2024-08-06, temperature 1.00, max 2,048 tokens **[V]** [molison2025llm, p. 4].
- Number of human and zero-/few-shot solutions actually scanned: not reported **[I]** [molison2025llm, p. 4].
- Tool: SonarQube Free Server edition, default quality profile **[V]** [molison2025llm, p. 4].
- Correctness check: 15 solutions per approach **[V]** [molison2025llm, p. 8].
- Replication package on Zenodo **[V]** [molison2025llm, p. 11].

## Key findings

- **[V]** SonarQube found no security vulnerabilities in any group [molison2025llm, p. 5].
- **[V]** The odds of a SonarQube bug in GPT-4o code vs human code were tiny in every setting and level (odds ratios 0.00 to 0.019, all $p < 0.001$) [molison2025llm, p. 8]. **[I]** The 0.00 means one group had no bugs at all, so it is not a real estimate.
- **[V]** Code smells were mixed: zero-shot code had 3.57 times the odds of a smell on introductory problems, while fine-tuned code had fewer smells; several comparisons were not significant [molison2025llm, p. 8].
- **[V]** Estimated fix effort was lower for AI code on easy tasks but not on competition tasks (human 6.95 min, few-shot 7.33, fine-tuned 7.60) [molison2025llm, p. 7].
- **[V]** Pass@1 was 1.0 for humans, 0.87 few-shot, 0.53 zero-shot and 0.47 fine-tuned [molison2025llm, p. 8].
- **[A]** The fine-tuned model has "the best quality metrics" yet struggled with complex solutions [molison2025llm, p. 8].
- **[A]** On competition-level problems, AI code may carry more critical-severity issues than human code [molison2025llm, p. 6].

## Limitations and weaknesses

- **[A]** Python only, puzzle-style tasks, one model, and possible prompt sensitivity [molison2025llm, p. 10].
- **[A]** SonarQube may not capture all aspects of quality, and its fix-time estimates may not match real developer effort [molison2025llm, p. 10].
- **[I]** Leakage: the few-shot prompt contains the human solutions it is compared against, so few-shot vs human is not an independent comparison [molison2025llm, p. 4].
- **[I]** A SonarQube "bug" is a rule-based warning, not a failing program. Quality was also measured on solutions that fail the tests, so "fewer bugs" does not mean "more correct" [molison2025llm, p. 8].

- **[I]** The human baseline is competition code written to be correct, not maintainable, as the authors admit [molison2025llm, p. 9].
- **[I]** Group sizes are not reported, there are no confidence intervals and no correction for many comparisons, and the Pass@1 check uses only 15 solutions per approach [molison2025llm, p. 8].
- **[I]** APPS has been public since 2021, so GPT-4o may have seen it in training; this is not discussed [molison2025llm, p. 11].
- **[I]** Some numbers are inconsistent, for example few-shot Pass@1 is 0.86 in the introduction and 0.87 in the table [molison2025llm, p. 1; molison2025llm, p. 8].

## Gaps and future work this paper leaves open

- **[A]** The authors suggest other LLMs, more languages and quality metrics, retrieval-augmented generation and cost–quality trade-off studies [molison2025llm, p. 10].
- **[I]** It studies single small programs, not applications, and has no original app to compare against. This is the core of G1.
- **[I]** Its "fewer bugs" result conflicts in direction with studies that use other units or tools. Whether quality tools agree, and whether they predict real maintenance effort, is G3.
- **[I]** Public benchmark data and possible training exposure link to G5.

## Why it matters for your research

This paper gives you a ready-made recipe for the code-quality part of a study: SonarQube default profile, bugs, smells, severity and fix effort **[I]**. It also gives a strong warning. The model with the best quality scores was the least correct, so always report quality and correctness side by side, ideally quality only for code that works. Avoid its design mistakes: do not show the baseline to the model, report group sizes, use a realistic human baseline, and run several samples per task.

## Connections to other papers

- **[V]** Agrees with zhao2025vibe that correctness and another quality dimension diverge: there, 79.3% of working solutions failed security tests [zhao2025vibe, p. 6].
- **[V]** Seems to disagree with he2026speed, where adopting Cursor raised repository SonarQube warnings by 30.26% [he2026speed, p. 7]. **[I]** The units differ (single puzzle solutions vs whole repositories over months), so neither result refutes the other.
- **[V]** zhu2026understanding builds directly on this paper: its first experiment mirrors this design, without fine-tuning [zhu2026understanding, p. 3].
- **[A]** zhu2026understanding also criticises it: SonarQube "issues" mix superficial linting with real design flaws, so the authors switch to PyExamine [zhu2026understanding, p. 2]. With that tool, larger models wrote more Long Method smells than humans **[V]** [zhu2026understanding, p. 7].

# Comparing the Quality of Code Generated by Vibe Coding Tools

> **Takeaway:** Three popular vibe-coding platforms built the same web app from one prompt, and each left a different code-quality "fingerprint" (Lovable 22.83 SonarQube issues per 1,000 lines vs about 9 for v0 and Replit). With only 9 apps and no check that the apps work, this is an early signal, not a verdict [mota2026comparing, p. 5].

| Item | Detail |
|---|---|
| Authors | Gustavo da Mota, Kiev Gama |
| Year · Venue | 2026 · arXiv (page header names the 40th Brazilian Symposium on Software Engineering, SBES 2026, emerging-results track) |
| Status | Preprint (not peer-reviewed): the artefact link is "anonymized for review" and acceptance is not stated |
| Identifier | arXiv 2608.16302 (DOI 10.48550/arxiv.2608.16302) |
| Paper type | Exploratory comparative study (static analysis), short emerging-results paper |
| Relevance to us | High — RQ1, RQ3, RQ4, RQ5 |

## In plain words

Vibe-coding platforms such as Lovable, v0 and Replit promise to build a whole web app from a short text description. People use them because they are fast, but nobody knows much about the quality of the code they produce. This paper gives all three platforms the same request for a small wildlife-reporting web app and studies the code that comes out. It uses SonarQube, a common tool that scans code for problems without running it. The authors explicitly do not try to name a winner; they want "initial evidence" about code quality **[A]** [mota2026comparing, p. 2]. For us it is the closest study to rebuilding an app by vibe coding with commercial tools.

## How they did it

1. They wrote one zero-shot prompt (a single request with no example code) based on a "Nature Park Wildlife App" exercise: register wildlife sightings, send safety reports, subscribe to notifications, and show a map **[V]** [mota2026comparing, p. 2].
2. Each platform received the same prompt three times, with no extra instructions and no manual fixes **[V]** [mota2026comparing, p. 2].
3. Each of the nine resulting projects was scanned with SonarQube **[V]** [mota2026comparing, p. 2].
4. They counted issues by type (code smells, bugs, security hotspots), by severity (minor, major, critical), and SonarQube's estimated time to fix them (remediation effort) **[V]** [mota2026comparing, p. 2; mota2026comparing, p. 5].
5. Because the apps differ in size, they also report numbers per KLOC (per 1,000 lines of code) **[V]** [mota2026comparing, p. 2].

6. They grouped SonarQube's 39 issue tags into five clusters of their own design **[V]** [mota2026comparing, p. 3].

## Data and setup

- 9 web apps, 3 per platform: Lovable, v0 and Replit, generated 6–21 May 2026 on free tiers **[V]** [mota2026comparing, p. 2].
- Code size (NCLOC, non-comment lines of code): Lovable 17,081, v0 27,583, Replit 48,411 **[V]** [mota2026comparing, p. 5].
- 1,107 SonarQube issues in total: 390 (Lovable), 254 (v0), 463 (Replit) **[V]** [mota2026comparing, p. 5].
- Only three runs per tool, because of free-tier token limits and time **[A]** [mota2026comparing, p. 6].
- SonarQube version and rule profile are not reported **[I]** [mota2026comparing, p. 2].
- No statistical tests; the analysis is descriptive **[I]** [mota2026comparing, p. 6].
- Prompt, generated code and reports are shared in an artefact package **[V]** [mota2026comparing, p. 6].

## Key findings

- **[V]** Per 1,000 lines, Lovable had 22.83 issues vs 9.21 (v0) and 9.56 (Replit), and 220.77 minutes of fix effort vs 34.12 and 34.87 [mota2026comparing, p. 5].
- **[V]** Almost all of Lovable's issues were code smells (389 of 390), mostly minor (85.6%). One rule about deprecated (outdated) APIs, S1874, alone accounted for 204 occurrences [mota2026comparing, p. 5].
- **[V]** Replit wrote the most code, with about 60% duplicated lines vs at most 7% for v0 and 0% for Lovable, and the highest share of major issues (46%) [mota2026comparing, p. 5; mota2026comparing, p. 6].
- **[V]** SonarQube "bugs" (rule-based warnings, not tested failures) were few: 1 (Lovable), 5 (v0), 8 (Replit); security hotspots were 6, 10 and 30 [mota2026comparing, p. 5].
- **[V]** Results varied a lot within one tool: one Lovable app had 59 issues and 215 minutes of effort, the other two had 167 and 164 issues [mota2026comparing, p. 3].
- **[A]** The authors see v0 as "the most balanced alternative" [mota2026comparing, p. 6].
- **[A]** They suggest the outdated APIs come from AI models over-learning API patterns common at training time; this explanation is not tested in the study [mota2026comparing, p. 6].

## Limitations and weaknesses

- **[A]** SonarQube cannot definitively measure code quality; it is an "auxiliary indicator" [mota2026comparing, p. 6].
- **[A]** Three projects per tool are open to unusual outputs, as Lovable's spread shows (standard deviation 62.1 vs 7.0 and 20.3) [mota2026comparing, p. 6].
- **[A]** The platforms are opaque; free tiers might use weaker models at busy times, which the authors call speculation [mota2026comparing, p. 6].

- **[I]** With n = 3 per tool, tool differences cannot be separated from run-to-run chance. One outlier app changes the ranking by raw issue count, though not by fix effort [mota2026comparing, p. 3].
- **[I]** Nobody checked that the apps run or do what the prompt asked, yet per-line comparisons assume they deliver the same features [mota2026comparing, p. 2].
- **[I]** Template and boilerplate code was not separated from generated code, so some issues (for example outdated APIs, or Replit's duplication) may come from starter templates [mota2026comparing, p. 2].
- **[I]** The text calls Replit's issues the most "severe", yet Replit is the only tool with no critical issues; "severe" there means major [mota2026comparing, p. 4].
- **[I]** The authors state that Claude generated all figures and tables; we re-checked the main table's per-line values and they are consistent [mota2026comparing, p. 6].

## Gaps and future work this paper leaves open

- **[A]** The authors call for more runs per tool, premium tiers and other static analysis tools [mota2026comparing, p. 6].
- **[I]** It measures quality but not fidelity, and has no original app or human-written baseline. Adding both is exactly G1.
- **[I]** It relies on one instrument (SonarQube) that its own authors call auxiliary, which feeds G3.
- **[I]** It runs platforms autonomously from one prompt, with no human steering or follow-up prompts. Comparing real human-steered vibe coding with autonomous runs is G6.

## Why it matters for your research

This is the design closest to your topic: commercial vibe-coding platforms building a full web app from a plain-language description **[I]**. You can reuse its setup almost directly: same prompt to several platforms, record tool versions and dates, share the generated code. To turn it into a reconstruction study, add three things it lacks: an original app to rebuild, tests that check whether the rebuild behaves like the original, and more runs per tool. Also separate template code from generated code, and use more than one quality tool.

## Connections to other papers

- **[V]** he2026speed also reports rising SonarQube warnings with an AI coding tool, +30.26% after Cursor adoption [he2026speed, p. 7]. **[I]** Both point to quality costs, but in very different settings (9 fresh apps vs hundreds of real repositories).
- **[V]** molison2025llm, also using SonarQube, found far fewer bugs in GPT-4o code than in human code [molison2025llm, p. 8]. **[I]** Mota found few bugs too, but many code smells, so the conclusion depends on which SonarQube category you look at.
- **[A]** zhu2026understanding argues that SonarQube mixes superficial linting with real design flaws [zhu2026understanding, p. 2]. **[I]** This is a reason to read Mota's minor-smell counts with care.

- **[A]** Mota writes that productivity gains "do not necessarily reflect structural quality" [mota2026comparing, p. 5]. **[V]** This matches practitioners' view of vibe-coded output as "fast but flawed" [fawzy2026vibe, p. 8].

# Understanding the Code and Architectural Smells Generated by LLMs and Autonomous Agents

> **Takeaway:** On small tasks, AI models write code with different kinds of "smells" (signs of hard-to-maintain design) than humans do, and in apps built by an AI agent team the number of architecture smells grows almost in step with code size (ρ = 0.94). The evidence is small and exploratory, so use it to form hypotheses, not conclusions [zhu2026understanding, p. 8].

| Item | Detail |
|---|---|
| Authors | Yuecai Zhu, Nikolaos Tsantalis, Peter C. Rigby |
| Year · Venue | 2026 · Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering (ASE 2026) |
| Status | Peer-reviewed (our copy is the arXiv v1 preprint under a different title, "AI-Generated Smells: An Analysis of Code and Architecture in LLM- and Agent-Driven Development"; the published version may differ) |
| Identifier | DOI 10.1145/3832783.3834413 (arXiv 2605.02741) |
| Paper type | Exploratory empirical study (two static-analysis experiments) |
| Relevance to us | High — RQ3 (main), RQ1, RQ4 |

## In plain words

Code can work and still be badly designed, which makes it hard to change later. Such design problems are called code smells; at the level of how files and modules fit together they are called architectural smells. This paper asks which smells AI-written code contains and whether things get worse when AI builds whole applications instead of small functions. The authors first compare five AI models with humans on small puzzle tasks. Then they let MetaGPT, a team of cooperating AI agents, build five small Python apps such as games, a flashcard tool and a bookstore back-end. This is the closest study in our set to measuring the design quality of AI-built applications.

## How they did it

1. Experiment I (small tasks). Five models (Gemini 2.5 Pro, Llama 3.3 70B, deepseek-coder-v2 16B, qwen3-coder 30B and 480B) solved 90 programming problems from the CodeContests dataset **[V]** [zhu2026understanding, p. 3; zhu2026understanding, p. 4].
2. Each model got a zero-shot prompt (just the task) and a few-shot prompt (adding a role, output rules and a code template). One human solution per problem served as the comparison **[V]** [zhu2026understanding, p. 4].

3. Experiment II (whole apps). MetaGPT, with Qwen-Coder 480B as its model, built five apps: Crossy Road game, Flashcard app, Pacman, Personal Finance and an Online Bookstore back-end **[V]** [zhu2026understanding, p. 5].
4. Each app was requested at four stages of increasingly detailed requirements **[V]** [zhu2026understanding, p. 5].
5. The prompts had to demand object-oriented, modular files, because early trials produced one giant "God Class" script **[V]** [zhu2026understanding, p. 5].
6. All code was scanned with PyExamine, a Python smell detector that reports code-level, structural (class-level) and architectural smells **[V]** [zhu2026understanding, p. 3; zhu2026understanding, p. 4].
7. In Experiment II, smell counts were related to code size (lines of code and number of files) with Spearman's ρ (a rank correlation from −1 to 1), and to scenario and stage with the Kruskal-Wallis test (a test for differences between groups) **[V]** [zhu2026understanding, p. 8; zhu2026understanding, p. 9].

## Data and setup

- Experiment I: 90 problems, 5 models, 2 prompts, plus 90 human solutions; one Python file per solution **[V]** [zhu2026understanding, p. 4].
- Experiment II: 5 scenarios × 4 stages. The number of repositories (about 20) and runs per prompt are not stated; our estimate of about 20 comes from the reported p-values **[I]** [zhu2026understanding, p. 9].
- Two smell types (Feature Envy, Potential Shotgun Surgery) were dropped after a manual check found many false alarms; the size of that check is not reported **[V]** [zhu2026understanding, p. 6].
- Temperature, model hosting and the PyExamine version are not reported **[I]** [zhu2026understanding, p. 4].
- Functional correctness of the generated code is not reported anywhere in the paper; the apps were not tested, as far as the paper says **[I]** (our check of all pages).
- Replication package on Zenodo **[V]** [zhu2026understanding, p. 10].

## Key findings

- **[V]** Larger models wrote more Long Method smells than humans: Qwen Coder 480B 11, Qwen Coder 30B 7, Gemini 2.5 Pro 5, deepseek 2, Llama 0, human 1, across 90 problems [zhu2026understanding, p. 7].
- **[V]** Temporal Field (a field used only some of the time) appeared only in human code (4 instances) [zhu2026understanding, p. 7].
- **[V]** Few-shot prompting with coding rules did not help: Long Method went from 11 to 13 (Qwen 480B) and from 5 to 8 (Gemini) [zhu2026understanding, p. 7].
- **[V]** In the agent-built apps the most common smells were Too Many Branches, High Response for a Class, Potential Improper API Usage, Scattered Functionality and Unstable Dependency [zhu2026understanding, p. 8].

- **[V]** Code size tracked smell counts: ρ = 0.53 for code-level, 0.59 for structural and 0.94 for architectural smells; number of files vs architectural smells ρ = 0.72 [zhu2026understanding, p. 9].
- **[V]** More detailed requirements had no significant effect on any smell level ($p > 0.8$); the scenario affected architectural smells ($p = 0.043$) [zhu2026understanding, p. 9].
- **[A]** The authors name a "Volume-Quality Inverse Law" and a "Modular Mirage" (files look separate but behaviour is scattered), and compare agents to "junior developers" [zhu2026understanding, p. 8; zhu2026understanding, p. 9].

## Limitations and weaknesses

- **[A]** No repair loop; only 90 problems; only MetaGPT, so other tools may behave differently [zhu2026understanding, p. 9].
- **[I]** Experiment I rests on a handful of smell instances (for example 11 vs 1) with no statistical test and no adjustment for code length [zhu2026understanding, p. 7].
- **[I]** More code mechanically gives more smell instances. So ρ = 0.94 does not show that bigger code is worse per line; no smells-per-line analysis is reported [zhu2026understanding, p. 8].
- **[I]** The claim that "running code is just as likely to be structurally flawed as failed code" has no supporting correctness data [zhu2026understanding, p. 2].
- **[I]** The "longitudinal" and "architectural rot" framing is doubtful: the four stage prompts are self-contained, so they may be four separate builds rather than one evolving codebase [zhu2026understanding, p. 5].
- **[I]** About 20 apps, one model, one framework, no human baseline and 15 tests without correction [zhu2026understanding, p. 9].
- **[I]** Minor inconsistencies: research-question numbering changes between sections, and sampling is called both "random" and "stratified" [zhu2026understanding, p. 4; zhu2026understanding, p. 9].

## Gaps and future work this paper leaves open

- **[A]** The authors call for architecture-aware agents, refactoring agents, long-term evolution benchmarks and semantic-cohesion metrics [zhu2026understanding, p. 10].
- **[I]** Its apps have no original to compare with and no functional tests, so fidelity and maintainability are never measured together (G1).
- **[I]** Its choice of PyExamine over SonarQube, and its size-driven counts, show why we need to know whether quality instruments agree and predict real maintenance effort (G3).
- **[I]** It has no feedback or repair loop, so whether loops improve quality is open (G4).
- **[I]** CodeContests may be in the models' training data, which is not discussed (G5).

## Why it matters for your research

Experiment II is close to your setting: an AI agent builds complete small apps from plain-language requirements **[I]**. It gives you a list of smells to look for in AI-built apps, such as

unstable dependencies and scattered functionality. It also teaches two design lessons. First, always divide smell counts by code size, or you will mostly measure how much code was written. Second, measure whether the app works, otherwise claims about "correct but messy" code cannot be tested.

## Connections to other papers

- **[V]** Builds on molison2025llm: its first experiment mirrors that study's design [zhu2026understanding, p. 3]. **[A]** It argues that SonarQube-based results such as "LLMs write more maintainable code" mix superficial linting with real flaws [zhu2026understanding, p. 2].
- **[V]** Contrasts with molison2025llm, where GPT-4o code had far lower odds of SonarQube bugs than human code [molison2025llm, p. 8]. **[I]** Different tools and smell types are used, so the two do not directly contradict each other.
- **[V]** Supports a size pattern seen elsewhere: adopting Cursor raised lines added by 281.3% in the first month [he2026speed, p. 8]. **[I]** If smells follow size, such studies need size-normalised measures.
- **[V]** MetaGPT nearly always failed on the E2EDev benchmark [liu2026benchmarking, p. 6]. **[I]** Here it did produce complete apps, but only after modularity was forced in the prompt.

# Speed at the Cost of Quality: How Cursor AI Increases Short-Term Velocity and Long-Term Complexity in Open-Source Projects

> **Takeaway:** In real open-source projects, adopting the agentic coding tool Cursor gave a short burst of extra code, while code complexity rose and stayed higher. The quality result is not robust to every statistical method, so treat it as strong-but-contested evidence, not settled fact.

| Item | Detail |
|---|---|
| Authors | Hao He, Courtney Miller, Shyam Agarwal, Christian Kästner, Bogdan Vasilescu |
| Year · Venue | 2026 · Proceedings of the 23rd International Conference on Mining Software Repositories (MSR 2026) |
| Status | Peer-reviewed. Our copy is the arXiv v3 version (with appendices), so page numbers follow that copy |
| Identifier | DOI 10.1145/3793302.3793349 (arXiv 2511.04427) |
| Paper type | Empirical study (quasi-experimental causal analysis of GitHub repository data) |
| Relevance to us | High — RQ3 (main), RQ4, RQ5 |

## In plain words

Many developers now use AI coding agents that can edit many files on their own. People claim these tools make teams faster, but nobody knew whether that speed lasts or whether the code gets worse. The authors looked at real open-source projects on GitHub that started using Cursor, an AI-powered code editor with an "agent" mode that can plan, edit and debug code with the human mainly supervising [he2026speed, p. 3]. They compared these projects with similar projects that did not adopt Cursor, month by month. This matters because if AI speed is bought with harder-to-maintain code, the speed may not pay off later.

## How they did it

1. Find adopters. They searched GitHub for Cursor configuration files (such as .cursorrules). The first commit of such a file counts as the moment a project "adopted" Cursor [he2026speed, p. 4].
2. Find look-alike controls. For each adopter they picked similar projects that never adopted Cursor, using propensity-score matching (a statistical way to find projects with a similar history of activity, size and popularity) [he2026speed, p. 4; he2026speed, p. 5].
3. Measure every month. For each project and month they counted commits and lines added (speed), and ran SonarQube (a widely used code-checking tool) to get

static-analysis warnings, code duplication and cognitive complexity (a score for how hard code is to understand) [he2026speed, p. 5].

4. Compare before and after. They used difference-in-differences (comparing how adopters changed over time against how controls changed, so shared trends cancel out). The main version is the Borusyak imputation estimator, chosen because simpler methods can be biased when projects adopt at different times [he2026speed, p. 6].
5. Test how speed and quality feed into each other. A second model (Arellano-Bond dynamic panel, a method for data where this month depends on last month) checked whether more complexity or warnings slow down later work [he2026speed, p. 6; he2026speed, p. 8].
6. Robustness checks. They re-ran the analysis with two other estimators and on many subsets [he2026speed, p. 9; he2026speed, p. 16].

## Data and setup

- 806 GitHub projects with at least 10 stars that adopted Cursor, with adoption dates between Jan 2024 and Mar 2025 and most adoptions between Aug 2024 and Mar 2025 [he2026speed, p. 2; he2026speed, p. 4].
- 1,380 matched control projects in the text; a table lists 801 treated and 1,172 control projects with 21,699 project-months [he2026speed, p. 2; he2026speed, p. 17]. The paper does not explain the difference.
- Monthly data from Jan 2024 to Aug 2025 [he2026speed, p. 5].
- Main languages: TypeScript 366, Python 118, JavaScript 60, Go 36, Rust 24 projects [he2026speed, p. 4].
- The comparison is against "current practice", not against no AI: controls may use Copilot or ChatGPT without leaving traces [he2026speed, p. 7].
- Replication package on Zenodo [he2026speed, p. 11].

## Key findings

- **[V]** Overall, lines added rose by +28.58% and commits by +2.63% (not significant) [he2026speed, p. 7].
- **[V]** The speed boost is short-lived: lines added +281.3% in month 1 and +48.4% in month 2, then no significant effect [he2026speed, p. 8].
- **[V]** With the main estimator, static-analysis warnings rose by +30.26% and code complexity by +41.64% (both $p < 0.001$); duplication +7.03% was not significant [he2026speed, p. 7].
- **[V]** These quality effects depend on the estimator. Another method (Callaway–Sant'Anna) gives negative, non-significant results: warnings −10.49%, complexity −3.80%. The authors say the quality findings need caution [he2026speed, p. 16].
- **[V]** Once codebase size is controlled, Cursor has no significant direct effect on warnings, but it still raises complexity (coefficient 0.086, $p < 0.01$) [he2026speed, p. 8].
- **[V]** More complexity and warnings go with slower later work: a 100% rise in complexity or warnings goes with a 64.5% or 50.3% drop in next-month lines added [he2026speed, p. 8].

- **[A]** The authors read the warning rise as mostly "more code, more warnings", while complexity is inherently higher in Cursor code [he2026speed, p. 10].

## Limitations and weaknesses

- **[A]** Only projects that commit Cursor files are seen, and how much they really used Cursor is unknown. The results are "intent-to-treat" (the effect of adopting, not of using heavily) [he2026speed, p. 7].
- **[A]** Unmeasured differences (team skill, practices) may remain despite matching [he2026speed, p. 7].
- **[A]** The quality metrics may miss aspects of quality, and complexity metrics were designed for human code [he2026speed, p. 11].
- **[I]** Warnings and complexity are totals for the whole project, so they grow with project size. "+30% warnings" mainly means "more code", not more warnings per line (based on [he2026speed, p. 8]).
- **[I]** The abstract does not mention the estimator sensitivity reported in the appendix [he2026speed, p. 16].
- **[I]** Some numbers are inconsistent: the discussion cites a 25.1% complexity rise, while the tables report +41.64% and a coefficient of about 9% [he2026speed, p. 10; he2026speed, p. 7; he2026speed, p. 8]. Use the table values.
- **[I]** Quality is measured by static analysis only; no tests or functional correctness are measured.

## Gaps and future work this paper leaves open

- **[A]** Replicate with other and newer AI tools, in company (enterprise) settings and in more languages [he2026speed, p. 7; he2026speed, p. 11; he2026speed, p. 16].
- **[A]** Find out why AI tools produce overly complex code [he2026speed, p. 11].
- **[I]** G1: it does not study rebuilding an app from its interface or behaviour. A reconstruction study still has to measure fidelity and maintainability together.
- **[I]** G3: its quality story changes with the measuring method, which supports checking whether quality tools agree for AI-built apps.

## Why it matters for your research

This is the strongest real-world causal evidence in our set that agent-style coding can raise code complexity. It does not study reconstruction, so it informs your quality question (RQ3) by analogy only. Reuse its metric set: SonarQube warnings, cognitive complexity and duplication. Avoid its main weakness: always normalise counts by code size (for example per 1,000 lines), and check results with more than one method. When you quote "+30% warnings / +41.6% complexity", always add that they are estimator-sensitive.

## Connections to other papers

- **[V]** Contrasts with borg2026echoes: a randomised experiment found no significant difference when a second developer evolved AI-assisted code [borg2026echoes, p. 33;

borg2026echoes, p. 38]. Our evidence matrix treats the two as not necessarily in conflict (contradiction C1).

- **[A]** ehsani2026faster repeats this paper's +30% / +41% figures without the caveats; count the effect once, from this paper [ehsani2026faster, p. 18].
- **[V]** Consistent with the idea that AI adds code volume: AI-assisted outliers of up to 10,788 lines in borg2026echoes [borg2026echoes, p. 30], and architectural smells that track code size in zhu2026understanding [zhu2026understanding, p. 8].
- **[V]** Uses a different meaning of "vibe coding" from fawzy2026vibe [fawzy2026vibe, p. 1], so results should not be pooled (C9).

# Echoes of AI: Investigating the downstream effects of AI assistants on software maintainability

> **Takeaway:** When a second developer had to change code that someone else wrote with an AI assistant, they were not clearly slower and the code quality was not clearly worse. The study can only rule out large effects, and it used late-2024 assistants, not today's coding agents.

| Item | Detail |
|---|---|
| Authors | Markus Borg, Dave Hewett, Nadim Hagatulah, Noric Couderc, Emma Söderberg, Donald Graham, et al. |
| Year · Venue | 2026 · Empirical Software Engineering (journal) |
| Status | Peer-reviewed; preregistered (registered report) |
| Identifier | DOI 10.1007/s10664-026-10889-1 |
| Paper type | Controlled experiment (two phases; Phase 2 is a randomised controlled trial) |
| Relevance to us | High — RQ3 (main), RQ4, RQ5 |

## In plain words

Most studies ask whether AI helps developers write code faster. This paper asks a different question: is AI-assisted code harder for the next person to change? That is what "maintainability" means here, how easily another developer can later evolve the code [borg2026echoes, p. 1; borg2026echoes, p. 2]. The authors ran a two-step experiment with mostly professional developers. One group wrote a feature, some with AI and some without, and a second group then extended that code without AI. This matters because a rebuilt app is only useful if people can keep working on it.

## How they did it

1. Preregistration. The plan, hypotheses and analysis were fixed and published before data collection (a registered report) [borg2026echoes, p. 2; borg2026echoes, p. 9].
2. Phase 1 (write). Participants added a feature to a small Java web app, with or without AI. Who used AI was decided by their own preference and habits, not by chance, so Phase 1 is observational [borg2026echoes, p. 10; borg2026echoes, p. 43].
3. Phase 2 (evolve). New participants, all without AI, were randomly given a Phase 1 solution, either AI-assisted (treatment) or not (control), and asked to add another feature. This is a randomised controlled trial (RCT: groups are assigned by chance, so differences can be attributed to the treatment) [borg2026echoes, p. 2; borg2026echoes, p. 12].

4. Measure. They recorded completion time, CodeHealth (a 1–10 maintainability score from the tool CodeScene), test coverage (share of code run by tests) and perceived productivity (a questionnaire) [borg2026echoes, p. 13; borg2026echoes, p. 14].
5. Analyse two ways. Classic significance tests plus Bayesian models (which give the probability that an effect is above or below zero), adjusted for Java skill and habitual AI use [borg2026echoes, p. 14; borg2026echoes, p. 15; borg2026echoes, p. 16].

## Data and setup

- Base system: RecipeFinder, a deliberately imperfect Java/Spring Boot web app of about 2,000 lines (47 files) [borg2026echoes, p. 64; borg2026echoes, p. 65].
- 449 valid sign-ups, 92.2% professional developers [borg2026echoes, p. 24].
- 151 valid participants overall; Phase 2 had 75 (40 treatment, 35 control) [borg2026echoes, p. 19; borg2026echoes, p. 30].
- Tools used in Phase 1: GitHub Copilot 21, ChatGPT 13, Cursor 9, JetBrains AI 5, Claude 4 [borg2026echoes, p. 27].
- Data collected Oct 2024 to Jan 2025, before agentic tools were common [borg2026echoes, p. 3; borg2026echoes, p. 17].
- Automated checks ran in GitHub Actions: 3 acceptance tests per task, CodeScene and JaCoCo coverage [borg2026echoes, p. 16; borg2026echoes, p. 17].
- Data and scripts are shared on Zenodo and GitHub [borg2026echoes, p. 74].

## Key findings

- **[V]** Phase 2 completion time: median 136 min (evolving AI code) vs 173 min (control); negligible effect, p = 0.56 [borg2026echoes, p. 33].
- **[V]** Phase 2 CodeHealth: 8.49 vs 8.44, p = 0.24, not significant [borg2026echoes, p. 38].
- **[V]** One Bayesian subgroup effect: code first written with AI by habitual AI users scored +0.10 CodeHealth after evolution (98% probability of a positive effect). This small effect favours AI [borg2026echoes, p. 39].
- **[V]** Developer skill mattered more: higher Java skill in Phase 1 gave +0.16 CodeHealth in Phase 2 [borg2026echoes, p. 40].
- **[V]** Test coverage (median 0.70 in both groups, p = 0.49) and perceived productivity (p = 0.37) did not differ [borg2026echoes, p. 35; borg2026echoes, p. 42].
- **[V]** Phase 1 (not randomised): AI users finished 30.7% faster (p = 0.0029) [borg2026echoes, p. 43].
- **[V]** Phase 1 AI-assisted code introduced more code duplication (23% vs 11%) and more complex conditionals (13% vs 5%), and some AI users added up to 10,788 lines [borg2026echoes, p. 30; borg2026echoes, p. 46].
- **[A]** The authors see code bloat and "cognitive debt" as the main risks, not worse file-level quality [borg2026echoes, p. 57; borg2026echoes, p. 58].

## Limitations and weaknesses

- **[A]** The trial is underpowered: 75 of 128 planned Phase 2 participants, so only large effects can be ruled out [borg2026echoes, p. 19; borg2026echoes, p. 61].
- **[A]** Short-term only, one small Java system, individual developers, no security analysis [borg2026echoes, p. 58].
- **[A]** Many people worked over several sittings, so some times are self-estimated [borg2026echoes, p. 18].
- **[I]** Phase 1 groups were chosen by preference, so "AI code" also means "code from people who like AI".
- **[I]** CodeHealth is a product of the first author's employer and was customised for this study; no open tool was used for the main outcome [borg2026echoes, p. 13; borg2026echoes, p. 75].
- **[I]** Some numbers are inconsistent: the habitual-user speedup is 55.9% in the abstract but about 60% in the results [borg2026echoes, p. 1; borg2026echoes, p. 33].
- **[I]** Code volume, the risk the authors highlight, was not an outcome of the randomised phase.

## Gaps and future work this paper leaves open

- **[A]** Test third-generation coding agents (such as Claude Code), second developers who also use AI, security, and long-term effects [borg2026echoes, p. 63].
- **[I]** G1: this "second developer changes the code" design has never been applied to reconstructed apps [borg2026echoes, p. 13].
- **[I]** G3: its static measures and its time measure did not move together, so we still do not know which maintainability tools predict real effort.

## Why it matters for your research

This is the only randomised evidence in our set on whether AI-assisted code is harder to maintain. Its design is directly reusable: give a rebuilt app and the original to a second developer (or an agent), ask for the same small change, and compare time and quality. Copy its preregistration and power analysis habits. Avoid its weak spots: plan enough participants, use an open static-analysis tool alongside any vendor metric, and measure code size as an outcome.

## Connections to other papers

- **[V]** Contrasts with he2026speed, where Cursor adoption raised project-level complexity [he2026speed, p. 7]. The tools, scale and metrics differ, so the two need not conflict (evidence matrix C1).
- **[V]** Its Phase 1 finding of large AI code volume fits with he2026speed's +281.3% lines added in month 1 [he2026speed, p. 8].
- **[A]** ehsani2026faster notes that technical-debt tools disagree on what counts as debt, which matters because this paper relies on one tool [ehsani2026faster, p. 5].

- **[V]** Like borg2026echoes, asdaque2026novice finds that the human's experience shapes outcomes with AI code [asdaque2026novice, p. 3].

# Vibe Coding: An experiment with test-driven development

> **Takeaway:** On two tiny Python tasks, an AI agent produced the most correct code fastest, but its code was more complex. Humans working with an AI were not better than humans alone. The agent also used much newer AI models, so the comparison is not fair.

| Item | Detail |
|---|---|
| Authors | Moritz Mock, Barbara Russo |
| Year · Venue | 2026 · Empirical Software Engineering (journal; printed volume dated 2027) |
| Status | Peer-reviewed |
| Identifier | DOI 10.1007/s10664-026-10950-z |
| Paper type | Small controlled experiment with professionals, plus exploratory repeated runs of automated and agentic workflows |
| Relevance to us | Medium — RQ1, RQ2, RQ3, RQ4, RQ5 (mainly design lessons) |

## In plain words

The authors ask how humans and AI chatbots can work together as peers when writing code [mock2026vibe, p. 1; mock2026vibe, p. 2]. They use test-driven development (TDD: you write tests first, then the code that makes them pass). They compared four ways of working on the same small tasks: a human alone, a human writing tests while the AI writes code, a script letting one AI do everything, and an AI agent platform [mock2026vibe, p. 7; mock2026vibe, p. 8]. Here "vibe coding" means the human-plus-AI version, with fixed prompts the human could not change [mock2026vibe, p. 15]. This matters because it is one of the few experiments that labels a condition "vibe coding" and compares it with humans and agents.

## How they did it

1. Define four workflows: solo (human only), collaborative (human writes tests, AI writes code; the paper's "vibe coding"), fully automated (scripts drive one AI to write both tests and code), and agentic (the MetaGPT X platform) [mock2026vibe, p. 7; mock2026vibe, p. 8; mock2026vibe, p. 17].
2. Give everyone the same two small tasks: centre one word, then two words, in a line of text [mock2026vibe, p. 12].
3. Build an answer key: 12 checks (assertions) derived from the specification by category-partition testing (a systematic way to cover each type of input) [mock2026vibe, p. 11; mock2026vibe, p. 12].
4. Run the human sessions. 16 professionals were split into solo and collaborative groups, with a 40-minute limit [mock2026vibe, p. 15; mock2026vibe, p. 16].
5. Run the automated workflows several times each [mock2026vibe, p. 42].

6. Measure correctness (share of the 12 checks passed), McCabe complexity (a count of decision paths in code), test coverage, code similarity, iterations and time [mock2026vibe, p. 14].
7. Compare groups with rank-based tests (Kruskal-Wallis, then one-sided Mann-Whitney), with Cliff's delta effect sizes and Holm-Bonferroni correction for multiple tests [mock2026vibe, p. 10; mock2026vibe, p. 11].

## Data and setup

- 16 professional developers from Italy, Denmark and Brazil: 9 solo, 7 collaborative [mock2026vibe, p. 15; mock2026vibe, p. 16].
- Many had less than 1 year of TDD or Python experience [mock2026vibe, p. 16].
- Collaborative and fully automated workflows used GPT-3.5 Turbo; agentic runs used ChatGPT 5 or Claude Sonnet 4 [mock2026vibe, p. 14; mock2026vibe, p. 17].
- 3 fully automated runs and 12 agentic runs (2 models x 2 prompt versions x 3 runs) [mock2026vibe, p. 42].
- Prompts told the AI not to use built-in functions; a failing test triggered up to five automatic re-prompts [mock2026vibe, p. 19; mock2026vibe, p. 20].
- Human time was self-reported; automated time was machine-measured [mock2026vibe, p. 13; mock2026vibe, p. 17].
- Replication package on GitHub [mock2026vibe, p. 43].

## Key findings

- **[V]** Correctness: agentic 94.58% of checks passed on average, versus fully automated 83.3%, solo 62.0% and collaborative 60.8% [mock2026vibe, p. 41; mock2026vibe, p. 42].
- **[V]** Collaborative vibe coding was not significantly different from solo coding in correctness [mock2026vibe, p. 25]. Human results ranged from 8.3% to 100% [mock2026vibe, p. 26].
- **[V]** Agentic code was more complex: mean McCabe 11.67 vs about 6.0–6.6 for the other workflows [mock2026vibe, p. 41; mock2026vibe, p. 42].
- **[V]** Agentic code had lower branch coverage by the answer-key tests (65.33%) than solo (83.89%) and collaborative (79.00%) code [mock2026vibe, p. 41; mock2026vibe, p. 42].
- **[V]** Agentic code kept the main solution but added helper methods, logging and validation [mock2026vibe, p. 24].
- **[V]** Agentic runs were fastest: 75 s on average vs 800 s (fully automated), 2,057 s (collaborative) and 2,266 s (solo) [mock2026vibe, p. 41; mock2026vibe, p. 42].
- **[A]** The authors call agentic solutions "potentially more difficult to maintain", but maintainability was not measured directly [mock2026vibe, p. 35].
- **[I]** Caution on statistics: for some comparisons the reported one-sided p-values point the wrong way relative to the effect sizes (for example Table 9 [mock2026vibe, p. 25] and Table 10 [mock2026vibe, p. 27]). The main conclusions match the group averages, but do not quote single p-values from this paper without this caveat.

## Limitations and weaknesses

- **[A]** Human and automated workflows were tested in different settings, so cross-comparisons need care [mock2026vibe, p. 36].
- **[A]** Few participants, so low statistical power; tasks are simple [mock2026vibe, p. 36; mock2026vibe, p. 37].
- **[A]** The agentic part is exploratory [mock2026vibe, p. 17].
- **[I]** Model confound: agents used GPT-5 and Claude Sonnet 4, humans' AI used GPT-3.5 Turbo. The agent's advantage cannot be separated from the newer model [mock2026vibe, p. 17].
- **[I]** The "vibe coding" condition used scripted prompts participants could not modify [mock2026vibe, p. 15], so it is not free-form vibe coding.
- **[I]** Repeated runs of one setup were treated like independent people in the tests.
- **[I]** McCabe complexity is the only maintainability measure.

## Gaps and future work this paper leaves open

- **[A]** Try other practices, AI models and agent platforms; test agents under different prompts [mock2026vibe, p. 17; mock2026vibe, p. 40].
- **[I]** G6: a fair human-steered vs autonomous comparison with the same model is still missing [mock2026vibe, p. 17].
- **[I]** G1: larger, application-level tasks with richer maintainability measures are needed.

## Why it matters for your research

Reuse its evaluation framework: an answer-key test suite derived from the specification for correctness, complexity and coverage for internal quality, and time and iterations for effort. That maps well onto measuring fidelity in a rebuilt app. Avoid its flaws: hold the AI model constant across conditions, use more than one maintainability measure, and use larger tasks. Do not cite its results as evidence about app reconstruction.

## Connections to other papers

- **[V]** Disagrees with liu2026benchmarking, where MetaGPT nearly always failed on larger web apps [liu2026benchmarking, p. 6]. Task size and model generation differ (evidence matrix C6).
- **[V]** Its higher agentic complexity echoes he2026speed's rise in complexity after adopting an agentic tool [he2026speed, p. 7], though at a tiny scale.
- **[V]** Its "vibe coding" is narrower than fawzy2026vibe's minimal-review prompting [fawzy2026vibe, p. 1]; do not pool (C9).
- **[V]** Like borg2026echoes, human results varied widely, and individual skill mattered [borg2026echoes, p. 40].

# Novice Developers Produce Larger Review Overhead for Project Maintainers while Vibe Coding

> **Takeaway:** On GitHub, AI-agent pull requests from less experienced users were bigger, accepted less often and needed more review than those from experienced users. For us, this means the human's experience is a variable to control when studying vibe coding.

| Item | Detail |
|---|---|
| Authors | Syed Ammar Asdaque, Imran Haider, Muhammad Umar Malik, Maryam Abdul Ghafoor, Abdul Ali Bangash |
| Year · Venue | 2026 · Proceedings of the 23rd International Conference on Mining Software Repositories (MSR 2026) |
| Status | Peer-reviewed (5-page paper; track not stated in the PDF). Our copy is the arXiv v1 version |
| Identifier | DOI 10.1145/3793302.3793597 (arXiv 2602.23905) |
| Paper type | Empirical study (repository mining) |
| Relevance to us | Low — RQ3 (indirect), RQ4, RQ5 |

## In plain words

Some people hope that AI agents let beginners do the work of experienced developers. This paper tests that idea with real open-source contributions. A pull request (PR) is a proposed code change that project maintainers review before accepting ("merging") it. The authors compared AI-agent PRs from low-experience and high-experience GitHub users [asdaque2026novice, p. 1]. They asked whether beginners' contributions are larger and harder to get accepted [asdaque2026novice, p. 2]. This matters because if beginners' AI code pushes more checking work onto reviewers, the speed gain is partly an illusion.

## How they did it

1. They used AIDev, a public dataset of pull requests created with AI agents such as Copilot, Codex, Claude Code, Cursor and Devin. They kept projects with more than 100 stars [asdaque2026novice, p. 2].
2. They removed bot accounts (usernames containing "bot" or matching known agent names) [asdaque2026novice, p. 2].
3. They gave each user an experience score: total commits divided by GitHub account age. The lower two quartiles (bottom half) became "low experience", the upper two "high experience" [asdaque2026novice, p. 2].
4. They measured each PR's size (commits, files changed) and its review effort (acceptance rate, days to merge, review comments) [asdaque2026novice, p. 2].

5. They compared groups with Mann-Whitney U tests (a comparison that does not assume normally distributed data) and Chi-square tests for acceptance, corrected for running many tests [asdaque2026novice, p. 2; asdaque2026novice, p. 4].
6. They read 15 low-experience feature PRs with the most review comments to see what caused friction [asdaque2026novice, p. 3].

## Data and setup

- AIDev subset: 33,596 PRs and 1,796 users before filtering [asdaque2026novice, p. 2].
- Final sample: 22,953 PRs from 1,719 users (860 low-experience, 859 high-experience) [asdaque2026novice, p. 2].
- PRs grouped into 11 categories (for example feature, fix, docs, chore) [asdaque2026novice, p. 2; asdaque2026novice, p. 3].
- No comparison with human-only PRs [asdaque2026novice, p. 2].
- Analysis in Python; replication package on GitHub [asdaque2026novice, p. 2; asdaque2026novice, p. 5].

## Key findings

- **[V]** Low-experience PRs contained 2.15x more commits; for feature PRs, 4.20 vs 1.58 mean commits [asdaque2026novice, p. 2; asdaque2026novice, p. 3].
- **[V]** They changed 1.47x more files [asdaque2026novice, p. 3].
- **[V]** They had a 31% lower acceptance rate; for documentation PRs, 75.39% vs 93.06% [asdaque2026novice, p. 3].
- **[V]** They took 5.16x longer from creation to merge; for chore PRs, 2.83 vs 0.61 days [asdaque2026novice, p. 3].
- **[V]** They drew 4.52x more review comments. But both groups averaged under 1 comment per PR in every category, so the big ratio rests on small numbers [asdaque2026novice, p. 3; asdaque2026novice, p. 4].
- **[A]** From 15 PRs, the authors name two friction sources: "Infrastructure Mismatch" (AI code that ignores build or CI rules) and "Integration Friction" (AI code missing system context) [asdaque2026novice, p. 3; asdaque2026novice, p. 4].
- **[A]** The authors conclude that low-experience users "focus on generating more code while shifting verification burden onto reviewers" [asdaque2026novice, p. 1].

## Limitations and weaknesses

- **[A]** Commit count "conflates activity with skill": someone experienced but rarely active on GitHub counts as a novice [asdaque2026novice, p. 4].
- **[A]** Results may not hold under a broader definition of vibe coding [asdaque2026novice, p. 4].
- **[A]** Project review policies may influence acceptance and comments [asdaque2026novice, p. 4].
- **[I]** The two groups may work on different projects and use different agents; neither is controlled.

- **[I]** It is unclear whether tests treat each PR as independent. Users made about 13 PRs each on average, so busy users could dominate [asdaque2026novice, p. 2].
- **[I]** Results are ratios of means without effect sizes or confidence intervals, and "31% lower" is not stated as relative or absolute [asdaque2026novice, p. 3].
- **[I]** No code quality is measured; review effort is only a proxy. The friction categories rest on 15 PRs and two examples.

## Gaps and future work this paper leaves open

- **[A]** Study industrial settings and follow developers over time [asdaque2026novice, p. 4].
- **[A]** Train developers in verification skills [asdaque2026novice, p. 4].
- **[I]** G6: it suggests operator experience matters, which a human-steered vs autonomous reconstruction study should measure or fix.
- **[I]** Add code-level quality measures (static analysis, defects after merge) to separate review friction from code quality.

## Why it matters for your research

This paper does not study app reconstruction or measure code quality, so its relevance is low. Its one useful lesson is design: if humans steer the AI in your study, record their experience, because it changes outcomes. A better skill measure than commit count is advisable, such as a short pre-test. Also avoid its statistical weak spot: if one person or app contributes many data points, use models that account for that grouping.

## Connections to other papers

- **[V]** Agrees with borg2026echoes that the human's skill matters: higher Java skill improved later code health more than AI use did [borg2026echoes, p. 40].
- **[V]** Its definition of vibe coding (supervised agent use) differs from fawzy2026vibe's minimal-review prompting [fawzy2026vibe, p. 1]; do not pool (evidence matrix C9).
- **[V]** Its higher review burden converges with practitioners' "fast but flawed" view [fawzy2026vibe, p. 8] (evidence matrix G-A8, rated weak).
- **[V]** Like he2026speed, it mines GitHub data for agent use [he2026speed, p. 4], but compares users rather than before and after adoption.

# Vibe Coding in Practice: Motivations, Challenges, and a Future Outlook – a Grey Literature Review

> **Takeaway:** Practitioners who write about vibe coding mostly do it for speed, and they mostly describe the result as "fast but flawed", often skipping quality checks. This is evidence about how people feel and behave, not a measurement of the code.

| Item | Detail |
|---|---|
| Authors | Ahmed Fawzy, Amjed Tahir, Kelly Blincoe |
| Year · Venue | 2026 · Proceedings of the IEEE/ACM 48th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP 2026) |
| Status | Peer-reviewed. Our copy is the arXiv v1 pre-acceptance version; the underlying sources are non-peer-reviewed grey literature |
| Identifier | DOI 10.1145/3786583.3786866 (arXiv 2510.00328) |
| Paper type | Systematic grey literature review with thematic analysis |
| Relevance to us | Medium — RQ1, RQ3 (perceptions only), RQ5 |

## In plain words

"Vibe coding" is a new term, and most of what we know about it comes from blogs and forums, not research papers. The authors define it as building software mainly by describing goals to an AI in plain language, prompting again and again, and barely reviewing the code [fawzy2026vibe, p. 1]. They read 101 first-hand online accounts and asked four questions. Why do people vibe code, what is it like, what do they think of the code, and how do they check quality [fawzy2026vibe, p. 2]? This matters for us because your project title says "through vibe coding", and this paper describes what that workflow looks like in practice.

## How they did it

1. They ran a grey literature review (a systematic review of non-peer-reviewed sources such as blogs and forum posts), following published guidelines [fawzy2026vibe, p. 3].
2. They built a Google search string in six stages, switching to the words practitioners actually use, and checked it against a small set of known key posts [fawzy2026vibe, p. 3; fawzy2026vibe, p. 4].
3. They screened results until no new themes appeared, and also followed links from the first 100 results (snowballing) [fawzy2026vibe, p. 4].
4. Each source was scored 0–3 on five quality criteria (authority, evidence, objectivity, currency, purpose); only sources scoring 10 or more out of 15 were kept [fawzy2026vibe, p. 4; fawzy2026vibe, p. 5].

5. They cut each source into "behavioural units" (quotes describing one behaviour or opinion) and grouped them into themes per question. This is thematic analysis (finding recurring patterns in text) [fawzy2026vibe, p. 5].
6. Results are counts and percentages of quotes per theme [fawzy2026vibe, p. 6; fawzy2026vibe, p. 8].

## Data and setup

- 154 sources retrieved, 101 included; 40 dropped for low quality and 13 by the inclusion rules [fawzy2026vibe, p. 5].
- 518 behavioural units: 140 on motivation, 132 on experience, 114 on perceived code quality, 132 on quality assurance [fawzy2026vibe, p. 5].
- Inclusion: English, 2022–2025, named author, public full text, and evidence of vibe-coding behaviour [fawzy2026vibe, p. 4].
- Coding was done mainly by the first author, with a subset cross-checked by a co-author [fawzy2026vibe, p. 4; fawzy2026vibe, p. 5].
- No code was inspected; no tools or models were tested.
- Replication package on Zenodo [fawzy2026vibe, p. 10].

## Key findings

- **[V]** Speed is the main reason: Speed & Efficiency is 87 of 140 motivation units (62%); Accessibility & Empowerment 20 (14%) [fawzy2026vibe, p. 6].
- **[V]** The experience is mostly positive: Instant Success & Flow is 85 of 132 units (64%); Code Breakdown or Abandonment is 14 (11%) [fawzy2026vibe, p. 8].
- **[V]** The code is seen as "Fast but Flawed" in 78 of 114 units (68%) and "Fragile or Error-Prone" in 22 (19%); only 3 units (3%) call it high quality [fawzy2026vibe, p. 8].
- **[V]** Quality checks are often weak: Skipped QA 48 of 132 units (36%), Uncritical Trust 24 (18%), Delegated QA to AI 13 (10%), Reprompting Instead of Debugging 6 (5%) [fawzy2026vibe, p. 8].
- **[A]** The authors describe a "speed–quality trade-off paradox": people knowingly accept flawed code to move fast [fawzy2026vibe, p. 9].
- **[A]** They warn of a "new class of vulnerable developers" who can build but not debug, with reported apps lacking login protection or containing hardcoded secrets [fawzy2026vibe, p. 9].

## Limitations and weaknesses

- **[A]** The search cannot capture all grey literature, and some quality bias remains [fawzy2026vibe, p. 10].
- **[A]** Only people who chose to post online are included, so results do not generalise to all vibe coders [fawzy2026vibe, p. 10].
- **[A]** Theme coding is subjective and mostly done by one author [fawzy2026vibe, p. 10].
- **[I]** Percentages count quotes, not people, yet the discussion reads them as shares of people [fawzy2026vibe, p. 9]. One prolific source can contribute many quotes.

- **[I]** Accounts of careful developers who did not skip quality checks were excluded by design [fawzy2026vibe, p. 4]. This likely inflates the "Skipped QA" share.
- **[I]** Maintainability is barely covered: "Sloppy or Low Maintainability" is only 5 units (4%) [fawzy2026vibe, p. 8].
- **[I]** No inter-rater agreement statistic is reported.

## Gaps and future work this paper leaves open

- **[A]** Does vibe coding produce different defects and vulnerabilities than other AI-assisted or human coding? Which review practices work? How does practice change with experience [fawzy2026vibe, p. 10]?
- **[I]** G6: its human-steered prompting workflow has not been compared with autonomous agents in app reconstruction [fawzy2026vibe, p. 1].
- **[I]** G1: its "fast but flawed" perception has not been tested on measured fidelity and maintainability of rebuilt apps.

## Why it matters for your research

Use this paper for two things. First, its clear definition of vibe coding, which you can adopt or contrast in your methods. Second, its list of typical behaviours (re-prompting, run-and-see testing, letting the AI check itself), which a reconstruction study should either imitate or control. Do not use it as evidence that vibe-coded code is low quality: no code was measured. Weight it as perception evidence only.

## Connections to other papers

- **[V]** Other papers define "vibe coding" differently. asdaque2026novice uses a supervised agent workflow [asdaque2026novice, p. 1], and mock2026vibe uses fixed scripted prompts [mock2026vibe, p. 15]. Do not pool their results (evidence matrix C9).
- **[A]** Agrees with ehsani2026faster, which links vibe coding to "fast-integration debt", also from grey sources [ehsani2026faster, p. 17].
- **[V]** The perceived "fast but flawed" pattern converges with higher review burden for novice agent users [asdaque2026novice, p. 3] (evidence matrix G-A8, rated weak).
- **[V]** he2026speed gives measured, not perceived, evidence on quality after agentic tool adoption [he2026speed, p. 7].

# Faster Code, Deeper Debt? A Multivocal Literature Review on Technical Debt and Its Early Signs in LLM-Assisted Software Development

> **Takeaway:** This review maps what academics and practitioners say about long-term "technical debt" from AI-written code. Its clearest message for us is that there are no standard benchmarks or AI-specific metrics for this debt, and most evaluations only check whether code works.

| Item | Detail |
|---|---|
| Authors | Ramtin Ehsani, Shriya Rawal, Yuanfang Cai, Preetha Chatterjee |
| Year · Venue | 2026 · ACM Transactions on Software Engineering and Methodology (TOSEM) |
| Status | Peer-reviewed. Our copy is the arXiv pre-acceptance manuscript, so counts may differ slightly from the published version |
| Identifier | DOI 10.1145/3820165 (arXiv 2606.14796) |
| Paper type | Multivocal literature review (secondary study of papers plus blogs and industry reports) |
| Relevance to us | High — RQ3 (main), RQ4, RQ5; RQ1 (minor) |

## In plain words

"Technical debt" means shortcuts in code that save time now but make later changes harder, like a loan that must be repaid with interest. AI tools let people write code very fast, and many worry this creates hidden debt. The authors collected what both researchers and practitioners have written about this up to January 2026. They sorted the material into types of debt, ways to reduce it, tools to detect it, and benchmarks and metrics to measure it [ehsani2026faster, p. 3; ehsani2026faster, p. 7]. This matters because a rebuilt app must be maintained, and we need to know how debt can even be measured.

## How they did it

1. They followed published guidelines for a multivocal literature review (a review that includes "grey" sources such as blogs and company reports as well as peer-reviewed papers) [ehsani2026faster, p. 6].
2. One search string combined 13 kinds of debt (code, design, documentation and others) with AI terms such as "LLM", "ChatGPT" and "Copilot" [ehsani2026faster, p. 9].
3. Grey sources came from Google (first 10 result pages). Each was scored on 22 quality criteria, and only those scoring above 11 were kept [ehsani2026faster, p. 9].

4. Academic sources came from four digital libraries. Two authors screened them, then added papers by snowballing (following reference lists both ways). Only peer-reviewed work was kept [ehsani2026faster, p. 11; ehsani2026faster, p. 12].
5. Two authors extracted data from every source; a third resolved conflicts. Agreement was high (Cohen's kappa, a chance-corrected agreement score, of 0.93 for debt types) [ehsani2026faster, p. 12; ehsani2026faster, p. 13].
6. They coded each source against a known 13-type debt taxonomy and added new categories where needed. Results are counts of mentions, not measured effects [ehsani2026faster, p. 14; ehsani2026faster, p. 16].

## Data and setup

- 104 sources in total: 31 peer-reviewed papers and 73 grey sources [ehsani2026faster, p. 1; ehsani2026faster, p. 3].
- Academic search: 551 hits, 27 kept after screening, plus 4 from snowballing [ehsani2026faster, p. 11; ehsani2026faster, p. 12].
- Grey search: about 375,000 Google hits, 132 unique sources checked, 73 kept [ehsani2026faster, p. 9].
- Grey sources by tier: 38 company blogs, 18 developer platforms, 14 media outlets, 3 personal blogs [ehsani2026faster, p. 10].
- Search done in January 2026 [ehsani2026faster, p. 8].
- Supplementary data on figshare [ehsani2026faster, p. 4].

## Key findings

- **[V]** The most-mentioned debt types (academic / grey mentions): code 14 / 56, design 2 / 30, documentation 0 / 17 [ehsani2026faster, p. 16].
- **[V]** Six new AI-related debt types were proposed: governance (0 / 19), prompt (8 / 5), fast-integration (0 / 13), data (9 / 0), ethical (3 / 2) and provenance (1 / 1) [ehsani2026faster, p. 16].
- **[A]** "Fast-integration debt" means quickly adding AI output without checking it. The authors link it directly to vibe coding [ehsani2026faster, p. 17]. It rests on grey sources only.
- **[V]** The main advice in practice is human review: 58 of 73 grey sources recommend a human in the loop [ehsani2026faster, p. 20].
- **[A]** No standard benchmarks or datasets exist for debt in AI code; benchmarks focus "almost exclusively on binary correctness" (works or not) [ehsani2026faster, p. 23].
- **[A]** Existing tools such as SonarQube are seen as not designed for AI-specific debt, and no new metric is widely used [ehsani2026faster, p. 22; ehsani2026faster, p. 24].
- **[A]** Commercial AI-aware quality tools are rarely tested independently [ehsani2026faster, p. 28].
- **[A]** Reported from He et al. (not this review's own finding): Cursor adoption was followed by about +30% static-analysis warnings and +41% code complexity [ehsani2026faster, p. 18].

- **[I]** The review leaves out the original study's caveat: under another estimator these quality effects turn negative and non-significant [he2026speed, p. 16]. Cite he2026speed, not this review, for these numbers.

## Limitations and weaknesses

- **[A]** Search terms centred on "debt" may miss studies framed as maintainability or code quality [ehsani2026faster, p. 30].
- **[A]** Company blogs make up 52% of the grey sources, which may bias results [ehsani2026faster, p. 30].
- **[A]** English only; search-engine rankings are undocumented [ehsani2026faster, p. 30].
- **[I]** Counting how often something is mentioned is not the same as evidence that it happens. Two of the new debt types (governance, fast-integration) have no academic support at all [ehsani2026faster, p. 16].
- **[I]** No quality appraisal of the 31 academic papers is described, so weak position papers count like empirical studies [ehsani2026faster, p. 11; ehsani2026faster, p. 21].
- **[I]** The rule excluding correctness-only studies means maintainability comparisons that never say "debt" may be missing (for example, molison2025llm is not cited) [ehsani2026faster, p. 12].
- **[I]** It is about AI-assisted coding in general, not app reconstruction.

## Gaps and future work this paper leaves open

- **[A]** Build community benchmarks and new metrics for maintainability, adaptability and reproducibility across runs [ehsani2026faster, p. 29].
- **[A]** Run longitudinal studies of how AI-related debt builds up over time [ehsani2026faster, p. 25].
- **[I]** G1: its finding that benchmarks check correctness only supports our main gap, a joint check of fidelity and maintainability [ehsani2026faster, p. 23].
- **[I]** G3: it reports that debt tools disagree, which motivates testing whether maintainability tools agree for AI-built apps [ehsani2026faster, p. 5].
- **[I]** Its grey-only categories (fast-integration debt) could be tested empirically with controlled vibe-coding sessions.

## Why it matters for your research

Use this paper as a map, not as proof. Its debt taxonomy can structure the maintainability part of your study, and its tool list (SonarQube, ESLint, Pylint, CodeScene) shows common measuring options [ehsani2026faster, p. 23]. Cite it for the field-level gap: no standard maintainability benchmark for AI code. Do not cite it for effect sizes; for the Cursor numbers, cite he2026speed directly.

## Connections to other papers

- **[A]** Re-reports he2026speed's figures [ehsani2026faster, p. 18]; the primary study adds that the warning rise runs mainly through more code [he2026speed, p. 8]. Count the effect once.

- **[V]** Its "fast-integration" concern agrees with practitioners' "fast but flawed" view of vibe-coded output in fawzy2026vibe [fawzy2026vibe, p. 8]. Both rest on grey sources (evidence matrix G-A8).
- **[V]** Its claim that tools disagree fits the conflicting results from different instruments in our corpus, such as borg2026echoes' CodeHealth [borg2026echoes, p. 13] and he2026speed's SonarQube totals [he2026speed, p. 7] (C3).

# PropGen: Automated Property Generation for Property-Based Testing of Mobile Apps

> **Takeaway:** An AI model can write down valid behaviour rules of an Android app by actually using the app, but not from the app's description alone (912 of 985 rules valid vs 0 of 240). For us, this is a method for turning an original app's observed behaviour into tests a rebuild must pass [xiong2026propgen, p. 8; xiong2026propgen, p. 9].

| Item | Detail |
|---|---|
| Authors | Yiheng Xiong, Shiwen Song, Bo Ma, Ting Su, Xiaofei Xie |
| Year · Venue | 2026 · Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering (ASE 2026) |
| Status | Peer-reviewed |
| Identifier | DOI 10.1145/3832783.3837536 (arXiv 2604.13463) |
| Paper type | Tool paper with empirical evaluation |
| Relevance to us | Medium — RQ2, RQ4, RQ5 (method only; not RQ3) |

## In plain words

Many app bugs do not crash the app. The app just does the wrong thing, for example a setting that has no effect. Such bugs are hard to find because there is no "oracle" (a rule that says what the correct result is). Property-based testing helps: you write rules like "if I delete a note, it is no longer in the list", and a tool tries many random action sequences to see if a rule is broken. Writing those rules by hand is slow and expensive, and apps rarely have written specifications **[A]** [xiong2026propgen, p. 1]. PropGen lets an AI model explore a running Android app, record what it sees, and turn that evidence into such rules automatically.

## How they did it

1. Exploration. A multimodal LLM (an AI model that reads both images and text) looks at the screenshot, the numbered on-screen widgets, the app name and the screen (activity) names from the app's manifest file. It guesses which functions the app has, and PropGen tries each guess step by step **[V]** [xiong2026propgen, p. 4].
2. Evidence. Each tried action is saved as "behavioural evidence": the screen before, the action, the screen after, what changed, and whether it worked **[V]** [xiong2026propgen, p. 5].
3. Grounding. Widget names are filtered against a list of text strings taken from the app package by static analysis (reading the files without running them) **[V]** [xiong2026propgen, p. 5]. **[I]** So PropGen is grey-box: it mostly watches the running app, but also reads the manifest and a string list. The authors describe it as exploring "a black-box app" **[A]** [xiong2026propgen, p. 11].

4. Property writing. The model first writes a rule in plain language with three parts: precondition (when it applies), interaction (what the user does) and postcondition (what must be true after). It then turns the rule into code for the testing tool Kea **[V]** [xiong2026propgen, p. 6].
5. Refinement. When a human confirms that a reported "bug" is a false alarm, the model finds which of the three parts is wrong and fixes only that part **[V]** [xiong2026propgen, p. 6].

## Data and setup

- 12 open-source Android apps (for example OmniNotes, Markor, AnkiDroid, NewPipe), from 57,529 to 403,785 lines of code **[V]** [xiong2026propgen, p. 7].
- Model: GPT-5.2 (snapshot gpt-5.2-2025-12-11) at temperature 1.0. 3 hours per app for exploration and rule writing, then 6 hours of testing **[V]** [xiong2026propgen, p. 7].
- Comparisons: DroidAgent (another AI exploration tool), a naive baseline where the LLM writes 20 rules per app from the app's description only, and four bug-finding tools (Genie, Odin, PBFDroid, VisionDroid) **[V]** [xiong2026propgen, p. 7].
- Two graduate students, not co-authors, judged validity independently. Their agreement was high: Cohen's kappa (an agreement score where 1 is perfect) of 0.91, 0.82 and 0.81 **[V]** [xiong2026propgen, p. 7].
- No repeated runs are reported **[I]** [xiong2026propgen, p. 8].

## Key findings

- **[V]** PropGen found 1,282 app functions, of which 1,210 were valid (94.4%) and 977 were executed correctly (76.2%). DroidAgent found 575, with 491 valid (85.4%) and 187 correctly executed (32.5%) [xiong2026propgen, p. 8].
- **[V]** PropGen wrote 985 rules, and 912 were valid (92.6%; 78.7% to 98.8% per app) [xiong2026propgen, p. 8].
- **[V]** The naive baseline that only read the app description wrote 240 rules, and none were valid, because it referred to widgets that do not exist [xiong2026propgen, p. 9].
- **[V]** 127 rules caused false alarms; 118 of them (92.9%) were fixed, mostly by changing the postcondition (64) or precondition (53) [xiong2026propgen, p. 9].
- **[V]** The rules found 25 previously unknown functional bugs; 5 were fixed by developers at the time of writing [xiong2026propgen, p. 9].
- **[V]** Only 3 of the 25 bugs were found again by the four older tools. Those tools found 13 unknown bugs themselves, of which PropGen missed 10 [xiong2026propgen, p. 10]. **[A]** The authors call the approaches "complementary rather than substitutable" [xiong2026propgen, p. 10].
- **[V]** Cost was about $14.86 per app for PropGen vs $13.01 for DroidAgent [xiong2026propgen, p. 9].

## Limitations and weaknesses

- **[A]** Human judging can be subjective; only 12 apps were tested; the open-source apps may be in the model's training data [xiong2026propgen, p. 11].

- **[A]** The model sometimes invents functions that do not exist or invents widgets while writing rules [xiong2026propgen, p. 9].
- **[I]** Not fully automatic: refinement starts only after a human confirms a false alarm, and that human effort is not measured [xiong2026propgen, p. 6].
- **[I]** Single runs at temperature 1.0, so we do not know how stable the numbers are [xiong2026propgen, p. 7].
- **[I]** The naive baseline never sees the app, so it is a weak comparison [xiong2026propgen, p. 7].
- **[I]** Three of the four comparison tools (Genie, Odin, PBFDroid) come from the authors' own research group, and the scope check was validated by the same group [xiong2026propgen, p. 8]. Only VisionDroid is independent.
- **[I]** The contamination check is partial: showing that the bugs were not previously reported does not show that the model never saw how the apps behave [xiong2026propgen, p. 11].

## Gaps and future work this paper leaves open

- **[A]** The authors plan a larger and broader set of apps, better step planning, and combining PropGen with other test oracles [xiong2026propgen, p. 10; xiong2026propgen, p. 11].
- **[I]** Nobody has used properties mined from an original app to test an AI-rebuilt version of it. This would serve the fidelity side of G1.
- **[I]** Mobile apps and their hidden back-end behaviour are barely studied in reconstruction (G8); PropGen's runtime exploration could help there.
- **[I]** Testing six app builds released after the model's training cutoff **[A]** [xiong2026propgen, p. 11] is one of the few contamination controls in our corpus, relevant to G5.
- **[I]** Its rules use on-screen behaviour rather than the original's internal identifiers, which is relevant to G7.

## Why it matters for your research

PropGen does not build or rebuild apps, so it says nothing about code quality **[I]**. Its value is as a method. It shows how to recover what an app does from the running app, without source code or a specification. You could use the same idea to write tests from your original app and then check how many of them the rebuild satisfies. Its strongest lesson: an AI that has not observed the real app invents behaviour, so ground any test or rebuild in observed behaviour **[V]** [xiong2026propgen, p. 9]. Its evaluation design, with two independent annotators and a kappa score, is a good template.

## Connections to other papers

- **[V]** Like bai2026recreationworld, which uses accessibility and DOM tests [bai2026recreationworld, p. 9], it checks behaviour on the running app rather than in the code.
- **[V]** Its post-cutoff app builds are one of the few explicit contamination controls, next to yang2026programbench's no-internet setup and ablation [yang2026programbench, p. 6].

- **[I]** In our project it is the only paper that recovers a behaviour specification from a running app rather than building an app [xiong2026propgen, p. 4]. It is a measuring tool for the other papers' setting, not a competitor.
- **[V]** kim2026programdistill's test oracles use the original app's selectors (internal element identifiers) [kim2026programdistill, p. 35]. **[I]** PropGen's rules are instead grounded in visible widgets and observed behaviour, which may make them fairer to rebuilds that name things differently.

# Research Gaps and Opportunities

A research gap is an open question. The evidence must show that it is open, and a novelty check must fail to find a study that already answers it. A novelty check is a set of targeted searches that asks: "has someone already done this?" Our checks ran on 2026-11-12 in OpenAlex, Crossref, arXiv and web search. Semantic Scholar and Google Scholar were not searched, and web results came from US indexing only. So "we found no study" means only that these searches found none. It does not mean that no such study exists.

Papers found during the novelty checks are not part of our 23 analysed papers. We read only their abstracts (or, where noted, one HTML section). Statements about them are author claims **[A]**. We name them in plain text with their identifiers, so you can look them up.

How the gaps were ranked **[I]**: each gap got a score for evidence (1–3), novelty (1–3) and beginner feasibility (1–3). The three scores were multiplied. These scores are our own judgement.

| Gap | In short | Novelty verdict | Confidence | Beginner feasibility | Score |
|---|---|---|---|---|---|
| G1 (top) | Behavioural fidelity and validated maintainability of rebuilt apps, both against the originals | novel (narrow claim) | medium-high | moderate | 18 |
| G2 | Bias when the same AI family builds, tests and judges a benchmark | partially novel | medium | easy | 12 |
| G3 | Do maintainability tools agree, and do they predict real maintenance effort, for AI-built apps? | partially novel (narrowed) | medium | easy to moderate | 12 |
| G4 | Does feedback improve fidelity and code quality together in rebuilding? | partially novel | medium | moderate | 8 |
| G5 | How much does contamination inflate rebuild scores? | partially novel | medium-low | moderate | 8 |
| G6 | Human-steered vibe coding versus autonomous agents for rebuilding | partially novel | medium | moderate to hard | 8 |
| G7 | Bias from tests tied to the original's internal names | mostly addressed (narrow remainder) | low-medium | easy | 6 |
| G8 | Native mobile apps with hidden server behaviour | partially addressed | low | hard | 2 |

**[I]** G6 ranks low only because studies with human participants are harder for a beginner. It is the gap closest to the words "through vibe coding" in the project title, and it can be added to the G1 study as an optional extra arm.

## G1. Fidelity and maintainability of AI rebuilds, measured against the original (top recommendation)

> **Top recommendation.** No study we found rebuilds existing apps from what can be observed and then measures, on the same rebuilds, both how faithfully they behave and how maintainable they are, using the original app as the comparison.

**Why it is a gap.**

- **[V]** Rebuild benchmarks with behavioural tests describe the rebuilt code only by size and structure: 89.4% of rebuilds were smaller, with fewer files [bai2026recreationworld, p. 15]. **[A]** The authors call this "architectural substitution rather than implementation quality" [bai2026recreationworld, p. 16].
- **[V]** Rebuilt programs are shorter and use fewer files than the originals, but no quality or maintainability measure is applied [yang2026programbench, p. 9].
- **[V]** kim2026programdistill does not measure the code quality of rebuilds [kim2026programdistill, p. 14], and chen2025iwr scores only behaviour and appearance [chen2025iwr, p. 7].
- **[V]** Screenshot-to-code studies use rough proxies such as lines of code and reuse [wu2026looks, p. 4], or ratings by 5 developers [zhou2025declarui, p. 10].
- **[V]** The only maintainability score measured relative to an original is a single Maintainability Index, in a preprint that rebuilds from requirements, not from observed behaviour [pan2026functional, p. 3].
- **[V]** Whenever both are measured, passing tests and other quality dimensions go in different directions [molison2025llm, p. 8; zhao2025vibe, p. 6; wu2026looks, p. 7]. So passing tests says little about maintainability.
- **[A]** The Vibe Code Bench authors state that passing functional tests does not mean the code is maintainable, secure or documented [tran2026vibe, p. 9].
- **[V]** The one validated maintainability design in our papers, a second developer changing the code, has never been applied to rebuilds [borg2026echoes, p. 13].

**Novelty verdict.** Novel for the narrow claim. Confidence: medium-high. Measuring fidelity alone, or code quality alone, is not new. The combination, with the original as the baseline, is what we did not find (searches on 2026-11-12). Confidence is not "high" because we read the closest papers only at abstract level, most are preprints, and the field publishes every month.

Closest prior work (abstracts only, **[A]**):

- MirrorCode: AI can rebuild entire programs from behavior alone (arXiv 2606.30182, preprint). Agents rebuild 25 command-line programs without source code. No code-quality measure in the abstract.

- UI2App (arXiv 2607.06306, preprint). Multi-page web apps from screenshots only. No code-quality measure in the abstract.
- VISTA (arXiv 2605.26144, preprint). Full-stack web and Android apps from design hand-offs, with behaviour probes. No maintainability measure and no original app in the abstract.
- WebCoderBench (ACL 2026, DOI 10.18653/v1/2026.acl-long.535). Web apps from specifications. Its "maintainability" is comment rate plus linting and redundancy, with no original app.
- Is Agent Code Less Maintainable Than Human Code? (CodeThread; EMNLP 2026, DOI 10.18653/2026.emnlp-main.2528). Agents solve fewer follow-up tasks on agent code than on human code. Issue-level tasks, not app rebuilding.
- Trajectory-Level Security Debt in LLM Coding Agents (DOI 10.1609/aaaiss.v10i1.43533). Security scans of ProgramBench rebuilds, with no comparison to the originals.
- Software Aging in LLM-Generated Applications (arXiv 2608.26391, preprint) and SWE-CI (arXiv 2603.03823, preprint). Maintainability over time, but not for rebuilds.

**What you could do.** **[I]** Produce the first "fidelity and maintainability" profile of AI rebuilds. It answers a simple question: does a rebuild that behaves like the original also stay as easy to change as the original? It would also show whether smaller rebuilds are simpler, or just incomplete (contradiction C8).

**Suggested study design for a beginner.** **[I]**

1. Choose 8–10 small web apps (about 1,000–5,000 lines of code, with at most a simple back-end). Take about 5 from the released E2EDev set, which comes with validated tests [liu2026benchmarking, p. 9]. Take the other 3–5 from apps released after the AI models' training cutoff, or write them yourself.
2. Write browser tests (for example with Playwright) against each original. Find page elements by their role and visible text, not by internal names. Keep only tests that pass 100% on the original.
3. Pick 2–3 AI builders, for example one terminal coding agent and one commercial vibe-coding platform. Record their versions and the dates.
4. Give each builder only the running original and screenshots. Give no source code and no hidden identifiers.
5. Run each builder 3 times per app. This gives about 60–90 rebuilds.
6. Measure fidelity: the share of your tests that pass on each rebuild, plus a secondary visual score.
7. Measure maintainability on each rebuild and on its original, and report the ratio: SonarQube issues per thousand lines, cognitive complexity per function, duplication, and a second analysis tool.
8. Add a standard change task: two small feature changes, each with its own acceptance tests, done by a fixed "maintainer" AI agent (3 runs) on the original and on each rebuild. This follows the idea of borg2026echoes [borg2026echoes, p. 13]. An optional pilot with 4–6 student developers can follow its hand-off design.

9. Analyse: preregister your hypotheses, use mixed-effects models with the app as a random effect, report means with spread and bootstrap confidence intervals, and correlate fidelity with the maintainability ratio. Divide all counts by code size, because AI-assisted work changes code volume [he2026speed, p. 8].

**Feasibility: moderate.** The tools are free or have free tiers, and E2EDev provides apps and tests. The main effort is writing the tests for your own apps and managing 60–90 rebuilds. API costs are moderate. You will need help with the statistics.

**Risks.**

- Few apps, so results may not generalise.
- AI tools change between runs. Record versions and run all arms in the same week.
- Finite test suites give only a lower bound on fidelity [yang2026programbench, p. 12; bai2026recreationworld, p. 22].
- Static tools are not validated for AI code (see G3).
- An AI "maintainer" mixes up the AI's skill with how maintainable the code is.
- The E2EDev apps are public, so the AI may have seen them during training (contamination).

## G2. Same-family bias between benchmark builder, test writer and judge

> **Gap.** No study we found measures whether AI judges and AI-written tests in app-generation or rebuild benchmarks favour apps built by their own AI family.

**Why it is a gap.**

- **[V]** In some benchmarks the AI family that wrote the tests or tasks also topped the results: Claude wrote ProgramBench's tests and Claude models lead [yang2026programbench, p. 3; yang2026programbench, p. 31]; GPT-5 drafted Vibe Code Bench's specifications and a GPT model ranks first [tran2026vibe, p. 3; tran2026vibe, p. 6]. Similar patterns appear in kim2026programdistill and zhao2025vibe [kim2026programdistill, p. 2; zhao2025vibe, p. 17].
- **[V]** In other papers the overlap shows no visible advantage [lu2025webgen, p. 8; bai2026recreationworld, p. 11; wu2026looks, p. 13].
- **[V]** Agreement between AI judges and humans ranged from 36.1% to 86.4% depending on the judge model, reported as raw agreement without a chance correction [tran2026vibe, p. 9]. Some judges were tuned and tested on the same data [chen2025iwr, p. 17; si2025benchmarking, p. 17].
- **[I]** No paper in our set tests this possible self-preference directly.

**Novelty verdict.** Partially novel. Confidence: medium. Self-preference of AI judges and self-bias of benchmark generators are known problems, with ready-made methods to measure them. We found no study that measures them in app-generation or rebuild benchmarks (searches on 2026-11-12). The pattern could also reflect real ability differences, which is exactly what a study must separate.

Closest prior work ([A], abstracts):

- Silencer: From Discovery to Mitigation of Self-Bias in LLM-as-Benchmark-Generator (NeurIPS 2025, DOI 10.52202/085713-4317; arXiv 2505.20738). Models score higher on benchmarks they generated. Not app generation.
- Preference Leakage: A Contamination Problem in LLM-as-a-judge (arXiv 2502.01534, preprint). Judges favour generators from their own family.
- Play Favorites (arXiv 2508.06709, preprint) and Quantifying and Mitigating Self-Preference Bias of LLM Judges (arXiv 2604.22891, preprint). Methods to measure self-bias.
- WebDevJudge (arXiv 2510.18560, preprint). AI judges fall short of human experts on web-development quality.

**What you could do.** [I] Estimate how much benchmark rankings move when the judge's AI family changes, and recommend judging across families with chance-corrected agreement.

**Suggested study design for a beginner.** [I]

1. Download the released Vibe Code Bench apps and run records [tran2026vibe, p. 11]. Check first that they can be replayed.
2. Pick about 20 apps built by 4–6 models from different AI families.
3. Re-judge every workflow with browser-agent judges from three families (for example Claude, GPT and an open-weights model), using fixed prompts.
4. Have two human raters label a stratified sample of about 300 steps. This is your gold standard.
5. Estimate self-preference: a judge's score for apps from its own family minus its score for other families, after accounting for the human labels.
6. Report kappa agreement, ranking changes (Kendall tau) and the self-bias estimate with confidence intervals, using a logistic mixed model.

**Feasibility: easy.** No new apps need to be built. The cost is judge API calls and about 15–20 hours of human labelling.

**Risks.** The human sample may be too small to detect small biases. Commercial judges change over time. The released run records may not be enough to replay every app.

## G3. Do maintainability tools agree, and do they predict real maintenance effort, for AI-built apps?

> **Gap.** For whole AI-generated applications, no study we found checks whether several maintainability tools agree with each other after adjusting for size, and whether they predict how hard the code is to change.

**Why it is a gap.**

- [A] One study chose PyExamine over SonarQube's "superficial linting" [zhu2026understanding, p. 2]. A review notes that technical-debt tools disagree on what

counts as debt [ehsani2026faster, p. 5], and that SonarQube and general linters were not designed for LLM-specific debt [ehsani2026faster, p. 22].

- **[A]** Another study calls SonarQube only an "auxiliary indicator" [mota2026comparing, p. 6]. **[V]** The randomised trial used a customised CodeHealth score from the first author's employer [borg2026echoes, p. 13; borg2026echoes, p. 75].
- **[V]** Whether AI code looks better or worse changes with the tool and the unit: fewer SonarQube bugs [molison2025llm, p. 8], more repository warnings [he2026speed, p. 7] that run mainly through code size [he2026speed, p. 8], and smell counts that track code size closely [zhu2026understanding, p. 8].
- **[V]** The randomised maintenance outcome showed no difference [borg2026echoes, p. 33], while static measures in the study's first phase did show some [borg2026echoes, p. 30]. The two kinds of measure were not linked.

**Novelty verdict.** Partially novel, narrowed. Confidence: medium. A 2026 peer-reviewed paper already shows, for single code changes, that classic maintainability metrics do not explain agent maintenance outcomes, so we narrowed the gap to whole applications and to agreement between tools (searches on 2026-11-12).

Closest prior work (**[A]**, abstracts):

- Is Agent Code Less Maintainable Than Human Code? (CodeThread; EMNLP 2026, DOI 10.18653/2026.emnlp-main.2528). Partly fills the original question at the level of single issues.
- Beyond Functional Correctness: Static Code Quality Profiling of LLM-Generated Code (arXiv 2610.19167, preprint). Three tools on many single programs; no tool-agreement or human check in the abstract.
- Comparing Open Source with Software Code Generated by AI Tools from Software Maintainability Quality Factor Perspective (DOI 10.37394/232018.2025.13.58). Three tasks, no validation.

**What you could do.** **[I]** Give guidance on which static tools, if any, a rebuild study such as G1 should trust. It would also test the explanation we offer for contradictions C1 to C3.

**Suggested study design for a beginner.** **[I]**

1. Collect about 60 released AI-built full-stack apps from 4 or more models, for example from Vibe Code Bench [tran2026vibe, p. 11] and possibly E2EDev outputs [liu2026benchmarking, p. 9].
2. Run several tools on each app: SonarQube (issues per thousand lines, cognitive complexity), ESLint complexity rules, a Maintainability Index tool, a duplication tool, and optionally the free tier of CodeScene. Divide everything by code size.
3. Define two change requests per app and let a maintainer AI agent attempt them (3 runs). Record success and cost.
4. Ask 3 expert raters to rank a 20-app subsample by maintainability.
5. Measure agreement between tools (Kendall's W and pairwise Spearman correlations).
6. Use mixed models to test how well each tool predicts maintenance success, controlling for size and functional pass rate.

**Feasibility: easy to moderate.** Existing apps, free tools and scripted change tasks. The statistics are standard.

**Risks.** Agent success depends on the maintainer model. Apps from different models differ in features, not only in quality (control with the functional pass rate). Rater expertise varies.

## G4. Does feedback improve both fidelity and code quality in rebuilding?

> **Gap.** No controlled study we found tests whether feedback loops in app rebuilding improve behavioural fidelity and code quality together.

**Why it is a gap.**

- **[V]** In one preprint, self-repair, environment repair and planning each lowered the code-quality score without improving correctness, on a single model [pan2026functional, p. 8].
- **[V]** Browser self-testing goes together with better results, but this is only a correlation, and the system prompt demanded testing [tran2026vibe, p. 7; tran2026vibe, p. 16].
- **[V]** Targeted loops improve exactly what they check: compiling and navigation [zhou2025declarui, p. 10], and build success [wu2026looks, p. 12].
- **[A]** Agents' own checks missed behavioural differences from the reference [kim2026programdistill, p. 12].

**Novelty verdict.** Partially novel. Confidence: medium. Feedback effects on code quality are studied for code snippets and for apps built from specifications, and effects on functionality are studied for full-stack apps. We found no controlled rebuild study that compares feedback types on fidelity and maintainability together (searches on 2026-11-12). Nearby work is dense, so the novelty lies only in the combination.

Closest prior work (**[A]**, abstracts):

- From Runnable Code to Shippable Applications (TDDev; DOI 10.1145/3832783.3834388). Test-driven loops raise functional accuracy on 20 web apps; no code-quality outcome in the abstract.
- Static Analysis as a Feedback Loop (DOI 10.1109/scam67354.2025.00017; arXiv 2508.14419). Tool feedback improves snippets.
- Comparative Analysis of Iterative Prompt Refinement and Quality of LLM-Generated Code (SSRN, DOI 10.2139/ssrn.7435043, preprint) and a related SSRN preprint on security across iterations (DOI 10.2139/ssrn.6958668).
- Generation of Web Apps with Agentic IDEs (Zenodo DOI 10.5281/zenodo.19208142; arXiv 2608.20903). Five apps; iteration rarely adds regressions.
- MACBench (DOI 10.18653/2026.emnlp-main.2172) and FronTalk (arXiv 2601.04203). Multi-turn front-end generation.

**What you could do.** **[I]** Resolve contradiction C4: show whether feedback trades code quality for fidelity, or improves both.

**Suggested study design for a beginner. [I]**

1. Reuse the G1 pipeline. Start with 3 apps, then grow to 6.
2. Define four arms: no feedback; generic self-repair; behavioural-difference feedback (failing tests from the original shown to the agent); static-analysis feedback (SonarQube or ESLint findings).
3. Use one open agent harness, 2 models, 3 runs per arm, and the same iteration budget in every arm (6 apps give 144 rebuilds).
4. Measure the G1 fidelity and maintainability outcomes plus code size.
5. Analyse with a mixed model and planned comparisons against the no-feedback arm.

**Feasibility: moderate.** The scripting is straightforward, but the number of runs is large.

**Risks.** The behavioural-feedback arm sees the tests, so score it on a held-out test split. Equal iteration budgets may still use different amounts of tokens. Two models limit how far results generalise.

## G5. How much does contamination inflate rebuild scores?

> **Gap.** No study we found measures how much rebuild fidelity rises when the original app is public and well known, compared with matched apps the AI cannot have seen.

**Why it is a gap.**

- **[V]** Most reference apps are popular public projects. Explicit controls appear in only a few papers: an overlap audit [bai2026recreationworld, p. 18], no internet plus an ablation [yang2026programbench, p. 6], text overlap with the original [xiao2025benchmarking, p. 5], apps built after the model cutoff [xiong2026propgen, p. 11], and keeping task code off public repositories [borg2026echoes, p. 66].
- **[A]** pan2026functional cannot rule out that models saw the projects during training [pan2026functional, p. 2]. **[V]** Its example prompt names the original project [pan2026functional, p. 3], and interface hints raised one task from 0.17 to 0.67 [pan2026functional, p. 4].
- **[V]** With internet access, 20–36% of the stronger models' tasks were flagged as cheating, mostly by looking up the original source code [yang2026programbench, p. 6].

**Novelty verdict.** Partially novel. Confidence: medium-low. Building contamination-limited benchmarks is a known practice. We found no study that measures the size of the contamination effect on rebuild fidelity under the same black-box setup (searches on 2026-11-12). Matching public and private apps on difficulty is hard, so a null result would be hard to interpret.

Closest prior work (**[A]**):

- The Next Challenge for Agentic Cybersecurity: A Realistic, Contamination-Free Reverse Engineering Benchmark (SRE-Bench; arXiv 2608.11469, preprint). Private programs for binary reverse engineering, not app rebuilding.
- General-purpose work such as LiveBench (arXiv 2406.19314) and Benchmark Data Contamination of Large Language Models: A Survey (arXiv 2406.04244).

**What you could do.** [I] Estimate how far published rebuild scores may be inflated by memorisation. This is needed to interpret the fidelity numbers in RQ2.

**Suggested study design for a beginner.** [I]

1. Pick 5 popular public apps.
2. Build or find 5 matched apps of similar size and features that are private or released after the model cutoffs. Optionally add a "renamed clone" of a public app.
3. Give identical black-box inputs to 2 models, 3 runs each.
4. Measure fidelity with G1-style tests, text overlap with the original source, and whether the agent names the original project unprompted.
5. Report paired differences with bootstrap confidence intervals. This can run inside G1 as its public-versus-new app factor.

**Feasibility: moderate.** Writing 5 matched apps is the main cost; small apps that create, read, update and delete records are enough.

**Risks.** Difficulty may not match well. Apps you write yourself may be easier. The sample is small.

## G6. Human-steered vibe coding versus autonomous agents for rebuilding

> **Gap.** No study we found compares a person vibe-coding a rebuild of an existing app with an autonomous agent using the same AI model, scored on fidelity and maintainability.

**Why it is a gap.**

- **[A]** The ProgramBench authors name human-guided rebuilding as future work [yang2026programbench, p. 12].
- **[V]** Rebuild and generation benchmarks have no human-developer baseline [yang2026programbench, p. 20; tran2026vibe, p. 5; lu2025webgen, p. 7].
- **[V]** Practitioners describe vibe coding as plain-language prompting with little review [fawzy2026vibe, p. 1].
- **[V]** The only human-versus-agent comparison in our set used 2 tiny tasks, scripted prompts that participants could not change, and different model generations for humans and agents [mock2026vibe, p. 15; mock2026vibe, p. 17].

**Novelty verdict.** Partially novel. Confidence: medium. The value of human guidance in vibe coding has been shown for reproducing reference images as SVG code and for coding puzzles, but not for rebuilding software apps (searches on 2026-11-12). The closest work is strong and recent, and its authors could extend it.

Closest prior work (**[A]**, abstracts):

- Why Human Guidance Matters in Collaborative Vibe Coding (arXiv 2602.10473, preprint; data at OSF DOI 10.17605/osf.io/bzdrv). 737 participants; human-led instructions beat AI-led ones on reproducing reference images; code quality not measured.

- CentaurEval (arXiv 2512.04111, preprint). Humans versus LLMs at four levels of human intervention on coding templates.
- SWE-Together (arXiv 2606.29957, preprint) and Professional Software Developers Don't Vibe, They Control (arXiv 2512.14012, preprint).

**What you could do.** [I] Show whether the human vibe-coding workflow in the project title adds fidelity or quality beyond an autonomous agent with the same model, and at what cost in time.

**Suggested study design for a beginner.** [I]

1. Get ethics approval and recruit 12–16 students or junior developers (a pilot with about 8 is realistic).
2. Pick 2 small apps of similar size from the G1 set you wrote or that were released recently.
3. Condition A: the participant may only prompt a vibe-coding tool, sees the running original, and has 90 minutes.
4. Condition B: the same tool and model run autonomously from the same first prompt, 3 times.
5. Keep the AI model identical in both conditions, which avoids the model confound in mock2026vibe [mock2026vibe, p. 17].
6. Counterbalance which app each person does first.
7. Measure G1 fidelity and maintainability, time, number of prompts, and prompting strategy from logs.
8. Preregister, then analyse with paired Wilcoxon tests or mixed models, with effect sizes.

**Feasibility: moderate to hard.** It needs ethics approval and participant recruitment.

**Risks.** Small sample. Learning effects (counterbalance). Differences in participant skill (use a pre-test). The time limit may favour the agent.

## G7. How much do tests tied to the original's internal names distort scores?

> **Gap.** No study we found re-scores the same rebuilds with tests that use the original's internal names and with tests that do not, to measure how much this changes fidelity scores and rankings.

**Why it is a gap.**

- [V] E2EDev tests depend on hidden labels (data-testid) from the original, and these labels were given to the AI in the prompt [liu2026benchmarking, p. 16]. kim2026programdistill's tests use the original's selectors [kim2026programdistill, p. 35].
- [V] 6.5% of zhao2025vibe's security tests were tied to the reference implementation and had to be rewritten [zhao2025vibe, p. 19].
- [V] Interface hints change measured fidelity a lot [pan2026functional, p. 4], and some benchmarks give the AI a structural map as input [wu2026looks, p. 3; zhou2025declarui, p. 4].

**Novelty verdict.** Mostly addressed as a method; a narrow remainder is open. Confidence: low-medium. Several 2026 benchmarks already use tests that do not depend on internal names. The only open part is measuring the size of the distortion on the same rebuilds (searches on 2026-11-12), and any of the papers below could answer it as a side analysis.

Closest prior work (**[A]**):

- Do LLM-Generated Web Apps Behave Correctly? A Contract-Grounded Benchmark for Behavioral Conformance (ConformWeb; EMNLP 2026, DOI 10.18653/2026.emnlp-main.2018).
- VISTA (arXiv 2605.26144) and UI2App (arXiv 2607.06306), whose interaction metric credits any valid implementation.
- BackendForge (DOI 10.18653/2026.findings-emnlp.2236). Black-box tests against an API contract.
- In our set: implementation-independent tests in ProgramBench and RecreationWorld [yang2026programbench, p. 5; bai2026recreationworld, p. 9].

**What you could do.** **[I]** Give a correction factor or a caution for reading E2EDev-style and ProgramDistill-style fidelity figures.

**Suggested study design for a beginner.** **[I]**

1. Take 10 apps from the released E2EDev set [liu2026benchmarking, p. 9].
2. Generate rebuilds without the data-testid hints (2 models, 3 runs).
3. Score each rebuild with the original tests and with the same scenarios rewritten to find elements by role and visible text. Check that the rewritten tests pass 100% on the originals.
4. Compare pass rates and model rankings (Kendall tau).

**Feasibility: easy.** It mostly means rewriting tests.

**Risks.** Rewritten tests may be less strict. Check them on deliberately broken versions of the apps.

## G8. Native mobile apps with hidden server behaviour

> **Gap.** No study we found asks an AI to rebuild a native mobile app, including the back-end behaviour it must infer by watching the running app, and scores the result for fidelity.

**Why it is a gap.**

- **[V]** Only bai2026recreationworld covers native desktop and Android rebuilding [bai2026recreationworld, p. 7]. Other mobile work uses a web proxy [wu2026looks, p. 9] or UI-only code with no back-end [zhou2025declarui, p. 2].
- **[A]** Back-end-dependent interactions and mobile apps are named as future work [xiao2025benchmarking, p. 11; kim2026programdistill, p. 14].
- **[V]** Functional success drops when apps need external services [tran2026vibe, p. 6].

**Novelty verdict.** Partially addressed. Confidence: low. Android generation from specifications and designs exists, and RecreationWorld rebuilds Android apps from a running original. The narrow remainder is inferring a hidden back-end (searches on 2026-11-12).

Closest prior work ([A]): VISTA (arXiv 2605.26144); AppForge (arXiv 2510.07740), 101 Android apps from text specifications; BackendForge (DOI 10.18653/2026.findings-emnlp.2236); FullStack-Agent (arXiv 2602.03798).

**What you could do.** [I] Test whether "interaction lags structure" gets worse when the app's state lives on a hidden server.

**Suggested study design for a beginner.** [I]

1. Pick 3–5 small open-source Android apps with a simple web back-end.
2. Give the agent an emulator and network-traffic observation, but no source code. Use 2 models, 3 runs each.
3. Measure UI-test pass rate, whether the rebuilt back-end answers replayed requests like the original, and the G1 maintainability measures.

**Feasibility: hard.** It needs Android tooling, back-end hosting and traffic capture. Better as later work.

**Risks.** Emulator setup cost, few apps, and limited visibility of network traffic.

## Cybersecurity follow-up: security-control preservation in black-box AI reconstruction

> **The question.** When an AI agent rebuilds a web app only from what it can observe, are the original's security protections kept, dropped, or replaced with new weaknesses?

Security protections (security controls) include login checks, rules about who may see or edit which data, protection against cross-site attacks, safe session handling and safe password storage. This topic was checked separately on 2026-11-12, in the security novelty check report.

### Verdict

Partially novel, with a clearly defensible new core. Confidence: moderate. [I]

- **Already done.** Measuring the absolute security of apps that AI builds from a written specification. Also, one study scanned AI rebuilds of command-line programs with static tools, but did not compare them with the originals: Trajectory-Level Security Debt in LLM Coding Agents (Rajput et al. 2026, AAAI Symposium Series, DOI 10.1609/aaaiss.v10i1.43533; arXiv 2609.35199) [A].
- **Done in a neighbouring setting.** Whether AI code translation keeps security: When Code Crosses Borders (Chang et al. 2026, IEEE TIFS, DOI 10.1109/tifs.2026.3746885) [A]. There, the AI sees the original source code.
- **New core.** A differential test: the same security tests run on the original and on the rebuild of multi-user web apps that were rebuilt from behaviour only. We found no study that does this (searches on 2026-11-12).

- **Why only moderate confidence.** Several close papers appeared between June and November 2026, Semantic Scholar was rate-limited, and Google Scholar was not searched. Re-run the check before you submit.

Other close work (**[A]**, abstracts): Last Secure Code Benchmark (arXiv 2610.14766, preprint) rebuilds repositories from a written specification; Understanding the (In)Security of Vibe-Coded Applications (arXiv 2606.23130, preprint) audits real vibe-coded apps; A.S.E 2.0 (arXiv 2611.06294, preprint) finds that withholding a safeguard from the AI's context lowers secure results; BaxBench (arXiv 2502.11844) and DualGauge (DOI 10.18653/2026.findings-emnlp.1335) test back-ends against exploits.

**Why it matters, from our papers.**

- **[V]** Rebuilds are much smaller than their originals [bai2026recreationworld, p. 15]. **[I]** The missing code may include security checks.
- **[V]** Working code is often insecure: 79.3% of the best setting's correct solutions failed security tests (preprint) [zhao2025vibe, p. 6].
- **[V]** Vibe Code Bench does not measure security [tran2026vibe, p. 9]. 14.8% of its failures were cases where the agent wrongly blocked allowed actions, often through misconfigured access rules [tran2026vibe, p. 8]. **[I]** So rebuilt access control can fail in both directions: too open or too strict.
- **[V]** Static security scores can reward incomplete code: in one preprint, a model got a 97.37 security score with only a 29.76 functional score [pan2026functional, p. 6].
- **[V]** An AI that runs an app can recover valid behaviour rules much better than one that only reads a description [xiong2026propgen, p. 8]. **[I]** This suggests a way to derive security tests from the original's behaviour.

## Sharpened title and core idea

Working title **[I]**: "Rebuilding What You Cannot See: Security-Control Preservation in Black-Box AI Reconstruction of Web Applications."

**[I]** Many security controls cannot be seen from the user interface. Examples are server-side permission checks, cross-site request forgery (CSRF) token checks, password hashing, rate limiting and cookie settings. Others are visible, such as login forms and "permission denied" pages. The hypothesis: AI agents keep the controls they can see and drop the ones they cannot. A rebuild could then behave correctly but be less secure than the original. We call this the "security-observability gap".

## Research questions

1. **Preservation.** What share of an original web app's security controls (authentication, authorization including per-object checks, session management, CSRF protection, input and output handling, secrets and password storage) survive in AI rebuilds made from observable artefacts? Measure it with security tests that pass on the original and run unchanged on the rebuild, and count both missing and over-strict controls.
2. **Observability and input.** Does preservation depend on whether a control is visible in the UI or server-side only, and on what the agent is given: screenshots plus a flow

description, recorded interaction traces, live black-box access with two test accounts, or (as a control) the source code? Does functional fidelity predict preservation?

3. **Regression.** Which weaknesses appear in the rebuild that the original does not have, after adjusting for size and confirming them by hand or by exploit? Does a cheap mitigation help, such as a security-aware instruction or a step where the agent probes the original as a logged-out user and as a second user?

## Study design for a beginner

1. Choose 6–10 small open-source web apps with several user roles and clear ownership rules, which you may legally modify and can run in Docker. Candidates to check: apps that follow the RealWorld "Conduit" specification, and small Django, Flask or Express apps for notes, bookmarks or to-do lists. Check licences first.
2. Write 20–40 security tests per app against the original, using browser tests and direct HTTP requests. Examples: protected pages reject logged-out users; user B cannot read, edit or delete user A's data; normal users cannot perform admin actions; requests without a CSRF token fail; logout ends the session; passwords are not stored in plain text; script code typed into a text field is shown as harmless text.
3. Keep only tests that pass on the original. Before running any agent, tag each test as "UI-visible" or "server-side only", and preregister this.
4. Also write functional user-flow tests, so you can read security results in light of what the rebuild actually implements.
5. Use 2–3 agent tools with 1–2 models, the input conditions from RQ2, and 3 runs per setting. Run everything in isolated containers without internet, so the agent cannot download the original code.
6. Measure: the security-control preservation rate (for implemented features and overall), the over-restriction rate, new weaknesses found by static scanners (Semgrep and CodeQL, compared with the original and checked by hand), dynamic scanner alerts (OWASP ZAP) on both versions, exploit-confirmed weaknesses, and the functional pass rate.
7. Analyse preservation per category with confidence intervals, use a mixed-effects logistic regression, and compare conditions on the same tests with paired tests. Never compare raw finding counts without adjusting for size [pan2026functional, p. 6].

**Feasibility: moderate.** It needs Docker, Playwright, basic web-security knowledge (the OWASP Top 10) and an API budget. Expect about 1–2 days per app to build and check the security tests. The cost per app is unknown, so pilot with 2 apps first.

**Main risks.** Your own tests may favour some designs (phrase them through the app's outside behaviour only). Static scanners are noisy, so rely mainly on dynamic tests. Popular apps may be memorised. Few apps, so report results per app. Missing features can look "secure", which the conditional rate handles. Tools and models drift, so record versions and dates.

## Target venues

These are suggestions **[I]**; deadlines and calls were not checked.

- **Security conferences:** top tier USENIX Security, ACM CCS, NDSS and IEEE S&P; more reachable for a first paper ACSAC, AsiaCCS, ESORICS, EuroS&P, DIMVA and RAID.
- **Security journals:** IEEE TIFS and Computers & Security.
- **Software-engineering venues:** ICSE, FSE, ASE, ISSTA, ESEM for a smaller study, and the journals TOSEM and EMSE.
- **Workshops, a good first step:** LLM4Code, AISec and the AAAI Symposium Series.

### Ethics and dual use

**[I]** Better app cloning could help attackers, for example to build copies of real sites for phishing. Clones can also break copyright or terms of service. So:

- Frame the work as quality assurance for legitimate rebuilding, such as modernising or moving your own apps. The contribution is a test method and a mitigation, not a better cloning tool.
- Use only open-source apps whose licences allow modification.
- Run everything locally in isolated containers. Never target live third-party sites or commercial apps.
- Use no real user data or credentials.
- Do not release prompts or tools tuned for cloning proprietary apps.
- If you find weaknesses in the original projects, report them privately to the maintainers before publishing (responsible disclosure).
- Include an ethics section. Discuss ethics approval with a supervisor, especially if you add a user study.

### How to combine it with G1

**[I]** The security tests can be added to the G1 study as an extra outcome. One pipeline then measures fidelity, maintainability and security preservation, all against the same originals and the same rebuilds. To keep this manageable, run the G1 design on 4–6 multi-user web apps and add 20–40 security tests per app.

## Recommended next steps

A checklist, in order:

1. **Decide whether to add the recommended papers.** This is a scope change, so it is your decision. Our advice **[I]**: add at least the five high-priority papers before you finalise a study design, because they are the closest prior work to G1. Papers are added with rlab add, which checks their metadata, and each one then needs a full-text analysis.
2. **Choose your study direction.** Option A: G1 alone. Option B: the security topic alone. Option C (our suggestion): the G1 design with security preservation as an extra outcome.

3. **Run a small pilot.** Pick 2 apps: one small app from the released E2EDev set [liu2026benchmarking, p. 9] and, for the security idea, one small open-source multi-user app with logins. Check the licences.
4. **Set up the pilot.** Run each original locally in Docker. Write 10–15 behaviour tests (and optionally 5–10 security tests) that find elements by role and visible text, and confirm that all pass on the original.
5. **Rebuild and measure.** Rebuild each app 3 times with one AI coding agent, using only the running app and screenshots, with no access to the original's code. Measure test pass rate, lines of code, SonarQube issues per thousand lines and cognitive complexity per function, on originals and rebuilds.
6. **Write down time, API cost and setup problems.** Use them to decide the size of the full study. The pilot tests your setup; it does not answer the research question.
7. **Re-run the novelty check before you write a paper.** Several close papers appeared between June and November 2026.
8. **Talk to a supervisor** about the statistics (mixed-effects models) and, for the security topic, about ethics approval and responsible disclosure.
9. **Record tool and model versions and dates** for every run, because AI tools change quickly.
10. **Optionally check the published versions** of zhou2025declarui and zhu2026understanding, since we analysed their arXiv versions.

## Recommended papers not yet included (19)

Found during the gap novelty checks on 2026-11-12. Not analysed yet; priority is our judgement **[I]**.

| Priority | Paper | Identifier | Why it matters |
|---|---|---|---|
| high | Is Agent Code Less Maintainable Than Human Code? (CodeThread) | DOI 10.18653/2026.emnlp-main.2528; arXiv 2606.21804 | Maintainability measured by follow-up tasks; template for G1's maintenance task; narrows G3 |
| high | VISTA: An End-to-End Benchmark for Visual Spec-to-Web-App Coding Agents | arXiv 2605.26144 | Designs to full-stack web and Android apps with behaviour probes (G1, G7, G8) |
| high | UI2App: Benchmarking Visual Interaction Inference in Executable Web Application Generation | arXiv 2607.06306 | Screenshot-only multi-page web apps (G1, G7) |
| high | MirrorCode: AI can rebuild entire programs from behavior alone | arXiv 2606.30182 | Rebuilding programs from behaviour alone (G1) |
| high | Why Human Guidance Matters in Collaborative | arXiv 2602.10473 | Human-led versus AI-led vibe coding, 737 participants (G6) |

| | | | |
|---|---|---|---|
| | Vibe Coding | | |
| medium | Do LLM-Generated Web Apps Behave Correctly? (ConformWeb) | DOI 10.18653/2026.emnlp-main.2018 | Black-box behaviour tests without a reference app (G7) |
| medium | WebCoderBench | DOI 10.18653/v1/2026.acl-long.535; arXiv 2601.02430 | Web-app metrics including simple maintainability proxies (G1, G3) |
| medium | From Runnable Code to Shippable Applications (TDDev) | DOI 10.1145/3832783.3834388; arXiv 2605.17242 | Test-driven feedback loops for full-stack apps (G4) |
| medium | Silencer: Self-Bias in LLM-as-Benchmark-Generator | DOI 10.52202/085713-4317; arXiv 2505.20738 | Method for measuring generator self-bias (G2) |
| medium | Preference Leakage: A Contamination Problem in LLM-as-a-judge | arXiv 2502.01534 | Same-family judge bias (G2) |
| medium | WebDevJudge | arXiv 2510.18560 | Reliability of AI judges for web development (G2) |
| medium | Trajectory-Level Security Debt in LLM Coding Agents | DOI 10.1609/aaaiss.v10i1.43533 | Static security scans of rebuilt programs (G1 and security topic) |
| medium | SWE-CI | arXiv 2603.03823 | Maintainability as stability over code evolution (G1, G3) |
| low | AppForge | arXiv 2510.07740 | Native Android app generation (G8) |
| low | CentaurEval | arXiv 2512.04111 | Value of human intervention (G6) |
| low | Beyond Functional Correctness: Static Code Quality Profiling of LLM-Generated Code | arXiv 2610.19167 | Multi-tool static profiling at scale (G3) |
| low | The Next Challenge for Agentic Cybersecurity (SRE-Bench) | arXiv 2608.11469 | Building contamination-free benchmarks (G5) |
| low | BackendForge | DOI 10.18653/2026.findings-emnlp.2236; arXiv 2607.11042 | Black-box back-end tests (G7, G8) |
| low | Comparative Analysis of Iterative Prompt Refinement | DOI 10.2139/ssrn.743 | Iteration and maintainability on JavaScript tasks (G4) |

| | | | |
|---|---|---|---|
| | and Quality of LLM-Generated Code | 5043 (SSRN preprint) | |

### Add only if you choose the security topic (7 more)

MirrorCode, UI2App and the security-debt paper are already in the table above.

| Paper | Identifier | Why it matters |
|---|---|---|
| Last Secure Code Benchmark: Is the Code You Vibe Secure? | arXiv 2610.14766 | Rebuilding repositories from a specification, with exploit-style security tests |
| When Code Crosses Borders: A Security-Centric Study of LLM-based Code Translation | DOI 10.1109/tifs.2026.3746885; arXiv 2509.06504 | Security preservation in AI code translation (the closest idea in another setting) |
| Understanding the (In)Security of Vibe-Coded Applications | arXiv 2606.23130 | Real-world weaknesses of vibe-coded apps |
| A.S.E 2.0: Understanding How Repository Context Impacts the Security of LLM-Generated Code | arXiv 2611.06294 | Safeguards the AI cannot see are not reproduced |
| Security Analysis of LLM-Generated Web API Backends | DOI 10.1109/access.2026.3727682 | Method that combines static scans, dynamic scans and manual penetration testing |
| BaxBench: Can LLMs Generate Correct and Secure Backends? | arXiv 2502.11844 | Exploit-based back-end security tests |
| DualGauge | DOI 10.18653/2026.findings-emnlp.1335; arXiv 2511.20709 | Paired functional and security tests |

## Glossary

| Term | Plain meaning |
|---|---|
| Abstract-only | We read only the paper's short summary, not the whole paper. None of our 23 papers is abstract-only. |
| Ablation | An experiment that removes one part of a system to see how much that part matters. |
| Access control (authorization) | Rules about who may see or change which data in an app. |
| Agent (AI agent) | An AI system that takes actions by itself, such as clicking, running |

| | |
|---|---|
| | commands and editing files, to reach a goal. |
| API | A defined way for one program to talk to another. "API cost" means the fee for using a commercial AI model. |
| arXiv | A free public website where researchers post papers, often before peer review. |
| Authentication | Checking who a user is, for example with a login. |
| Baseline | The thing you compare against. In G1 the baseline is the original app. |
| BDD (behaviour-driven development) tests | Tests written as plain-language scenarios ("given, when, then") that a tool can run automatically. |
| Behavioural test | An automatic check that does something in the app, such as clicking or typing, and verifies the result. |
| Benchmark | A fixed set of tasks and scoring rules used to compare AI systems. |
| Binary | The compiled, machine-readable form of a program, as opposed to its human-readable source code. |
| Bootstrap | A statistical method that re-samples the data many times to estimate how uncertain a result is. |
| Code quality | How well code is written: readable, simple and free of risky patterns. |
| Code smell | A pattern in code that suggests it may be hard to maintain, such as a very long function. |
| Cognitive complexity | A score for how hard a piece of code is for a person to follow. |
| Cohen's kappa (kappa) | A measure of agreement between two raters that corrects for agreement by chance. 1 means perfect agreement; 0 means chance level. |
| Confidence interval | A range of values that probably contains the true value. Wider ranges mean more uncertainty. |
| Contamination | When an AI model has already seen the test material (here, the original app's code) during training, so its score may be too high. |
| Correlation (Spearman, Kendall) | A number between -1 and 1 that shows how strongly two measures rise or fall together. Kendall's tau is often used to compare two rankings. |
| CRUD app | A simple app that mainly creates, reads, updates and deletes records. |
| CSRF (cross-site request forgery) | An attack that tricks a logged-in user's browser into sending an unwanted request. Apps defend against it with secret tokens. |
| CWE | Common Weakness Enumeration, a standard list of types of software security weakness. |
| DAST (dynamic | Probing or attacking the running app to find security problems. |

| | |
|---|---|
| security testing) | |
| Difference-in-differences | A statistical design that compares the change over time in a group that got a treatment with the change in a similar group that did not. |
| Differential testing | Running the same tests on two versions (here, the original and the rebuild) and comparing the results. |
| Docker | A tool that runs software in isolated containers, so it behaves the same on any computer. |
| Effect size | A number that shows how large a difference is, not only whether it exists. |
| Estimator | The statistical method used to calculate an effect from data. Different accepted estimators can give different answers. |
| Evidence matrix | Our table of all findings from the papers, each with its source, label and strength. |
| Evidence strength | How much we trust a finding (strong, moderate or weak), based on sample size, design and agreement between studies. |
| Exploit | A concrete attack that shows a security weakness can really be used. |
| Full text | The whole paper, as opposed to its abstract. All 23 papers were read in full. |
| Functional fidelity | How closely a rebuilt app behaves like the original. |
| Grey literature | Material not published through academic peer review, such as blogs, reports and talks. |
| IDOR / BOLA | A security flaw where a user can access another user's data just by changing an identifier in a request. |
| KLoC | Thousand lines of code. Issues "per KLoC" are counts divided by code size. |
| LLM (large language model) | The type of AI behind tools like ChatGPT and Claude. It produces text, including code. |
| LLM-as-judge (AI judge) | Using an LLM to score outputs in place of human raters. |
| Locator | The way a test finds an element on the page: by an internal name, or (better for rebuilds) by its role and visible text. |
| Maintainability | How easily people can understand, fix and change code later. |
| Maintainability Index (MI) | An older single-number maintainability score, computed from code size and complexity. |
| Mixed-effects model | A statistical model that handles repeated measurements on the same units, such as several rebuilds of the same app. |
| Multimodal model (MLLM, VLM, LVLM) | An AI model that works with images (and sometimes video) as well as text. |

| | |
|---|---|
| Multivocal literature review | A review that combines academic papers with grey literature. |
| Novelty check | Targeted searches to see whether a proposed study has already been done. |
| Oracle | The reference that decides whether an output is correct, such as a test suite or a judge. |
| OWASP Top 10 | A widely used list of the ten most important web-application security risks. |
| p-value / significance test | A statistical check of whether a difference could easily be due to chance. A small p-value (for example below 0.05) suggests it probably is not. |
| Pass@1 | The share of tasks solved correctly on the first attempt. |
| Peer review | Checking of a paper by independent experts before it is published. |
| Pilot study | A small trial run that tests the setup before the full study. |
| Playwright | A free tool that controls a web browser automatically, used to write behavioural tests. |
| Preprint | A paper shared publicly before (or without) peer review, often on arXiv. Flagged and weighted lower in this guide. |
| Preregistration | Publishing your hypotheses and analysis plan before collecting data, so you cannot change them after seeing the results. |
| Pull request | A proposed change to a code project that others review before it is merged. |
| Quasi-experiment | A study that estimates a cause-and-effect relationship without random assignment, for example by comparing matched groups. |
| Randomised controlled trial (RCT) | An experiment that assigns participants to groups by chance, so the groups can be compared fairly. |
| Repository mining | Studying large numbers of code projects (for example on GitHub) to find patterns. |
| Research gap | An open question that the evidence shows is unanswered and that a novelty check could not find already answered. |
| Reverse engineering (in this project) | Rebuilding an existing app only from what can be observed (screens, behaviour, documentation), without its source code. |
| SAST (static security testing) | Scanning code for security problems without running it. Semgrep and CodeQL are examples of such scanners. |
| Screening | Checking search results against inclusion rules to decide which papers to keep. |
| Security control | A protection built into an app, such as a login check or a rule about who may edit which data. |
| Self-preference bias | When an AI judge or test writer favours outputs from its own AI |

| | |
|---|---|
| | family. |
| Snowballing | Finding more papers by following the references and citations of papers you already have. |
| SonarQube | A widely used tool that scans code and reports bugs, code smells and security warnings. |
| SQL injection | An attack that sneaks database commands into an app through input fields. |
| Static analysis | Checking code automatically without running it, for example with SonarQube or PyExamine. |
| Statistical power / underpowered | Power is a study's chance of detecting a real effect. An underpowered study has too few participants to detect small effects. |
| Systematic review | A review that tries to collect every study on a question using a fixed, documented procedure. |
| Technical debt | Future extra work caused by quick or poor solutions now. |
| Thematic analysis | A method for finding and counting recurring themes in text, such as interviews or blog posts. |
| Vibe coding | Building software mainly by describing what you want to an AI in plain language and accepting the code it writes, with little manual review. |
| Visual fidelity | How closely a rebuilt app looks like the original. |
| XSS (cross-site scripting) | An attack that injects script code into a web page so that it runs in other users' browsers. |

# About this guide

## How it was made

This guide was put together on 2026-11-12 from the files of the research project, which started on 2026-11-11. The project used the rlab tool in this workspace for every routine step, so that metadata, citations and page numbers are checked by a program rather than typed from memory.

1. **Scope (2026-11-11).** You chose the meaning of "reverse engineering" (rebuilding from behaviour, UI or documentation, not from source code or binaries), a time window of 2020–2026, and to include preprints with a flag.
2. **Search (2026-11-11).** 33 searches in OpenAlex, Crossref and arXiv, plus web searches and snowballing, returned 1,208 raw hits.
3. **Screening.** 23 papers were included, 22 kept as reserves, and the rest excluded with reasons (including one retracted paper, suspected predatory venues, vendor blogs and opinion-only studies).

4. **Full texts.** All 23 full texts were obtained legally (arXiv, ACL Anthology, Springer open access). None is abstract-only.
5. **Analysis and critical review.** Each paper was analysed from its full text with page references. A second reviewer then checked every analysis against the paper, in three review batches.
6. **Synthesis (2026-11-12).** All findings went into an evidence matrix (204 findings, contradictions C1 to C10). The study methods were compared side by side.
7. **Gap analysis (2026-11-12).** Eight candidate gaps were tested with 38 targeted searches plus 9 web searches. A separate security novelty check used about 20 direct OpenAlex queries, arXiv and Crossref queries, and 3 web searches.
8. **Research report (2026-11-12).** A plain-language report summarised everything. This guide reorganises the same material for reading: the overview, one brief per paper, and these closing sections.

Every citation in this guide was checked automatically: each one names a paper in the project and a page that exists in our PDF.

## Limitations

- **Size.** The review rests on 23 papers. That is enough to map the field, but it is not a complete systematic review.
- **Preprints.** 7 of 23 papers are preprints. All four papers that rebuild apps from observed behaviour (bai2026recreationworld, chen2025iwr, kim2026programdistill and yang2026programbench) are preprints, and this is the group closest to the topic. Conclusions about that group are less certain.
- **Search date.** Searches ran on 2026-11-11 and 2026-11-12. Later papers are not covered, and this field changes every month.
- **Sources not searched.** Semantic Scholar and Google Scholar were not queried in the novelty checks, one Semantic Scholar query was rate-limited, and web results were US-indexed only. Every "we found no study" statement is limited to the sources we used.
- **Paper versions.** For 11 of the 16 published papers we analysed the arXiv version, not the publisher's final version. Page numbers refer to our PDFs. For zhou2025declarui, one table in the arXiv version looked suspicious [zhou2025declarui, p. 9], and the published version has not been checked.
- **Metadata.** Two papers show a title mismatch only because the preprint title differs from the published title. zhao2025vibe is recorded as a preprint although the project notes say it was published at ICML 2026.
- **Papers found but not analysed.** About 19 relevant papers (plus 7 for the security topic) were found in the novelty checks but not analysed. What this guide says about them comes from abstracts only. Some of them (MirrorCode, UI2App, VISTA and CodeThread) are close to G1 and could change its novelty verdict once read in full.
- **Our judgement.** Evidence-strength ratings and gap scores are our own ratings **[I]**. Another team might rate some differently.

- **Mostly web, Python and command-line apps.** Only one paper covers native desktop and Android rebuilding [bai2026recreationworld, p. 7]. Conclusions may not hold for mobile apps.

# Appendix A: Full Research Report

- Project: 2026-11-11-ai-reverse-engineering-vibe-coding
- Full title: *Evaluating AI-Assisted Reverse Engineering for Software Application Reconstruction through Vibe Coding: Functional Fidelity, Code Quality, and Maintainability*
- Date: 2026-11-12
- Written for: the project owner, a beginner researcher
- Based on: 23 full-text paper analyses (papers/*/analysis.md), evidence-matrix.md (204 rows, contradictions C1–C10), reports/methodology-comparison.md, gap-report.md (gaps G1–G8), reports/novelty-check-security-fidelity.md, RESEARCH-QUESTIONS.md, search/search-log.md and rlab status.

**How to read this report.**

- Citations look like [borg2026echoes, p. 13]: the paper's key (its folder name in papers/) and the page of the PDF we hold.
- Each important claim has a label:
- **[V]** (verified): we checked it in the paper's own data, tables or results.
- **[A]** (author claim): the authors say it, but we could not check it ourselves.
- **[I]** (interpretation): our own reasoning, built from the cited papers.
- A *preprint* is a paper that has not yet passed peer review (checking by independent experts before publication). Seven of our 23 papers are preprints. We name them as such and give them less weight.
- Research terms are explained the first time they appear. All of them are also in the glossary (§9).

## 1. Summary in plain language

We asked a simple question. If you use an AI model to rebuild an existing app only from what you can see (its screens, its behaviour, its documentation), how good is the result? "Good" has three parts here. Does the rebuilt app behave like the original (*functional fidelity*)? Is its code well written (*code quality*)? Can people change it easily later (*maintainability*)?

We read 23 recent papers in full. The main answer is that AI rebuilds are still far from complete. In the closest studies, an exact behavioural copy of the original was very rare: 2.8% of apps in one study and 0 of 200 programs in another [bai2026recreationworld, p. 11; yang2026programbench, p. 6]. Both of these studies are preprints. Rebuilt apps usually copy what the screen looks like better than what happens when you click or type.

Code quality and maintainability are much less studied. Studies of AI-written code in general give mixed results, and they often use different measuring tools that do not agree. Almost no study of rebuilt apps measures maintainability properly. No study we found compares the rebuild's maintainability with the original app's.

That missing comparison is the main research gap (G1). It is also a realistic first study for you. A related security question can be added to the same study: does the rebuild keep the original's security protections?

## 2. What we did

**Scope.** "Reverse engineering" here means rebuilding an app from its behaviour, user interface (UI) or documentation. It does *not* mean reading its source code or its compiled program files (*binaries*). We searched for work from 2020 to 2026. Preprints were allowed but flagged. You chose these settings on 2026-11-11.

**Sources.**

- OpenAlex, Crossref and arXiv (three large academic databases), searched through rlab.
- Web searches for very recent work.
- *Snowballing* through OpenAlex. Snowballing means following the reference lists and citations of central papers to find more papers.

**Searches and screening (2026-11-11).**

- 33 searches are logged in search/search-log.md. Together they returned 1,208 raw hits (1,074 unique within each search; duplicates across searches were not counted).
- Most of these searches were run by five search helpers, one per theme: vibe coding, UI-to-code, whole-app generation, code quality, and black-box reverse engineering. They also used snowballing and web searches (research-log.md).
- *Screening* means checking each candidate against the inclusion rules. We included 23 papers, kept 22 as reserves, and excluded the rest with reasons (search/screening.md). Exclusions included one retracted paper, suspected *predatory* venues (publishers that take money without real review), vendor blogs, and studies that only measured opinions.

**Searches for the gap analysis (2026-11-12).**

- 38 more targeted searches are logged on 2026-11-12, plus 9 web searches. They were used to check whether each possible gap was already filled (gap-report.md).
- A separate check for the security topic used about 20 direct OpenAlex queries, arXiv and Crossref queries, and 3 web searches. One Semantic Scholar query was rate-limited and gave no results (reports/novelty-check-security-fidelity.md §4).

**Papers and reading depth** (from rlab status):

| Item | Count |
|---|---|
| Papers included | 23 |
| Full text available and read | 23 |
| Abstract-only | 0 |
| Published (peer-reviewed venue) | 16 |
| Preprints | 7: bai2026recreationworld, chen2025iwr, kim2026programdistill, mota2026comparing, pan2026functional, yang2026programbench, zhao2025vibe |

| | |
|---|---|
| Retracted papers in the evidence | 0 |
| Analyses completed and critically reviewed | 23 |

Note on zhao2025vibe: our database record calls it a preprint. The project notes say it was published at ICML 2026, without a DOI. We still count it as a preprint here, which is the more cautious choice.

**Analysis steps.**

1. Each paper was analysed from its full text. A second reviewer checked every analysis against the paper (three review batches).
2. All findings were collected in an *evidence matrix*: a table of 204 findings, each with its source, label and strength.
3. The study methods were compared side by side (reports/methodology-comparison.md).
4. Eight possible research gaps were tested with a *novelty check*. A novelty check is a set of targeted searches that asks: "has someone already done this?"

## 3. Answers to the research questions

**Evidence strength** describes how much we trust an answer:

- **strong**: enough data, sound design, and several studies agree;
- **moderate**: sound design but a single study or few data;
- **weak**: small or unclear samples, design problems, or preprints that nobody has repeated.

### RQ1. What AI approaches exist for rebuilding apps from observable artefacts, and how do they relate to vibe coding?

**Answer.** We found six families of approaches **[V]**:

1. **Screenshot or design to code.** The AI sees a picture of a page or app and writes code that looks the same [si2025benchmarking, p. 4; zhou2025declarui, p. 3; wu2026looks, p. 6].
2. **Interaction to code.** The AI sees a video of someone using a web page, or before/after screenshots of an interaction, and rebuilds the page [chen2025iwr, p. 5; xiao2025benchmarking, p. 2]. chen2025iwr is a preprint.
3. **Agentic rebuilds from a running original.** An *AI agent* (an AI that can act on its own: click, run commands, write files) explores a live app or a runnable program and rebuilds it without its source code [bai2026recreationworld, p. 3; kim2026programdistill, p. 3; yang2026programbench, p. 2]. All three are preprints. This family is the closest match to "AI-assisted reverse engineering".
4. **Written specification to a full app.** The AI gets a text description or a list of requirements and builds the whole app [liu2026benchmarking, p. 3; lu2025webgen, p. 7; tran2026vibe, p. 3; pan2026functional, p. 2]. pan2026functional is a preprint.
5. **Commercial vibe-coding platforms and AI code editors**, such as Lovable, v0, Replit and Cursor [mota2026comparing, p. 2; he2026speed, p. 4]. mota2026comparing is a preprint.
6. **Recovering behaviour rules from a running app.** One tool learned rules about how an Android app behaves just by running it [xiong2026propgen, p. 4].

**How this relates to vibe coding [I].** Real-world vibe coding means a person describes goals in natural language, re-prompts when things go wrong, and reviews the code very little [fawzy2026vibe, p. 1]. Almost all rebuild studies instead test *autonomous* agents with no person steering them. Human-steered vibe coding has been studied only in small or indirect studies [mock2026vibe, p. 15; asdaque2026novice, p. 1]. Also, different papers use the words "vibe coding" for different things (see §5, C9). Their results should not be added together.

**Evidence strength: moderate.** The picture is consistent, but 7 of 23 papers are preprints and the field changes every month.

**What would change this answer.** New benchmarks of human-steered rebuilding. Also, several recent papers we found but have not yet analysed (§8, step 1).

## RQ2. How is functional fidelity measured, and how well do current approaches achieve it?

**How it is measured.** Every study needs an *oracle*: the reference that decides whether an output is correct, such as a test suite or a judge. We found four kinds **[V]**:

1. **Behavioural tests run against the original.** The same tests that pass on the original app are run on the rebuild [bai2026recreationworld, p. 9; kim2026programdistill, p. 4; yang2026programbench, p. 5; liu2026benchmarking, p. 5].
2. **LLM or agent judges.** An AI model reads natural-language test steps, uses the rebuilt app and decides pass or fail [lu2025webgen, p. 6; tran2026vibe, p. 4; chen2025iwr, p. 7]. An *LLM* (large language model) is the kind of AI behind tools like ChatGPT and Claude.
3. **Visual similarity.** Screenshots of the rebuild are compared with the original [si2025benchmarking, p. 3; wu2026looks, p. 4].
4. **Navigation graphs.** A map of which screen leads to which screen is compared with the original's map [zhou2025declarui, p. 7].

**How well it is achieved.** Fidelity is partial everywhere.

- **[V]** Full behavioural equivalence is rare: 2.8% of apps for the best model in one study [bai2026recreationworld, p. 11] and 0 of 200 programs in another [yang2026programbench, p. 6]. Both are preprints.
- **[V]** The best results at the level of whole workflows or requirements are about 49–62% [kim2026programdistill, p. 11; liu2026benchmarking, p. 6; tran2026vibe, p. 6].
- **[V]** Some benchmarks report lower figures: an interaction score below 25 for all 28 models tested (preprint) [chen2025iwr, p. 8], under 45% of tests passed (preprint) [pan2026functional, p. 5], and 27.8% accuracy [lu2025webgen, p. 8].
- **[V]** The more parts of the app depend on each other, and the larger the app, the lower the fidelity [kim2026programdistill, p. 8; yang2026programbench, p. 31; tran2026vibe, p. 6].
- **[V]** Interaction logic lags behind static structure. Rebuilds copy what is on the screen more reliably than what happens after an action [bai2026recreationworld, p. 16; chen2025iwr, p. 16; xiao2025benchmarking, p. 6]. In bai2026recreationworld, this structure-versus-action analysis covers only 4 of its 10 models. In one case a rebuild scored 100% visually but only 55.1% on behaviour tests [bai2026recreationworld, p. 18].
- **Caution about one well-known finding [V]**. In chen2025iwr, the "visual score is higher than the interaction score" result is partly built into the metrics themselves. The visual score is averaged only over the checkpoints the agent reached, while the interaction score counts all actions [chen2025iwr, p. 7]. So that gap is not a fair like-for-like comparison.

**[I]** Each benchmark uses a different oracle, input and app type. The percentages above therefore **cannot be ranked against each other**.

**Evidence strength. Moderate** for "fidelity is far from complete, and interaction is the weakest part". **Weak** for any exact number or any ranking across benchmarks. Most results come from a single run, and 5 of the 8 main fidelity benchmarks are preprints.

**What would change this answer.** Repeated runs with uncertainty reported. Rebuild benchmarks that are peer-reviewed. Tests that do not depend on the original's internal names (see RQ4).

## RQ3. What is known about the code quality and maintainability of AI-generated or vibe-coded app code, compared with human code?

**Answer: the evidence is mixed.**

- [V] **Real projects using Cursor (an AI code editor).** After teams adopted Cursor, project-level warnings from *SonarQube* (a widely used code-checking tool) rose by 30.26%, and *cognitive complexity* (a score for how hard code is to follow) rose by 41.64% [he2026speed, p. 7]. But this result depends on the statistical method used. With another accepted method (Callaway–Sant'Anna), both effects became negative and not significant [he2026speed, p. 16]. The rise in warnings also comes mainly from projects simply having more code [he2026speed, p. 8]. We count this result **once**, from he2026speed itself. A review paper (ehsani2026faster) repeats the same numbers, so it is not independent evidence.
- [V] **A randomised experiment on maintainability.** A *randomised controlled trial* (RCT) assigns participants to groups by chance, which makes fair comparison possible. This one was also *preregistered* (its plan was published before the data were collected). A second developer had to extend code that someone else had written with or without AI help. There was no significant difference in completion time (median 136 vs 173 minutes, p = 0.56) [borg2026echoes, p. 33] or in code health (8.49 vs 8.44) [borg2026echoes, p. 38]. However, the study had only 75 of the 128 planned participants, so it can rule out only large effects [borg2026echoes, p. 19].
- [V] **Single programs.** GPT-4o code had fewer SonarQube "bugs" than human solutions from a competitive-programming dataset [molison2025llm, p. 8]. Those human solutions were written to be correct, not to be maintainable, so they are a weak baseline. The results for *code smells* (patterns that suggest code will be hard to maintain) were mixed [molison2025llm, p. 8].
- [V] **Different kinds of problems.** LLM code showed different smell types from human code. For example, larger LLMs produced more "Long Method" smells [zhu2026understanding, p. 7]. In AI-generated multi-file apps, smell counts closely followed code size (ρ = 0.94) [zhu2026understanding, p. 8].
- [V] **Platform matters** (preprint, 9 apps only). Apps from Lovable had about 22.83 SonarQube issues per thousand lines, compared with about 9 for v0 and Replit [mota2026comparing, p. 5].
- [V] **Rebuilds are smaller and more concentrated than the originals** (two preprints).
- In bai2026recreationworld, 89.4% of rebuilds had less code than the original (median size 16.9% of the original) [bai2026recreationworld, p. 15]. This covers only 4 of the 10 models, and only desktop and Android apps.
- In yang2026programbench, rebuilds that passed at least 75% of tests had a median of 1,173 lines and 3 files, against 3,068 lines and 15 files in the originals [yang2026programbench, p. 9].
- [I] We cannot yet tell whether "smaller" means "simpler and cleaner" or "missing features".
- [V] **Perception.** Practitioners mostly describe vibe-coded output as "fast but flawed" (68% of coded statements) [fawzy2026vibe, p. 8]. This is opinion, not code measurement.

**[I]** The measuring tools disagree with each other: SonarQube, PyExamine, CodeHealth and the Maintainability Index give different pictures (see §5, C3). And no study measures validated maintainability of *rebuilt* apps against their originals.

**Evidence strength: weak to moderate.** The results conflict (C1–C3), and the instruments are not validated for AI code.

**What would change this answer.** A study that applies more than one size-adjusted instrument plus a real maintenance task to rebuilds *and* to their originals (gap G1). Also a study that tests whether the instruments agree with each other (gap G3).

## RQ4. What are the methodological strengths and weaknesses of current evaluations?

**Strengths [V]**:

- Behavioural tests that do not care how the rebuild is built, with controls against copying the original [yang2026programbench, p. 18; bai2026recreationworld, p. 9].
- Checks of the test suites themselves [yang2026programbench, p. 7; liu2026benchmarking, p. 14].
- Checks of whether AI judges agree with humans [tran2026vibe, p. 9; wu2026looks, p. 13].
- One preregistered RCT [borg2026echoes, p. 19].

**Recurring weaknesses:**

1. **Single runs, with no measure of uncertainty and no significance tests [V]** [lu2025webgen, p. 40; yang2026programbench, p. 10]. This matters because AI output changes a lot from run to run. In one study, differences between generation runs explained 73.5–92.5% of the variation in scores [tran2026vibe, p. 8].
2. **Judges and tests made by the same AI family that is being evaluated, or judges that are poorly validated [V]**. Agreement between AI judges and humans ranged from 36.1% to 86.4%, depending on which model was the judge [tran2026vibe, p. 9]. Some judges were tuned and tested on the same data [chen2025iwr, p. 17; si2025benchmarking, p. 17].
3. **Uneven handling of *contamination* [V]**. Contamination means the AI may have seen the original app's code during training. With internet access, 20–36% of the stronger models' tasks were flagged as cheating, mostly by looking up the original source code [yang2026programbench, p. 6].
4. **Maintainability is missing or measured only through rough proxies [V]** [pan2026functional, p. 3; wu2026looks, p. 4].
5. **Tests tied to the original's internal names [V]**. For example, some tests look for hidden identifiers (data-testid labels) taken from the original's code, and those names were given to the AI in the prompt [liu2026benchmarking, p. 16]. Others use the original's selectors [kim2026programdistill, p. 35] or give the AI a structural map as input [zhou2025declarui, p. 7].
6. **No human-developer baseline** in any rebuild or generation benchmark **[V]** [yang2026programbench, p. 20; tran2026vibe, p. 5; lu2025webgen, p. 7].

**Evidence strength. Strong** for weaknesses 1 and 4: we verified them across 12 or more papers. **Moderate** for the judge, contamination and test-coupling problems.

**What would change this answer.** New benchmarks that report repeated runs, judges from different AI families, and validated maintainability measures.

## RQ5. Which gaps remain, and what study could address them?

**Answer [I]**. The main gap that survived the novelty check is **G1**. We found no study that rebuilds existing apps from observable artefacts and then measures, on the same rebuilds, both:

- (a) behavioural fidelity against the original, and
- (b) maintainability, using more than one size-adjusted instrument plus a maintenance task, with the original app as the baseline (searches on 2026-11-12).

Measuring each part on its own is not new. Five more gaps are partly new; two are mostly or partly addressed (§6). A security version of the question is also partly new (§6).

**Evidence strength: moderate.** The evidence that the gap exists is strong [bai2026recreationworld, p. 15; yang2026programbench, p. 9; pan2026functional, p. 3; wu2026looks, p. 4; tran2026vibe, p. 9]. Our confidence in its novelty is medium-high. The field publishes every month, so the check must be repeated before you submit anything.

**What would change this answer.** A new paper that combines fidelity and maintainability against originals. Several close 2026 papers exist (MirrorCode, UI2App, VISTA, WebCoderBench). We read only their abstracts, so they must be analysed before we can be sure (§8).

## 4. Key papers to read first

Start with these five. They give you the core of the topic and the tools for a first study.

1. **bai2026recreationworld** (RecreationWorld; preprint). The closest match to the project. AI agents rebuild apps on five platforms from a running original, with no source code. It reports a 2.8% full-pass rate [bai2026recreationworld, p. 11] and also measures how much smaller the rebuilds are than the originals [bai2026recreationworld, p. 15].
2. **yang2026programbench** (ProgramBench; preprint). Rebuilds 200 programs from the program plus its documentation. It shows careful test design and contamination controls, and compares rebuilt code structure with the originals [yang2026programbench, p. 6; yang2026programbench, p. 9].
3. **borg2026echoes** (Echoes of AI; peer-reviewed, Empirical Software Engineering). The only randomised test of maintainability. Its "second developer extends the code" design is the model for the maintenance task in G1 [borg2026echoes, p. 13].
4. **liu2026benchmarking** (E2EDev; peer-reviewed, ACL 2026). It rebuilds existing web apps from their requirements and comes with human-reviewed tests that pass 100% on the originals [liu2026benchmarking, p. 5]. It is a good source of apps and tests for a pilot study, and it ran every setting 3 times. Note its test-coupling problem [liu2026benchmarking, p. 16].
5. **tran2026vibe** (Vibe Code Bench; peer-reviewed, CAIS 2026). The benchmark that uses the term "vibe coding" directly. It reports judge problems [tran2026vibe, p. 9] and run-to-run variation [tran2026vibe, p. 8], and it releases its generated apps [tran2026vibe, p. 11]. These apps can be reused for the easy follow-up studies G2 and G3.

## 5. Where studies disagree

Full entries are in evidence-matrix.md → "Contradictions". All explanations below are our interpretation **[I]**. We do not pick a side.

- **C1. Does AI assistance harm maintainability?** Project-level warnings and complexity rose after Cursor adoption [he2026speed, p. 7]. But a randomised experiment found no difference when a second developer extended AI-assisted code [borg2026echoes, p. 33; borg2026echoes, p. 38].
- Why they may differ: real open-source projects over many months versus one small Java task; whole-project totals (which grow with code size) versus per-file scores; and a result that changes with the statistical method [he2026speed, p. 16] versus an underpowered experiment [borg2026echoes, p. 19].
- **[I]** Both fit the idea "AI adds more code, so totals rise, but per-file quality and the next developer's effort are not clearly worse". Neither study settles the question.
- **C2 and C3. Which code-quality tool should we trust?** LLM code had fewer SonarQube bugs than human code in one study [molison2025llm, p. 8], but repository warnings rose in another [he2026speed, p. 7]. Another study chose a different tool (PyExamine) because it considered SonarQube "superficial linting" [zhu2026understanding, p. 2]. A large review notes that technical-debt tools disagree about what debt is **[A]** [ehsani2026faster, p. 5].
- Why: different units (single programs versus whole projects per month), baselines and tools.
- **[I]** The answer to "is AI code better or worse?" may depend on which tool you use.
- **C4. Does feedback (letting the AI test and fix its own work) help?** In one preprint, three repair strategies lowered non-functional quality without improving correctness, but only one model was tested [pan2026functional, p. 8]. Elsewhere, self-testing went together with better results [tran2026vibe, p. 7], and targeted repair loops improved compiling and navigation [zhou2025declarui, p. 10; wu2026looks, p. 12].
- **[I]** Feedback seems to improve whatever it checks. No study shows that it improves both fidelity and code quality together.
- **C5. Do rebuilds "look right but behave wrong"?** Several benchmarks found behaviour scores below visual scores [chen2025iwr, p. 8; lu2025webgen, p. 9]. But in bai2026recreationworld, the visual channel scored *lower* than the behaviour tests for 9 of 10 models [bai2026recreationworld, p. 11]. And chen2025iwr's gap is partly built into its metrics [chen2025iwr, p. 7].
- **[I]** The studies agree only on one point: static structure is copied more easily than interaction logic. The broader claim "visual always beats functional" is not supported.
- **C6. Do multi-agent frameworks help?** In E2EDev, multi-agent frameworks "do not consistently improve" over a plain LLM (the authors' words), and MetaGPT nearly always failed [liu2026benchmarking, p. 6]. In a small experiment, an agentic MetaGPT product scored highest [mock2026vibe, p. 41].
- Why: that experiment used a tiny task and newer models [mock2026vibe, p. 17].

- **C7. Is generated code secure?** One preprint gave a high static security score (87.40% on average) [pan2026functional, p. 6]. Another preprint found that 79.3% of the best setting's working solutions failed security tests [zhao2025vibe, p. 6].
- **[I]** The two measure different things. The first counts flagged code patterns compared with the original, and it can reward incomplete code: less code means fewer findings. The second runs targeted attack-style tests.
- **C8. More code or less code?** Rebuilds are much smaller than their originals [bai2026recreationworld, p. 15; yang2026programbench, p. 9]. But AI-assisted development adds code to existing projects [he2026speed, p. 8; borg2026echoes, p. 30].
- **[I]** Size means different things in the two settings. Quality counts must be divided by code size before comparing.
- **C9. Same words, different things.** "Vibe coding" means different things in different papers [fawzy2026vibe, p. 1; asdaque2026novice, p. 1; mock2026vibe, p. 15; he2026speed, p. 4]. Fidelity percentages use different oracles. Do not pool these results.
- **C10. The same AI family builds the test and wins it.**
- In some benchmarks, the AI family that wrote the tests or tasks also topped the results [yang2026programbench, p. 3; tran2026vibe, p. 3; kim2026programdistill, p. 2; zhao2025vibe, p. 17].
- In others it did not. For example, the Qwen3.7-Plus model used as the visual judge in bai2026recreationworld was the lowest-scoring of the 10 models (average 9.15) [bai2026recreationworld, p. 11].
- **[I]** No paper tests this possible bias directly. It is a warning sign, not proven bias.

A related weak point: zhao2025vibe reports that the functionally weakest model had the highest secure ratio on tasks that all models solved [zhao2025vibe, p. 6]. But the size of that shared task set is not reported. Treat this "most secure model" finding as **weak**.

## 6. Research gaps and possible contributions

A *research gap* is a question that the evidence shows is open and that a novelty check could not find already answered. Our checks used OpenAlex, Crossref, arXiv and web searches on 2026-11-12. "We found no study" means only that these searches found none. It does not mean that no such study exists.

| Gap | In short | Novelty | Confidence | Beginner feasibility |
|---|---|---|---|---|
| **G1** | Behavioural fidelity **and** validated maintainability of rebuilt apps, both measured against the originals | novel (narrow claim) | medium-high | moderate |
| G2 | Bias when the same AI family builds, tests and judges a benchmark | partially novel | medium | easy |
| G3 | Do maintainability tools agree, and do they predict real maintenance effort, for AI-built apps? | partially novel (narrowed) | medium | easy–moderate |
| G4 | Does feedback improve fidelity and code quality together in rebuilding? | partially novel | medium | moderate |
| G5 | How much does contamination inflate rebuild scores? | partially novel | medium-low | moderate |
| G6 | Human-steered vibe coding versus autonomous agents for rebuilding | partially novel | medium | moderate–hard |
| G7 | Bias from tests tied to the original's internal names | mostly addressed | low-medium | easy |
| G8 | Native mobile apps with hidden server behaviour | partially addressed | low | hard |

### G1 (top gap): joint fidelity and maintainability, against the original

**Why it is a gap.**

- **[V]** Rebuild benchmarks with behavioural tests measure the code only by size and structure [bai2026recreationworld, p. 15; yang2026programbench, p. 9], or not at all [kim2026programdistill, p. 14].
- **[V]** UI benchmarks use rough proxies [wu2026looks, p. 4] or ratings from 5 developers [zhou2025declarui, p. 10].
- **[V]** The only maintainability score measured relative to an original is a single Maintainability Index in a preprint that rebuilds from requirements, not from observed behaviour [pan2026functional, p. 3].
- **[V]** Wherever both are measured, functional correctness and other quality dimensions diverge [molison2025llm, p. 8; zhao2025vibe, p. 6; wu2026looks, p. 7]. So passing tests says little about maintainability.

- **[A]** The Vibe Code Bench authors state that passing functional tests does not mean the code is maintainable, secure or documented [tran2026vibe, p. 9].

**Suggested study [I]** (full design in gap-report.md).

- Apps: 8–10 small web apps. About 5 come from the E2EDev set, which has validated tests [liu2026benchmarking, p. 9]. The others are apps released after the AI models' training cutoff, or apps you write yourself.
- Input to the AI: the running original and screenshots only. No source code and no hidden identifiers.
- Builders: 2–3 AI tools, each run 3 times per app (about 60–90 rebuilds).
- Fidelity: browser tests written against the original. They find elements by role and visible text, not by internal names, and each test must pass 100% on the original first.
- Maintainability, measured on the rebuild **and** the original, reported as a ratio:
- SonarQube issues per thousand lines and cognitive complexity per function;
- code duplication, plus a second tool;
- a standard change task (two small feature changes) carried out by a fixed "maintainer" agent, in the spirit of borg2026echoes [borg2026echoes, p. 13].
- Analysis: preregistered hypotheses, *mixed-effects models* (statistical models that account for repeated measures on the same app), confidence intervals, and all counts divided by code size.

**Contribution [I]**. The first profile of AI rebuilds that answers: "does the rebuild that behaves like the original also stay as easy to change as the original?" It would also show whether smaller rebuilds are simpler or just incomplete (C8).

**Feasibility: moderate.** The tools are free or have free tiers. The main work is writing the tests and managing 60–90 rebuilds. You will need some help with the statistics.

## G2 and G3: two easy studies that need no new app generation

- **G2 [I]**. Re-judge the released Vibe Code Bench apps [tran2026vibe, p. 11] with judges from three AI families, plus about 300 human-labelled steps. Then check whether judges favour apps built by their own family. Related methods exist outside app generation, so the gap is only partly new.
- **G3 [I]**. Run several maintainability tools on about 60 released AI-built apps. Check whether the tools agree, and whether they predict how well a maintainer agent can make changes. A 2026 peer-reviewed paper (CodeThread, EMNLP 2026, not yet analysed) partly covers this for code changes at the level of single issues, so we narrowed the gap to whole applications.

## The cybersecurity follow-up you chose: security-control preservation

**Question [I]**. When an AI agent rebuilds a web app only from what it can observe, are the original's security protections kept, dropped, or replaced with new weaknesses? Examples of protections: login checks; rules about who may see or edit which data; protection against cross-site attacks; safe password storage.

**Verdict: partially novel, with a defensible new core** (confidence moderate; reports/novelty-check-security-fidelity.md).

- Already done: measuring the *absolute* security of AI-generated apps built from specifications. Also, one study (Rajput et al. 2026, peer-reviewed symposium paper, abstract plus HTML sections read) scanned AI rebuilds of command-line programs with static tools. It did not compare them with the originals **[A]**.
- Done in a neighbouring setting: whether code *translation* keeps security (Chang et al. 2026, IEEE TIFS) **[A]**. There, the AI sees the original source code.
- New core **[I]**: a **differential** test. The same security tests run on the original and on the rebuild of multi-user web apps that were rebuilt from behaviour only.

**Why it matters, from our papers.**

- **[V]** Rebuilds are much smaller than their originals [bai2026recreationworld, p. 15]. **[I]** Missing code may include security checks.
- **[V]** Working code is often insecure: 79.3% of the best setting's correct solutions failed security tests (preprint) [zhao2025vibe, p. 6].
- **[V]** Vibe Code Bench does not measure security [tran2026vibe, p. 9]. 14.8% of its failures were cases where the agent wrongly blocked allowed actions, often through misconfigured access rules [tran2026vibe, p. 8]. **[I]** So rebuilt access control can fail in both directions: too open or too strict.
- **[V]** Static security scores can reward incomplete code. In one preprint, a model got a 97.37 security score with only 29.76 functional score [pan2026functional, p. 6].

**Core hypothesis [I]**. AI agents keep the security controls you can *see* (such as a login form) and drop the ones you *cannot see* (such as server-side permission checks). We call this the "security-observability gap".

**Feasibility: moderate.** It needs Docker, Playwright, basic web-security knowledge (the OWASP Top 10) and about 1–2 days per app to build the security tests. It also needs an ethics section, because better app cloning could help attackers (for example, to make phishing copies of real sites). Use only open-source apps that you are allowed to modify, run everything locally, and never target live third-party sites.

**How it fits with G1 [I]**. The security test suite can be added to the G1 study as an extra outcome: fidelity, maintainability **and** security preservation, all measured against the same originals.

## 7. Limitations of this review

- **Size of the review.** It rests on 23 papers. That is enough to map the field, but not a complete *systematic review* (a study that tries to collect every study on a question with a fixed protocol).
- **Reading depth.** All 23 papers were read in full. None is abstract-only.
- **Preprints.** 7 of 23 papers are preprints and are not peer-reviewed. All four papers that rebuild apps from observed behaviour (bai2026recreationworld, chen2025iwr, kim2026programdistill, yang2026programbench) are preprints, and this is the group closest to our topic. Conclusions about that family are therefore less certain.
- **Search date.** Searches ran on 2026-11-11 (discovery) and 2026-11-12 (gap and novelty checks). Papers that appeared after that are not covered. This field changes monthly.
- **Sources not searched.** Semantic Scholar and Google Scholar were not queried in the novelty checks. One Semantic Scholar query in the security check was rate-limited and returned nothing. Web search results were indexed in the US only. Every "we found no study" statement is limited to the sources we used.
- **Versions of papers.** For 11 of the 16 published papers, the PDF we analysed is the arXiv version, not the publisher's final version: asdaque2026novice, ehsani2026faster, fawzy2026vibe, he2026speed, lu2025webgen, molison2025llm, tran2026vibe, xiao2025benchmarking, xiong2026propgen, zhou2025declarui and zhu2026understanding. Page numbers refer to our PDFs. The final versions may differ slightly. For zhou2025declarui, one table in the arXiv version looked suspicious (impossible percentages for 13 apps) [zhou2025declarui, p. 9], and the published version has not been checked.
- **Metadata issues** (not affecting findings). Two papers show a title mismatch only because the preprint title differs from the published title. zhao2025vibe is recorded as a preprint although it was published at ICML 2026.
- **Papers found but not yet analysed.** The gap and novelty checks found about 19 relevant 2025–2026 papers that are **not** included or analysed. Seven more are relevant only to the security topic. They are listed in §8. Statements about them in this report come from their abstracts only and are labelled **[A]**. Some of them (for example MirrorCode, UI2App, VISTA and CodeThread) are close to G1 and could change its novelty verdict once read in full.
- **Our judgement in the synthesis.** Evidence-strength ratings and gap scores (evidence × novelty × feasibility) are our own ratings **[I]**. Another team might rate some of them differently.
- **Mostly web, Python and command-line apps.** Only one paper covers native desktop and Android rebuilding [bai2026recreationworld, p. 7]. Our conclusions may not hold for mobile apps.

# 8. Recommended next steps

## Step 1: decide whether to add the recommended papers

Adding them is a scope change, so it is your decision. Our advice **[I]**: add at least the five high-priority papers before you finalise a study design, because they are the closest prior work to G1.

**From the gap report (19 papers; not analysed yet):**

| Priority | Paper | Identifier | Why |
|---|---|---|---|
| high | Is Agent Code Less Maintainable Than Human Code? (CodeThread) | DOI 10.18653/2026.emnlp-main.2528; arXiv 2606.21804 | Measures maintainability through follow-up tasks; template for G1's maintenance task; narrows G3 |
| high | VISTA | arXiv 2605.26144 | Design to full-stack web and Android apps with behaviour probes (G1, G7, G8) |
| high | UI2App | arXiv 2607.06306 | Screenshot-only multi-page web apps (G1, G7) |
| high | MirrorCode | arXiv 2606.30182 | Rebuilding programs from behaviour alone (G1) |
| high | Why Human Guidance Matters in Collaborative Vibe Coding | arXiv 2602.10473 | Human-led versus AI-led vibe coding, 737 participants (G6) |
| medium | ConformWeb (Do LLM-Generated Web Apps Behave Correctly?) | DOI 10.18653/2026.emnlp-main.2018 | Black-box behavioural tests without a reference app (G7) |
| medium | WebCoderBench | DOI 10.18653/v1/2026.acl-long.535; arXiv 2601.02430 | Web-app metrics including simple maintainability proxies (G1, G3) |
| medium | From Runnable Code to Shippable Applications (TDDev) | DOI 10.1145/3832783.3834388; arXiv 2605.17242 | Test-driven feedback loops for full-stack apps (G4) |
| medium | Silencer: Self-Bias in LLM-as-Benchmark-Generator | DOI 10.52202/085713-4317; arXiv 2505.20738 | Method for measuring generator self-bias (G2) |
| medium | Preference Leakage | arXiv 2502.01534 | Same-family judge bias (G2) |

| | | | |
|---|---|---|---|
| medium | WebDevJudge | arXiv 2510.18560 | Reliability of AI judges for web development (G2) |
| medium | Trajectory-Level Security Debt in LLM Coding Agents | DOI 10.1609/aaaiss.v10i1.43533 | Static security scans of rebuilt programs (G1 and security topic) |
| medium | SWE-CI | arXiv 2603.03823 | Maintainability as stability over code evolution (G1, G3) |
| low | AppForge | arXiv 2510.07740 | Native Android app generation (G8) |
| low | CentaurEval | arXiv 2512.04111 | Value of human intervention (G6) |
| low | Beyond Functional Correctness: Static Code Quality Profiling | arXiv 2610.19167 | Multi-tool static profiling at scale (G3) |
| low | SRE-Bench (The Next Challenge for Agentic Cybersecurity) | arXiv 2608.11469 | Building contamination-free benchmarks (G5) |
| low | BackendForge | DOI 10.18653/2026.findings-emnlp.2236; arXiv 2607.11042 | Black-box back-end tests (G7, G8) |
| low | Comparative Analysis of Iterative Prompt Refinement | DOI 10.2139/ssrn.7435043 (SSRN preprint) | Iteration and maintainability on JavaScript tasks (G4) |

**Add only if you choose the security topic (7 more):** arXiv 2610.14766 (Last Secure Code Benchmark); DOI 10.1109/tifs.2026.3746885 (security in LLM code translation); arXiv 2606.23130 (security of vibe-coded apps in the wild); arXiv 2611.06294 (A.S.E 2.0: unseen safeguards are not reproduced); DOI 10.1109/access.2026.3727682 (SAST, DAST and penetration testing of generated web back-ends); arXiv 2502.11844 (BaxBench); DOI 10.18653/2026.findings-emnlp.1335 (DualGauge). MirrorCode, UI2App and the security-debt paper are already in the table above.

All identifiers were copied from rlab search output and the novelty check. They are added with rlab add, which checks metadata. Each added paper then needs a full-text analysis.

## Step 2: choose your study direction

You have three options **[I]**:

| Option | What it is | Pros | Cons |
|---|---|---|---|
| A. G1 | Fidelity + maintainability of AI rebuilds against originals | Top-ranked gap; strongest evidence; closest to the project title | Software-engineering framing only |
| B. Security | Security-control | Fits your interest in | Needs web-security |

| | | | |
|---|---|---|---|
| topic | preservation in black-box rebuilds | cybersecurity; a clear new core | skills and an ethics section; confidence in novelty only moderate |
| C. Combine (our suggestion) | G1 design with security preservation as an extra outcome | One pipeline gives three outcomes; the security tests reuse G1's apps and runs | More work per app; keep the first version small |

A good way to keep option C manageable: run the G1 design on 4–6 multi-user web apps, and add 20–40 security tests per app.

## Step 3: run a small pilot experiment first

A *pilot* is a small trial run that tests your setup before the real study. Suggested pilot **[I]**:

1. **Pick 2 apps.** One small app from the released E2EDev set, which has validated tests [liu2026benchmarking, p. 9]. One small open-source multi-user app with logins (for example, an app that follows the "RealWorld/Conduit" specification), if you want to try the security idea. Check the licences first.
2. **Run each original locally** in Docker.
3. **Write 10–15 behaviour tests** with Playwright. Find elements by their role and visible text, not by internal names. Confirm that every test passes on the original.
4. **Optional security tests (5–10):** for example, a protected page rejects a logged-out user, and user B cannot edit user A's article. Confirm that they pass on the original.
5. **Rebuild each app 3 times** with one AI coding agent. Give it the running app and screenshots only, with no source code and no internet access to the original's code.
6. **Measure on the originals and the rebuilds:** test pass rate; lines of code; SonarQube issues per thousand lines; cognitive complexity per function.
7. **Write down** the time, the API cost, and any problems with the setup. Use this to decide the final study size.

The pilot tells you whether the full study is affordable and whether your tests work. It does not answer the research question.

## Step 4: other actions

- **Re-run the novelty check before you write a paper.** Several close papers appeared between June and November 2026.
- **Talk to a supervisor** about the statistics (mixed-effects models) and, for the security topic, about ethics approval and responsible disclosure.
- **Record tool and model versions and dates** for every run, because AI tools change quickly.
- **Optionally check the published versions** of zhou2025declarui and zhu2026understanding.

## 9. Glossary

- **Agent (AI agent).** An AI system that can take actions by itself, such as clicking, running commands and editing files, to reach a goal.
- **Baseline.** The thing you compare against. In G1 the baseline is the original app.
- **Behavioural test.** An automatic check that does something in the app (click, type) and verifies the result.
- **Benchmark.** A fixed set of tasks and scoring rules used to compare AI systems.
- **Binary.** The compiled, machine-readable form of a program, as opposed to its human-readable source code.
- **Code quality.** How well code is written: readable, simple, free of risky patterns.
- **Code smell.** A pattern in code that suggests it may be hard to maintain, such as a very long function.
- **Cognitive complexity.** A score for how hard code is for a person to follow.
- **Confidence interval (CI).** A range of values that probably contains the true value. It shows how uncertain a result is.
- **Contamination.** When an AI model has already seen the test material (here, the original app's code) during training, so its score may be too high.
- **Differential security testing.** Running the same security tests on the original and the rebuild, and comparing the results.
- **Estimator.** The statistical method used to calculate an effect from data. Different accepted estimators can give different answers.
- **Evidence matrix.** Our table of all findings, each with its source, label and strength.
- **Evidence strength (strong / moderate / weak).** How much we trust a finding, based on sample size, design and agreement between studies.
- **Full text / abstract-only.** Whether we read the whole paper or only its short summary.
- **Functional fidelity.** How closely a rebuilt app behaves like the original.
- **Grey literature.** Material not published through academic peer review, such as blogs, reports and talks.
- **LLM (large language model).** The type of AI behind ChatGPT, Claude and similar tools. It produces text, including code.
- **LLM judge.** An LLM used to score outputs in place of human raters.
- **Locator.** The way a test finds an element on the page: by an internal name, or (better for rebuilds) by its role and visible text.
- **Maintainability.** How easily people can understand, fix and change code later.
- **Maintainability Index (MI).** An older single-number score of maintainability, computed from code size and complexity.
- **Mixed-effects model.** A statistical model that handles repeated measurements on the same units, such as several rebuilds of the same app.
- **Multivocal literature review.** A review that combines academic papers with grey literature.
- **Novelty check.** Targeted searches to see whether a proposed study has already been done.

- **Oracle.** The reference that decides whether an output is correct, such as a test suite or a judge.
- **Peer review.** Checking of a paper by independent experts before it is published.
- **Pilot study.** A small trial run that tests the setup before the full study.
- **Preprint.** A paper shared publicly before (or without) peer review, often on arXiv.
- **Preregistration.** Publishing your hypotheses and analysis plan before collecting data, so you cannot change them after seeing the results.
- **Randomised controlled trial (RCT).** An experiment in which participants are assigned to groups by chance, so that the groups can be compared fairly.
- **Research gap.** An open question that the evidence shows is unanswered and that a novelty check could not find already answered.
- **Reverse engineering (in this project).** Rebuilding an existing app only from what can be observed (screens, behaviour, documentation), without its source code.
- **Screening.** Checking search results against inclusion rules to decide which papers to keep.
- **Security control.** A protection built into an app, such as a login check or a rule about who may edit which data.
- **Significance test / p-value.** A statistical check of whether a difference could easily be due to chance. A small p-value (for example below 0.05) suggests it probably is not.
- **Snowballing.** Finding more papers by following the references and citations of papers you already have.
- **SonarQube.** A widely used tool that scans code and reports bugs, code smells and security warnings.
- **Static analysis (SAST) / dynamic testing (DAST).** Static analysis checks code without running it; dynamic testing attacks or probes the running app.
- **Systematic review.** A review that tries to collect every study on a question using a fixed, documented procedure.
- **Technical debt.** Future extra work caused by quick or poor solutions now.
- **Underpowered.** A study with too few participants to reliably detect small effects.
- **Vibe coding.** Building software mainly by describing what you want to an AI in natural language and accepting the code it writes, with little manual review.
- **Visual fidelity.** How closely a rebuilt app looks like the original.

# Appendix B: Methodology Comparison

Project: 2026-11-11-ai-reverse-engineering-vibe-coding · Date: 2026-11-12 · Basis: 23 complete, full-text, critically reviewed analyses (papers/*/analysis.md) and evidence-matrix.md.

Labels: **[V]** verified in the paper, **[A]** author claim, **[I]** our interpretation. Seven papers are preprints (not peer-reviewed): bai2026recreationworld, chen2025iwr, kim2026programdistill, mota2026comparing, pan2026functional, yang2026programbench, zhao2025vibe. They are marked † below and carry less weight.

Terms used:

- An *oracle* is the reference that decides whether an output is correct, such as a test suite or a human judge.
- *Functional fidelity* means how closely a rebuilt app behaves like the original.
- An *LLM judge* is a language model used to score outputs in place of humans.

## 1. Overview table

All cells are **[V]** from the analyses unless marked otherwise. "n.r." = not reported.

| Paper | Design | Data / sample (N) | Setup | Metrics | Baselines | Statistical testing | Reproducibility |
|---|---|---|---|---|---|---|---|
| asdaque2026novice [asdaque2026novice, p. 2] | MSR repository mining (5-page paper) | 22,953 agentic PRs from 1,719 users, AIDev, repos >100 stars | Low- vs high-experience split by commits / account age | Commits and files per PR, acceptance, time to merge, review comments | High-experience users; no non-AI PRs | Mann-Whitney, Chi-square, Benjamini-Hochberg; no effect sizes or CIs; test unit unclear [asdaque2026novice, p. 4] | Public dataset + replication package [asdaque2026novice, p. 5] |
| bai2026recreationworld † [bai2026recreationworld, p. 7] | Benchmark + SFT transfer | 250 apps (50 per platform: Ubuntu, macOS, Windows, Android, Web) x 10 models; rollouts n.r. (probably 1) | Isolated VMs; Codex or Claude Code scaffolds (model and scaffold confounded) [bai2026recreationworld, p. 28] | Programmatic accessibility/DOM tests + VLM assertions; LOC, files, largest-file share | Model vs model | No CIs or tests on main scores; bootstrap CIs on secondary analyses [bai2026recreationworld, p. 12] | Release claimed **[A]** [bai2026recreationworld, p. 27]; proprietary models; COI not declared **[I]** |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| borg2026echoes [borg2026echoes, p. 24] | Preregistered 2-phase experiment; Phase 2 RCT | 151 professionals; Phase 1 n=76, Phase 2 n=75 (128 planned); 2 KLoC Java app | Second developer evolves AI-assisted or non-AI code without AI | Completion time, CodeScene CodeHealth (customised), JaCoCo coverage, perceived productivity | Non-AI code (control) | Wilcoxon/Welch, Cliff's δ / Cohen's d, bootstrap, Bayesian; no multiple-testing correction [borg2026echoes, p. 19] | Zenodo + GitHub [borg2026echoes, p. 74]; COI declared [borg2026echoes, p. 75] |
| chen2025iwr† [chen2025iwr, p. 6] | Benchmark | 113 video tasks from 100 websites x 28 LVLMs; runs n.r. | Video (≤64 frames) + assets → one HTML file; agent judge (Gemini-2.5-Pro) | IFS (action replay + assertions), VFS (OCR+DINO+MLLM), Final = 0.7 IFS + 0.3 VFS | Model vs model | None; weights tuned in-sample on 60 human-rated cases (ρ 0.57/0.65) [chen2025iwr, p. 17] | Release promised **[A]** [chen2025iwr, p. 1] |
| ehsani2026faster [ehsani2026faster, p. 1] | Multivocal literature review | 31 peer-reviewed + 73 grey sources (to Jan 2026) | Dual extraction; κ 0.93 formal [ehsani2026faster, p. 13] | Mention counts per debt type, tool, metric | Grey vs formal | None (descriptive) | figshare supplement [ehsani2026faster, p. 4] |
| fawzy2026vibe [fawzy2026vibe, p. 5] | Grey-literature review, thematic analysis | 101 sources, 518 coded units | Mainly single coder; subset cross-check [fawzy2026vibe, p. 4] | Theme frequencies (quotes, not people) | None | None; no agreement statistic | Zenodo package [fawzy2026vibe, p. 10] |
| he2026speed [he2026speed, p. 4] | Staggered DiD + dynamic-panel GMM | 806 Cursor-adopting repos + 1,172–1,380 matched controls; 21,699 repo-months | Monthly panel Jan 2024–Aug 2025 | Commits, lines added; SonarQube warnings, cognitive complexity, duplication | Matched non-adopters (may use other AI) | ATT + SEs, pre-trend tests, Sargan/AR; estimator-sensitive [he2026speed, p. 16] | Zenodo [he2026speed, p. 11]; SonarQube version n.r. |
| kim2026programdistill† [kim2026 | Benchmark (LLM-built) | 300 partial tasks x 9 models; full rebuild 12 | Browser CLI to a live reference; | Trace-replay binary and chain scores; | Model vs model; no | None (means only) | Not yet released **[A]** [kim2026pr |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| programdistill, p. 8] | | apps x 3 models = 36 runs | text observations only | atomic and cumulative recovery | human baseline | | ogramdistill, p. 1] |
| liu2026benchmarking [liu2026benchmarking, p. 3] | Benchmark from existing apps | 46 web apps, 244 requirements, 703 BDD tests; 6 frameworks x 6 LLMs x 3 runs | Requirements + data-testid anchors in prompt [liu2026benchmarking, p. 16] | Req.Acc, Test.Acc, cost, time | Vanilla LLM | Mean ± SD; no tests [liu2026benchmarking, p. 6] | Code + data released **[A]** [liu2026benchmarking, p. 9] |
| lu2025webgen [lu2025webgen, p. 5] | Benchmark + SFT | 101 instructions, 647 test cases x 16 settings; single run | Bolt.diy / OpenHands / Aider | UI-agent accuracy (half credit for PARTIAL), appearance 1–5 | Frameworks and models; no human baseline | None ("No" to error bars) [lu2025webgen, p. 40] | Code, data, weights **[A]** [lu2025webgen, p. 1] |
| mock2026vibe [mock2026vibe, p. 16] | Static-group comparison + exploratory agent runs | 9 solo + 7 collaborative professionals; 3 fully-automated + 12 agentic runs; 2 small Python tasks | GPT-3.5 for humans vs GPT-5/Claude Sonnet 4 for agentic [mock2026vibe, p. 17] | Baseline-test pass rate, McCabe, coverage, time | Solo human coding | KW, one-tailed MWU, Cliff's δ, Holm [mock2026vibe, p. 10] | GitHub package [mock2026vibe, p. 43] |
| molison2025llm [molison2025llm, p. 4] | Comparative static analysis | APPS; GPT-4o zero/few/fine-tuned vs humans; N per group n.r.; Pass@1 on 15 per approach | SonarQube default profile [molison2025llm, p. 5] | Bugs, smells, severity, fix effort, Pass@1 | APPS human solutions | ORs, MWU + Cliff's δ; no CIs or correction [molison2025llm, p. 8] | Zenodo [molison2025llm, p. 11] |
| mota2026comparing† [mota2026comparing, p. 2] | Exploratory static analysis | 9 apps (Lovable, v0, Replit x 3), one prompt | Free tiers, May 2026 | SonarQube issues, severity, effort, duplication, per KLOC | Tools vs each other; no human or | None (descriptive) [mota2026comparing, p. 6] | Code + reports shared [mota2026comparing, p. 6] |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| | | | | | functional check | | |
| pan2026functional† [pan2026functional, p. 2] | Benchmark | 38 Python repos x 16 LLMs x 3 runs; ~11 functional tests per task [pan2026functional, p. 5] | Zero-shot single-pass repository generation | Functional pass rate; reference-normalised MI, security, robustness, efficiency, resource | Model vs model; reference as normalisation anchor | None; reruns + rank robustness [pan2026functional, p. 10] | GitHub **[A]** [pan2026functional, p. 11] |
| si2025benchmarking [si2025benchmarking, p. 3] | Benchmark | 484 webpages (+80 HARD); human eval on 100 pages x 5 annotators | Screenshot → single HTML; greedy decoding | Block-Match, Text, Position, Color, CLIP; human win/tie/lose | MLLMs and prompts | Logistic regression validation (79.9%); no model tests [si2025benchmarking, p. 7] | Release claimed, no URL [si2025benchmarking, p. 10] |
| tran2026vibe [tran2026vibe, p. 3] | Benchmark | 50 test apps (964 workflows total) x 16 models; 1 generation each **[I]** | OpenHands-based harness, React/Vite + Supabase | Workflow pass (≥90% substeps) by LLM browser agent; cost; latency | Model vs model | Mean ± SE across apps; variance panel; human alignment [tran2026vibe, p. 8] | Harness + trajectories released **[A]** [tran2026vibe, p. 11] |
| wu2026looks [wu2026looks, p. 3] | Benchmark | 29 apps, 309 screens, 701 nav specs x 6 models = 174 single runs | Screenshots + page-relationship description → React/TS | Build, NavPass, Borda VLM visual, LoC/reuse/dead-component proxies | Model vs model | Wilson CIs for failure sample only [wu2026looks, p. 8] | Repo + one-command replay **[A]** [wu2026looks, p. 10] |
| xiao2025benchmarking [xiao2025benchmarking, p. 3] | Benchmark + user study | 127 pages / 374 interactions x 6 MLLMs x 3 prompts; user study 4 people | Before/after interaction screenshots → one HTML file | CLIP, SSIM, text, Implement Rate; human Usability Rate | Direct prompting | None; human–metric "consistency" claimed only **[A]** [xiao2025benchmarking, p. 6] | GitHub **[A]** [xiao2025benchmarking, p. 1] |
| xiong20 | Tool + | 12 Android | GPT-5.2 | Functionalit | DroidA | Cohen's κ | Zenodo |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 26propgen [xiong2026propgen, p. 7] | evaluation | apps; 985 properties; single run | runtime exploration + property synthesis | y / property validity, refinement, new bugs | gent, naive LLM, 4 bug detectors | 0.81–0.91 only | **[A]** [xiong2026propgen, p. 13] |
| yang2026programbench† [yang2026programbench, p. 4] | Benchmark | 200 programs x 9 models = 1,800 runs; 248,853 tests | Execute-only binary + docs, no internet, mini-SWE-agent [yang2026programbench, p. 5] | % resolved, % almost, % tests passed, structure stats | Model vs model; native test suites for coverage | None; Fleiss' κ for cheating judges [yang2026programbench, p. 17] | Open-sourced **[A]** [yang2026programbench, p. 2] |
| zhao2025vibe† [zhao2025vibe, p. 6] | Benchmark | 186 tasks from 100 Python repos x 12 agent settings; single pass@1 | Agents re-implement a masked feature | FuncPass, SecPass, CWE P/R/F1 | Settings vs each other | None [zhao2025vibe, p. 8] | GitHub **[A]** [zhao2025vibe, p. 1] |
| zhou2025declarui [zhou2025declarui, p. 6] | Technique + comparison + user study | 50 apps (250 pages); ablation 13 apps; 5 developers x 12 apps | CV + MLLM page-transition graph + compile/navigation refinement; human graph correction [zhou2025declarui, p. 4] | CLIP, SSIM, PCR, CSR, ACIC, AMCT; Likert readability and maintainability | Raw GPT-4o / Claude-3.5 | MWU on non-independent ratings only [zhou2025declarui, p. 10] | No code or data link (n.r.) |
| zhu2026understanding [zhu2026understanding, p. 4] | Exploratory static analysis (2 experiments) | Exp I: 90 problems x 5 LLMs x 2 prompts + 90 human; Exp II: ~20 MetaGPT apps **[I]** | PyExamine smell detection | Code / structural / architectural smell counts, LoC | Human solutions (Exp I) | Exp I none; Exp II Spearman / KW without correction [zhu2026understanding, p. 9] | Zenodo [zhu2026understanding, p. 10] |

## 2. Approach families

**F1. Visual UI-to-code (screenshot or design → code):** si2025benchmarking, xiao2025benchmarking, zhou2025declarui, wu2026looks.

- Strengths **[I]**: real webpages and apps as references [si2025benchmarking, p. 2; wu2026looks, p. 3]. Each paper also adds some reliability check:
- visual metrics partly validated against humans [si2025benchmarking, p. 7];
- a human-calibrated VLM judge with an anchor [wu2026looks, p. 13];
- navigation measured with app exploration [zhou2025declarui, p. 7].
- Weaknesses **[I]**:
- Fidelity is mostly visual. si2025benchmarking strips JavaScript [si2025benchmarking, p. 13].
- Interaction coverage is shallow: navigation edges only [wu2026looks, p. 5], or element presence as "Implement Rate" [xiao2025benchmarking, p. 4].
- Structural priors are given as input: a page-relationship description [wu2026looks, p. 3] or a human-corrected page-transition graph [zhou2025declarui, p. 4].
- Code quality is unmeasured or measured by proxies or Likert ratings.

**F2. Behaviour-observed reconstruction (video, live reference or executable → app):** chen2025iwr†, bai2026recreationworld†, kim2026programdistill†, yang2026programbench†.

- This family is the closest match to "AI-assisted reverse engineering" **[I]**.
- Strengths **[I]**: implementation-agnostic behavioural oracles [bai2026recreationworld, p. 4; yang2026programbench, p. 5; kim2026programdistill, p. 4]. The papers also use:
- leakage and isolation controls [yang2026programbench, p. 18; kim2026programdistill, p. 10; bai2026recreationworld, p. 9];
- oracle-quality audits: coverage [yang2026programbench, p. 7] and interaction-outcome share [bai2026recreationworld, p. 9].
- Weaknesses **[I]**:
- All four are preprints, and all use single runs.
- Same-family builders appear in yang2026programbench and kim2026programdistill (see Contradiction C10 in the evidence matrix).
- Code quality is measured only through size and structure proxies [bai2026recreationworld, p. 15; yang2026programbench, p. 9] or not at all [kim2026programdistill, p. 6].
- Oracles are coupled to the original's selectors in kim2026programdistill [kim2026programdistill, p. 35].

**F3. Specification-to-app benchmarks with executable oracles:** liu2026benchmarking, lu2025webgen, tran2026vibe, pan2026functional†, zhao2025vibe†.

- Strengths **[I]**: requirement-level executable tests. Several papers check their oracles:
- expert test audits [liu2026benchmarking, p. 14];
- judge–human alignment [tran2026vibe, p. 9; lu2025webgen, p. 9];
- reference-validated tests [pan2026functional, p. 2];
- upstream human tests with no LLM judge [zhao2025vibe, p. 5].
- Two papers start from existing apps, which brings them close to reconstruction: liu2026benchmarking and pan2026functional **[I]**.
- Weaknesses **[I]**:

- The input is a written specification, not observed behaviour.
- LLM judges and LLM-drafted tests are used (tran2026vibe, lu2025webgen).
- Oracles are thin: about 11 tests per task [pan2026functional, p. 5].
- Only pan2026functional measures maintainability, through one MI proxy [pan2026functional, p. 3].

**F4. Code-quality studies of LLM or vibe-coded code (static analysis):** molison2025llm, mota2026comparing†, zhu2026understanding, he2026speed (plus the non-functional arm of pan2026functional).

- Strengths **[I]**: established tools (SonarQube, PyExamine), and one quasi-causal design at scale [he2026speed, p. 7].
- Weaknesses **[I]**:
- The instruments disagree with each other (C3).
- Counts are often not normalised by size [zhu2026understanding, p. 8].
- Functional correctness is not checked [mota2026comparing, p. 2], or checked only on 15 samples [molison2025llm, p. 8].
- Human baselines are weak (competitive-programming submissions) [molison2025llm, p. 9].
- Tasks are algorithmic snippets, not applications (molison2025llm, zhu2026understanding Exp I).

**F5. Human-centred and workflow studies:** borg2026echoes (RCT), mock2026vibe (experiment), asdaque2026novice (mining), fawzy2026vibe (grey review), ehsani2026faster (multivocal review).

- Strengths **[I]**:
- The only randomised maintainability evidence, a hand-off evolution design [borg2026echoes, p. 13].
- Real developers [mock2026vibe, p. 16].
- Practitioner definitions of vibe coding [fawzy2026vibe, p. 1].
- A technical-debt taxonomy [ehsani2026faster, p. 16].
- Weaknesses **[I]**:
- Small or underpowered samples [borg2026echoes, p. 19; mock2026vibe, p. 13].
- Model-generation confound [mock2026vibe, p. 17].
- Perception or proxy outcomes [fawzy2026vibe, p. 8; asdaque2026novice, p. 2].
- Reviews rely heavily on grey sources [ehsani2026faster, p. 16].
- None studies reconstruction.

**F6. Behavioural-specification recovery from running apps:** xiong2026propgen.

- Strengths **[I]**: runtime-grounded property inference with independent annotators (κ 0.81–0.91) [xiong2026propgen, p. 7]. It is a candidate oracle generator for fidelity testing.
- Weaknesses **[I]**: 12 apps, single run, author-tool overlap in the baselines, and it generates no code.

## 3. Comparable results

**Rule:** numbers are compared only between papers that share the dataset, the metric and the setup. **No pair of papers in this corpus meets all three.** So all cross-paper statements below are directional, not quantitative.

| Pair (closest candidates) | What they share | Why the numbers are not comparable |
|---|---|---|
| si2025benchmarking vs xiao2025benchmarking | C4 validation-set pages, CLIP [si2025benchmarking, p. 2; xiao2025benchmarking, p. 2] | Static pages vs interaction regions; different subsets and models; different extra metrics |
| zhou2025declarui vs wu2026looks | Multi-screen mobile apps rebuilt from screenshots | PCR (graph edges, scored manually) vs NavPass (Playwright tests); native frameworks vs a React web proxy; different structural inputs |
| bai2026recreation world vs chen2025iwr | A functional channel plus a visual channel | Accessibility tests + VLM assertions vs action replay + OCR/DINO/MLLM; VFS only over reached checkpoints [chen2025iwr, p. 7] |
| lu2025webgen vs tran2026vibe | NL spec → web app; LLM browser judge | Atomic test cases with half credit vs workflows passing at ≥90% of substeps; different model generations |
| liu2026benchmarking vs pan2026functional | Rebuild of existing open-source apps against reference-validated tests | Web BDD via Selenium with data-testid anchors vs Python pytest on public APIs; different models |
| molison2025llm vs zhu2026understanding (Exp I) | Competitive-programming problems vs human solutions | APPS + SonarQube vs CodeContests + PyExamine; different models |
| he2026speed vs mota2026comparing | SonarQube issue measures | Change in repository-month totals vs per-KLOC issues in 9 generated apps |
| pan2026functional vs zhao2025vibe | Python repositories, test-based correctness, a "security" score | Whole-repo generation + static findings vs feature patch + targeted security tests (C7) |

Headline functional-fidelity figures, listed for orientation only. They are **not comparable** because each has a different oracle, input and app type **[I]**:

| Paper | Input → output | Oracle | Best reported |
|---|---|---|---|
| yang2026programbench † | Executable + docs → CLI program | ~770 behavioural tests per task (median) | 0% resolved; ≥95% of tests on 3% of tasks [yang2026programbench, p. 6] |
| bai2026recr | Running app → app (5 | Accessibility/DOM | 58.06% average; 2.8% full |

| | | | |
|---|---|---|---|
| eationworld† | platforms) | tests + VLM | programmatic pass [bai2026recreationworld, p. 11] |
| kim2026programdistill† | Live web app → web app | Browser-trace replay | 49.15% cumulative workflows (3 models, 12 apps) [kim2026programdistill, p. 11] |
| chen2025iwr† | Interaction video → single HTML | Agent replay + MLLM assertions | Final 36.35, IFS 24.39 [chen2025iwr, p. 8] |
| liu2026benchmarking | Requirements from existing apps → web app | BDD/Selenium tests | 53.75% Req.Acc [liu2026benchmarking, p. 6] |
| tran2026vibe | NL spec → full-stack web app | LLM browser workflows | 61.77% workflows [tran2026vibe, p. 6] |
| lu2025webgen | NL instruction → website | UI-agent test cases | 27.8% accuracy [lu2025webgen, p. 8] |
| pan2026functional† | Requirements → Python repo | pytest suites | <45% pass rate [pan2026functional, p. 5] |
| zhao2025vibe† | Feature description → patch | Upstream functional tests | 57.0% FuncPass [zhao2025vibe, p. 7] |
| wu2026looks | Screenshots + navigation map → React app | Playwright navigation | 92% NavPass [wu2026looks, p. 7] |
| zhou2025declarui | Screenshots/designs → mobile UI | Manual graph-edge coverage | 96.8% PCR [zhou2025declarui, p. 9] |

Within-paper model rankings are comparable, but single runs make near-ties uninterpretable:

- **[V]** tran2026vibe shows that generation variance dominates evaluator variance (73.5–92.5%) [tran2026vibe, p. 8].
- **[I]** Rank gaps of a few points in yang2026programbench, wu2026looks, lu2025webgen and zhao2025vibe cannot be separated from noise.

## 4. Contradictions

The full entries with both sides, differences and **[I]** explanations are in evidence-matrix.md → "Contradictions". Summary:

1. **C1 Maintainability harm.** Project-level warnings and complexity rise after Cursor adoption [he2026speed, p. 7], but a second developer's effort and CodeHealth show no significant difference [borg2026echoes, p. 33; borg2026echoes, p. 38]. Tools, units, scale and time differ, and the he2026speed effect depends on the estimator [he2026speed, p. 16].
2. **C2 Static issues.** LLM code has fewer SonarQube bugs than APPS humans [molison2025llm, p. 8], vs +30% repository warnings [he2026speed, p. 7]. The units and baselines differ, so the results are not comparable.

3. **C3 Instrument choice.** PyExamine is chosen over SonarQube's "superficial linting" [zhu2026understanding, p. 2]. Technical-debt tools disagree [ehsani2026faster, p. 5].
4. **C4 Iterative feedback.**
   - It hurts non-functional quality without functional gain [pan2026functional, p. 8].
   - It correlates with or enables better outcomes elsewhere [tran2026vibe, p. 7; zhou2025declarui, p. 10; wu2026looks, p. 12].
   - The studies differ in feedback type, outcome and design (intervention vs correlation).
1. **C5 Looks vs behaves.** chen2025iwr and bai2026recreationworld are only directionally consistent. VFS > IFS is partly built into chen2025iwr's metrics [chen2025iwr, p. 7]. In bai2026recreationworld the visual channel is below the programmatic one for 9 of 10 models [bai2026recreationworld, p. 11].
2. **C6 MetaGPT.** It nearly always fails in liu2026benchmarking [liu2026benchmarking, p. 6] but leads on tiny tasks with newer models in mock2026vibe [mock2026vibe, p. 41].
3. **C7 Security.** 87.40% static security [pan2026functional, p. 6] vs 79.3% of correct solutions failing targeted tests [zhao2025vibe, p. 6].
4. **C8 Code volume.** Rebuilds are smaller than the originals [bai2026recreationworld, p. 15; yang2026programbench, p. 9], while assisted development adds code [he2026speed, p. 8; borg2026echoes, p. 30].
5. **C9 Construct differences.** "Vibe coding" is defined differently in [fawzy2026vibe, p. 1; asdaque2026novice, p. 1; mock2026vibe, p. 15; he2026speed, p. 4; ehsani2026faster, p. 17], so these results must not be pooled. The fidelity scores of kim2026programdistill, liu2026benchmarking, lu2025webgen, bai2026recreationworld and chen2025iwr use different oracles and are not comparable.
6. **C10 Same-family construction or judging.** The same model family builds or judges the benchmark and then wins in yang2026programbench, tran2026vibe, kim2026programdistill and zhao2025vibe. There is overlap without a visible advantage in liu2026benchmarking, lu2025webgen, bai2026recreationworld, chen2025iwr and wu2026looks. This is untested everywhere **[I]**.

## 5. Methodological weaknesses across the field

Counts are our tally over the analyses **[I]**. Each pattern is verified in the cited papers.

1. **Single runs, no variance, no significance tests.**
   - Main results come from one run or generation per cell, or the number of runs is not reported. Examples: [bai2026recreationworld, p. 11; chen2025iwr, p. 8; kim2026programdistill, p. 12; lu2025webgen, p. 40; wu2026looks, p. 9; yang2026programbench, p. 10; zhao2025vibe, p. 6; xiong2026propgen, p. 8].
   - Studies that repeat runs: liu2026benchmarking (3 runs) [liu2026benchmarking, p. 5], pan2026functional (3 runs + 5 reruns) [pan2026functional, p. 5], and tran2026vibe's 6-app variance panel [tran2026vibe, p. 8].
   - No benchmark paper tests model differences for significance. Where measured, run-to-run variance is large: generation SE up to 5 pp [tran2026vibe, p. 8].
1. **Unvalidated or same-family oracles and judges.**

- Many papers use LLM-drafted tests, LLM-built tasks or LLM judges, often from the evaluated families (C10).
- Judge validation uses raw percent agreement without chance correction [tran2026vibe, p. 9; lu2025webgen, p. 9], small or author-labelled panels [wu2026looks, p. 13], or in-sample calibration [chen2025iwr, p. 17; si2025benchmarking, p. 17].
- Some automatic metrics are only declared "consistent" with humans [xiao2025benchmarking, p. 6].

1. **Contamination handling is uneven.**
   - Reference apps and programs are popular public projects in most benchmarks.
   - Explicit controls appear in only a few: an overlap audit [bai2026recreationworld, p. 18], no internet plus a language ablation [yang2026programbench, p. 6], BLEU against the original [xiao2025benchmarking, p. 5], post-cutoff builds [xiong2026propgen, p. 11], and keeping task code out of public repositories [borg2026echoes, p. 66].
   - pan2026functional acknowledges exposure [pan2026functional, p. 2], and its example prompt names the original project [pan2026functional, p. 3].
   - No contamination discussion was found for chen2025iwr, kim2026programdistill, liu2026benchmarking, si2025benchmarking, wu2026looks, zhao2025vibe, molison2025llm or zhu2026understanding (per their analyses) **[I]**.
1. **Code quality and maintainability are absent or measured by unvalidated proxies.**
   - The proxies used are size and structure [bai2026recreationworld, p. 15; yang2026programbench, p. 9], LoC/reuse/dead-component counts [wu2026looks, p. 4], a single MI index [pan2026functional, p. 3], Likert ratings [zhou2025declarui, p. 7], and raw smell counts not normalised by size [zhu2026understanding, p. 8].
   - No reconstruction benchmark combines a behavioural oracle with a validated maintainability measure or a hand-off maintenance task like the one in borg2026echoes [borg2026echoes, p. 13] **[I]**.
1. **Oracle coupling and incompleteness make fidelity scores incomparable.**
   - Tests are tied to the original's identifiers: data-testid anchors [liu2026benchmarking, p. 16] and selectors [kim2026programdistill, p. 35].
   - Structural priors are given as input [wu2026looks, p. 3; zhou2025declarui, p. 4].
   - Suites are thin: about 11 tests per task [pan2026functional, p. 5].
   - Finite suites can only give a lower bound [yang2026programbench, p. 12; bai2026recreationworld, p. 22].
   - Metrics are partly circular: navigation is scored against the method's own input graph [zhou2025declarui, p. 7], and VFS uses a different denominator [chen2025iwr, p. 7].
1. **Further recurring problems:**
   - No human-developer baseline in any reconstruction or generation benchmark [yang2026programbench, p. 20; tran2026vibe, p. 5; lu2025webgen, p. 7].
   - Small human samples [mock2026vibe, p. 16; zhou2025declarui, p. 10; xiao2025benchmarking, p. 11].
   - Underpowered RCT [borg2026echoes, p. 19].
   - Internal numeric inconsistencies, flagged in the analyses of borg2026echoes, he2026speed, liu2026benchmarking, molison2025llm, wu2026looks, xiao2025benchmarking, yang2026programbench and zhou2025declarui (see each paper's §9) **[I]**.

- Construct heterogeneity of "vibe coding" (C9).
- Almost exclusively web, Python or CLI targets. Only bai2026recreationworld covers native desktop and Android [bai2026recreationworld, p. 7]; mobile work uses a web proxy [wu2026looks, p. 9] or UI-only code [zhou2025declarui, p. 2].

## 6. What this means for our RQs

**RQ1 (approaches and their relation to vibe coding).**

- Six approach families exist (§2). Inputs range from screenshots, designs, interaction videos and before/after interaction screenshots to live running references, executables plus docs, and natural-language specs [si2025benchmarking, p. 4; chen2025iwr, p. 5; bai2026recreationworld, p. 3; kim2026programdistill, p. 3; yang2026programbench, p. 2; tran2026vibe, p. 3] **[V]**.
- Almost all evaluations run autonomous agents or single prompts. Practitioner vibe coding is human-steered natural-language prompting with little review [fawzy2026vibe, p. 1], and that is studied only in small human experiments [mock2026vibe, p. 15] or as a field proxy [he2026speed, p. 4] **[I]**.
- Evidence strength: **moderate**. The descriptive picture is consistent, but 7 of 23 papers are preprints and the field moves monthly.

**RQ2 (measuring functional fidelity and how well it is achieved).**

- Fidelity is measured by four oracle types **[I]**:
- behavioural test replay (bai2026recreationworld, kim2026programdistill, yang2026programbench, liu2026benchmarking, pan2026functional, zhao2025vibe);
- LLM or agent judges running natural-language tests (lu2025webgen, tran2026vibe, chen2025iwr);
- visual similarity (si2025benchmarking, xiao2025benchmarking, wu2026looks, zhou2025declarui);
- navigation graphs (wu2026looks, zhou2025declarui).
- Achieved fidelity is partial everywhere:
- full equivalence is rare [bai2026recreationworld, p. 11; yang2026programbench, p. 6];
- the best workflow-level or requirement-level pass rates are about 49–62% [kim2026programdistill, p. 11; liu2026benchmarking, p. 6; tran2026vibe, p. 6];
- interaction logic lags static structure (G-A2).
- Evidence strength: **moderate** for "fidelity is far from complete and interaction is the bottleneck". **Weak** for any specific number or cross-benchmark ranking (C9, §3).

**RQ3 (code quality and maintainability vs human code).**

- The evidence is mixed and does not support a single answer.
- Project-level warnings and complexity rise, but the result is estimator-sensitive and size-driven [he2026speed, p. 7; he2026speed, p. 16].
- No short-term downstream maintainability harm was found in an RCT [borg2026echoes, p. 38].
- LLM code has fewer SonarQube bugs than a weak human baseline, with mixed smells [molison2025llm, p. 8].

- LLM code shows different smell types [zhu2026understanding, p. 7].
- Quality is platform-dependent [mota2026comparing, p. 5].
- Rebuilds are smaller and more monolithic than the originals [bai2026recreationworld, p. 15; yang2026programbench, p. 9].
- Practitioners perceive the code as "fast but flawed" [fawzy2026vibe, p. 8].
- No study measures validated maintainability of *reconstructed* applications against their originals **[I]**.
- Evidence strength: **weak to moderate** (conflicting results, C1–C3).

**RQ4 (methodological strengths and weaknesses).**

- Strengths: implementation-agnostic behavioural oracles with leakage controls (F2), oracle audits (coverage, expert κ), judge–human alignment checks, and one preregistered RCT.
- Weaknesses: the six patterns in §5.
- Evidence strength: **strong** for missing variance and significance testing and for absent maintainability measurement (verified across 12+ papers). **Moderate** for judge, contamination and oracle-coupling problems.

**RQ5 (gaps and study design).**

- The convergent weak point is the missing joint evaluation of behavioural fidelity *and* validated maintainability for UI- or behaviour-based reconstruction **[I]**. The partial exceptions are pan2026functional (Python APIs, MI only), wu2026looks (proxies) and zhou2025declarui (Likert).
- Candidate gaps for the gap analyst are listed below. **None is claimed as a gap yet: each needs the novelty check in docs/research-standards.md §7.**

1. Joint fidelity + maintainability evaluation of AI-reconstructed apps against their originals, using static quality with size normalisation and more than one tool, plus a hand-off maintenance task as in borg2026echoes.
2. Human-steered vibe-coding reconstruction compared with autonomous agents. All benchmarks are autonomous; future-work statements: [yang2026programbench, p. 12; kim2026programdistill, p. 14].
3. Contamination-controlled reconstruction targets, such as post-cutoff, obfuscated or private reference apps (uneven handling, §5.3).
4. Judge and oracle independence: cross-family judges, chance-corrected agreement, and tests of builder- or judge-family self-preference (C10).
5. Whether iterative feedback or self-testing improves fidelity *and* code quality together (C4).
6. Validity of maintainability instruments for AI-generated code: SonarQube vs PyExamine vs CodeHealth vs MI (C3).
7. Black-box oracles that are not coupled to the original's identifiers (§5.5).
8. Native mobile and back-end-inclusive reconstruction.

- A design sketch **[I]**:
- reference apps of known provenance (post-cutoff or private);
- several vibe-coding tools and agents, plus a human-steered arm;
- ≥3 runs per cell;

- a behavioural oracle (trace replay or BDD, black-box selectors) plus a validated visual judge;
- size-normalised static quality from two tools plus a second-developer change task;
- cross-family judges with κ reporting and preregistered tests.
- Evidence strength: **weak** (pending novelty check).

# Appendix C: Evidence Matrix (all 204 claims)

One row per piece of evidence that bears on a research question. Source is a pandoc citation with the paper key and PDF page (checked by rlab cite-check). Label: **[V]** VERIFIED (checked in the paper's own data/results), **[A]** AUTHOR-CLAIM (stated by authors, not independently checkable), **[I]** INTERPRETATION (our reasoning). Strength: strong / moderate / weak (see docs/research-standards.md §5).

How this matrix was built (compare-methods, 2026-11-12):

- All 204 rows are merged from section 13 of the 23 analysis.md files (all status: complete, all full-text, all critically reviewed). Claims keep the analyst's label and wording except where the critical review required a correction (liu2026benchmarking cost/multi-agent row; pan2026functional non-functional row; see Notes).
- **Preprint downgrade**: the 7 preprints (bai2026recreationworld, chen2025iwr, kim2026programdistill, mota2026comparing, pan2026functional, yang2026programbench, zhao2025vibe; venue_kind: preprint) are downgraded one level (strong → moderate, moderate → weak; weak stays weak). The analyst's original strength is kept in Notes. No paper is abstract-only, so no abstract-only downgrade applies.
- The ehsani2026faster row restating he2026speed's +30% warnings / +41.6% complexity is kept for traceability but **not counted**; the effect is counted once, from he2026speed.
- "Design & N" is a short summary; full details are in reports/methodology-comparison.md §1.

| RQ | Claim / finding | Source | Label | Design & N | Strength | Notes |
|---|---|---|---|---|---|---|
| RQ 1 | Hybrid agents can rebuild runnable apps from a running reference (no source on desktop/Android) across Ubuntu, macOS, Windows, Android and Web; 10 frontier models evaluated on 250 tasks | [bai2026recreationworld, p. 3; bai2026recreationworld, p. 7; bai2026recreationworld, p. 11] | **[V]** | Benchmark; 250 apps x 10 models, 5 platforms, rollouts not reported (probably 1) | weak | preprint: downgraded from moderate |
| RQ 1 | LVLMs can be asked to rebuild an interactive webpage from a user-interaction video plus crawled assets (one HTML file); 28 models were evaluated on 113 real-site tasks | [chen2025iwr, p. 5; chen2025iwr, p. 8] | **[V]** | Benchmark; 113 video tasks x 28 LVLMs, single run | weak | preprint |
| RQ 1 | Vibe coding is framed as a driver of fast-integration | [ehsani2026faster, p. 17; ehsani2026f | **[A]** | Multivocal review; 31 | weak | analyst: grey only |

| | | | | | | |
|---|---|---|---|---|---|---|
| | debt, which leads to governance debt | aster, p. 26] | | formal + 73 grey sources | | |
| RQ 1 | Vibe coding defined as natural-language goal prompting plus iterative re-prompting with minimal code review; speed is the dominant motivation (87/140 motivation units, 62%) | [fawzy2026vibe, p. 1; fawzy2026vibe, p. 6] | [V] | Grey-literature review; 101 sources, 518 coded units | moderate | |
| RQ 1 | Common vibe-coding QA behaviours: skipped QA (36%), uncritical trust (18%), delegating QA to the AI (10%), reprompting instead of debugging (5%) of 132 QA units | [fawzy2026vibe, p. 8] | [V] | Grey-literature review; 101 sources, 518 coded units | weak | |
| RQ 1 | Reference-guided setting: agent interacts with a live reference web app via a browser CLI (structured text observations, no source, no screenshots) and edits a masked repo or a blank scaffold to reproduce its behaviour. | [kim2026programdistill, p. 3; kim2026programdistill, p. 6; kim2026programdistill, p. 24] | [V] | Benchmark; 300 partial tasks x 9 models; 12 apps x 3 models full | weak | preprint: downgraded from moderate; analyst: design fact |
| RQ 1 | Agentic/multi-agent frameworks do not consistently improve over a plain prompted LLM (authors' wording); by our count on Table 3, 9 of 24 multi-agent framework x backbone cells beat Vanilla on Req.Acc, mostly by small margins. MapCoder, ChatDev and MetaGPT cost several times more than Vanilla on small absolute costs; Self-Collaboration costs about the same. MetaGPT nearly always fails (0–5.4% Req.Acc) | [liu2026benchmarking, p. 6] | [V] | Benchmark; 46 web apps x 36 settings x 3 runs | moderate | 9/24 count and cost comparison are our arithmetic on Table 3 [I]; the paper states no cost multiplier |
| RQ 1 | Same frameworks score 71–91% Pass@1 on HumanEval but ~10–54% | [liu2026benchmarking, p. 27] | [V] | Benchmark; 46 web apps x 36 settings | moderate | |

| | | | | | | |
|---|---|---|---|---|---|---|
| | Req.Acc on E2EDev | | | x 3 runs | | |
| RQ 1 | Agent framework strongly affects outcome: DeepSeek-V3 scores 20.8% in Bolt.diy, 14.1% in Aider, 9.0% in OpenHands | [lu2025webgen, p. 8] | [V] | Benchmark; 101 instructions / 647 tests x 16 settings, single run | weak | |
| RQ 1 | Fine-tuning Qwen2.5-Coder-32B on 600 filtered agent trajectories lifts accuracy from 9.5% to 38.2% | [lu2025webgen, p. 8] | [V] | Benchmark; 101 instructions / 647 tests x 16 settings, single run | weak | |
| RQ 1 | "Vibe coding" operationalised as human-written TDD tests plus CLLM-generated code via fixed scripted prompts; agentic coding via MetaGPT X as the autonomous contrast | [mock2026vibe, p. 7; mock2026vibe, p. 8; mock2026vibe, p. 15; mock2026vibe, p. 17] | [V] | Static-group experiment; 16 professionals + 15 automated runs, 2 tiny tasks | moderate | |
| RQ 1 | Commercial vibe-coding platforms (Lovable, v0, Replit) generated complete web apps from a single zero-shot NL prompt without manual correction | [mota2026comparing, p. 2] | [V] | Exploratory static analysis; 9 apps (3 per platform) | weak | preprint |
| RQ 1 | Screenshot-to-HTML/CSS with MLLMs via direct, text-augmented and self-revision prompting, plus fine-tuned open models | [si2025benchmarking, pp. 4–5] | [V] | Benchmark; 484 (+80 HARD) pages; human eval 100 pages | moderate | |
| RQ 1 | Browser self-testing during generation correlates with accuracy across 16 models (r = 0.72), edit volume does not (r = 0.09) | [tran2026vibe, p. 7] | [V] | Benchmark; 50 test apps x 16 models, 1 generation each | weak | |
| RQ 1 | Single-agent tool-using harness (8 tools, ≤50 iterations, tsc+vite repair loop) rebuilds React+TS+Tailwind projects from app | [wu2026looks, p. 6] | [V] | Benchmark; 29 apps x 6 models = 174 single runs | moderate | |

| | | | | | | |
|---|---|---|---|---|---|---|
| | screenshots plus a page-relationship description | | | | | |
| RQ 1 | Interaction-to-Code: MLLM generates one HTML/CSS/JS file from before/after interaction screenshots; Direct, CoT, Mark, FAP prompts | [xiao2025benchmarking, pp. 2–5] | [V] | Benchmark; 127 pages / 374 interactions x 6 MLLMs x 3 prompts | moderate | |
| RQ 1 | Benchmark setting: agent gets only an execute-only binary plus usage docs, no internet, any implementation language, and must produce source plus a build script reproducing behaviour. | [yang2026programbench, p. 2; yang2026programbench, p. 4] | [V] | Benchmark; 200 programs x 9 models, 1 run each | weak | preprint: downgraded from moderate; analyst: design fact |
| RQ 1 | Pipeline: Grounding DINO + SAM component extraction, MLLM-built page transition graph, compiler/navigation feedback loop for declarative mobile UI code | [zhou2025declarui, pp. 3–6] | [V] | Technique + baselines; 50 apps (250 pages); user study 5 devs | moderate | |
| RQ 1 | PTG is reviewed and corrected by designers/developers (human in the loop) | [zhou2025declarui, p. 4] | [V] | Technique + baselines; 50 apps (250 pages); user study 5 devs | moderate | |
| RQ 1 | Multi-agent framework (MetaGPT + Qwen-Coder 480B) generated full multi-file Python apps (games, flashcards, finance dashboard, bookstore backend) from staged NL requirements; needed explicit modularity prompts to avoid monolithic scripts | [zhu2026understanding, p. 5] | [V] | Exploratory static analysis; 90 problems x 5 LLMs; ~20 MetaGPT apps | weak | |
| RQ 2 | Best agent averages 58.06% (Prog 58.19, VLM 57.92) but fully passes the programmatic suite on only 2.8% of apps (others <= 0.8%) | [bai2026recreationworld, p. 11] | [V] | Benchmark; 250 apps x 10 models, 5 platforms, rollouts not reported | weak | preprint: downgraded from moderate |

| | | | | | | |
|---|---|---|---|---|---|---|
| | | | | (probably 1) | | |
| RQ 2 | Static structure is reproduced more reliably than actions and computed outputs (structure best in 15/16 native comparisons; gaps 9.1-34.4 pts; Web 28.2-36.7 pts; four analysed models) | [bai2026recreationworld, p. 12; bai2026recreationworld, p. 16; bai2026recreationworld, p. 17] | **[V]** | Benchmark; 250 apps x 10 models, 5 platforms, rollouts not reported (probably 1) | weak | 4 of 10 models only; preprint: downgraded from moderate |
| RQ 2 | A recreation can score 100% visually yet only 55.1% programmatically | [bai2026recreationworld, p. 18] | **[V]** | Benchmark; 250 apps x 10 models, 5 platforms, rollouts not reported (probably 1) | weak | preprint |
| RQ 2 | Best model (GPT-5) scores Final 36.35 and IFS 24.39; IFS < 25 for all 28 models | [chen2025iwr, p. 8; chen2025iwr, p. 9] | **[V]** | Benchmark; 113 video tasks x 28 LVLMs, single run | weak | preprint: downgraded from moderate |
| RQ 2 | IFS < VFS for all 28 models (GPT-5 24.39 vs 64.25), but VFS averages only over reached checkpoints while IFS counts all actions, so the gap is partly metric-induced | [chen2025iwr, p. 7; chen2025iwr, p. 8] | **[V]** | Benchmark; 113 video tasks x 28 LVLMs, single run | weak | Bai and chen are only directionally consistent on looks-vs-behaves (see Contradictions); preprint |
| RQ 2 | Final Score falls from static (L1) to interactive tasks (GPT-5 61.85 vs 25.26 on L4); L2/L3 order varies by model; strata are about 9-12 tasks; category tables report Final Score only | [chen2025iwr, p. 16; chen2025iwr, p. 17] | **[V]** | Benchmark; 113 video tasks x 28 LVLMs, single run | weak | preprint |
| RQ 2 | Thinking variants add little to Final/IFS (e.g. Final 34.62 vs 34.00; 30.36 vs 30.31) and lower VFS in | [chen2025iwr, p. 8; chen2025iwr, p. 9] | **[V]** | Benchmark; 113 video tasks x 28 LVLMs, single | weak | preprint |

| | | | | | | |
|---|---|---|---|---|---|---|
| | all three pairs | | | run | | |
| RQ 2 | Best full-application rebuild from a live reference recovers 58.98% of atomic behaviours and 49.15% of cumulative workflows (12 apps); other models 21–42%. | [kim2026programdistill, p. 11] | [V] | Benchmark; 300 partial tasks x 9 models; 12 apps x 3 models full | weak | preprint; analyst: preprint, 36 single runs |
| RQ 2 | Partial-repair success falls as more dependent behaviours must be restored (best model 100% at depth 1 to 64% at depth 8; others to ≤32%), and the drop persists when context length is controlled. | [kim2026programdistill, p. 8; kim2026programdistill, p. 9; kim2026programdistill, p. 39] | [V] | Benchmark; 300 partial tasks x 9 models; 12 apps x 3 models full | weak | preprint; analyst: single runs, but consistent across 9 models |
| RQ 2 | Restoring UI plus logic is harder than logic alone (e.g. 92.9% vs 76.9% binary for the best model). | [kim2026programdistill, p. 38] | [V] | Benchmark; 300 partial tasks x 9 models; 12 apps x 3 models full | weak | preprint |
| RQ 2 | 59.2% of failed behaviours in full rebuilds were never observed in the reference; 27.9% had wrong state/result and 11.1% wrong observable form. | [kim2026programdistill, p. 12] | [V] | Benchmark; 300 partial tasks x 9 models; 12 apps x 3 models full | weak | preprint; analyst: classified by an LLM that is also an evaluated model |
| RQ 2 | Best setting (GPT-Engineer + Claude-Haiku 4.5) fully satisfies 53.75% of requirements (Test.Acc 69.41%) when rebuilding 46 small web apps from fine-grained requirements | [liu2026benchmarking, p. 6] | [V] | Benchmark; 46 web apps x 36 settings x 3 runs | moderate | |
| RQ 2 | Partial fulfilment is common: Soft Req.Acc exceeds strict Req.Acc by ~25–33 points across settings | [liu2026benchmarking, p. 25] | [V] | Benchmark; 46 web apps x 36 settings x 3 runs | moderate | |
| RQ | Best general-LLM agent | [lu2025webgen, p. 8] | [V] | Benchmark; | moderat | |

| | | | | | | |
|---|---|---|---|---|---|---|
| 2 | setting (Bolt.diy + DeepSeek-R1) reaches only 27.8% accuracy on 647 UI test cases for 101 websites built from scratch | | | 101 instructions / 647 tests x 16 settings, single run | e | |
| RQ 2 | Functional test cases are passed far less often than design-validation ones (e.g. 21.1% vs 44.3% for Bolt.diy + DeepSeek-R1; 17.1% vs 52.0% for Claude-3.5-Sonnet) | [lu2025webgen, p. 9] | [V] | Benchmark; 101 instructions / 647 tests x 16 settings, single run | moderate | |
| RQ 2 | Agentic runs passed more spec-derived baseline tests (mean TPR 94.58%) than solo (62.0%), collaborative (60.8%) and fully-automated (83.3%) workflows; significant after Holm-Bonferroni, but agentic used newer models (GPT-5/Claude Sonnet 4 vs GPT-3.5 Turbo) | [mock2026vibe, p. 25; mock2026vibe, p. 41; mock2026vibe, p. 42] | [V] | Static-group experiment; 16 professionals + 15 automated runs, 2 tiny tasks | weak | |
| RQ 2 | Collaborative vibe coding did not differ significantly from solo human coding in functional correctness (TPR p = 0.314); human results varied from 8.3% to 100% | [mock2026vibe, p. 25; mock2026vibe, p. 41] | [V] | Static-group experiment; 16 professionals + 15 automated runs, 2 tiny tasks | weak | |
| RQ 2 | Functional correctness diverged from static quality: Pass@1 human 1.0, few-shot 0.87, zero-shot 0.53, fine-tuned 0.47 (fine-tuned had the best quality metrics) | [molison2025llm, p. 8] | [V] | Comparative static analysis; GPT-4o vs APPS humans (N per group not reported) | weak | analyst: 15 solutions per approach |
| RQ 2 | In single-pass generation of 38 real Python applications, no LLM exceeds 45% functional test pass rate (averages 33.98% standard, 30.23% | [pan2026functional, p. 5] | [V] | Benchmark; 38 Python repos x 16 LLMs x 3 runs | weak | preprint; analyst: preprint, single study, ~11 tests/task |

| | | | | | | |
|---|---|---|---|---|---|---|
| | thinking). | | | | | |
| RQ 2 | Most failures (82.8%) occur after the code runs (requirement–implementation mismatch or runtime instability); only 17.2% are executability/dependency failures. | [pan2026functional, p. 7] | [V] | Benchmark; 38 Python repos x 16 LLMs x 3 runs | weak | preprint; analyst: coding procedure not reported |
| RQ 2 | Visual fidelity measured by Block-Match, Text, Position, Color (CIEDE2000) and CLIP on rendered screenshots | [si2025benchmarking, pp. 3–5] | [V] | Benchmark; 484 (+80 HARD) pages; human eval 100 pages | moderate | |
| RQ 2 | GPT-4o Direct best: Block 93.0, Text 98.2, Position 85.5, Color 84.1, CLIP 90.4 on 484 pages | [si2025benchmarking, p. 6] | [V] | Benchmark; 484 (+80 HARD) pages; human eval 100 pages | moderate | |
| RQ 2 | Authors state SOTA models fail to generate 30–40% of block elements on 80 HARD pages | [si2025benchmarking, p. 8] | [A] | Benchmark; 484 (+80 HARD) pages; human eval 100 pages | weak | |
| RQ 2 | HARD Block-Match (area-weighted, also penalises hallucinated blocks) ranges 56.6–75.1 across configurations (GPT-4o Direct 56.6) | [si2025benchmarking, p. 8] | [V] | Benchmark; 484 (+80 HARD) pages; human eval 100 pages | moderate | |
| RQ 2 | 49% of GPT-4V self-revision pages judged interchangeable with the original (κ = 0.32) | [si2025benchmarking, p. 6; si2025benchmarking, p. 16] | [V] | Benchmark; 484 (+80 HARD) pages; human eval 100 pages | weak | |
| RQ 2 | Best of 16 models (GPT-5.3-Codex) passes 61.77 ± 4.71% of browser workflows for web apps built from scratch from a text spec (50 test apps) | [tran2026vibe, p. 6] | [V] | Benchmark; 50 test apps x 16 models, 1 generation each | moderate | |
| RQ | Functional success drops | [tran2026vib | [V] | Benchmark; | weak | |

| | | | | | | |
|---|---|---|---|---|---|---|
| 2 | with difficulty (avg 44.3% easy, n = 23 apps, vs 6.0% hard, n = 8) and with external services (34.2% none, n = 37, vs 13.5% email + Stripe, n = 7); subgroups are tiny and tiers are defined from the models' own pass rates | e, p. 6] | | 50 test apps x 16 models, 1 generation each | | |
| RQ 2 | Most failed workflows (apps that started) are missing features (46.7%) or authorization problems (20.4%) | [tran2026vibe, p. 8] | [V] | Benchmark; 50 test apps x 16 models, 1 generation each | weak | |
| RQ 2 | Navigation fidelity via 701 state-isolated Playwright tests: NavPass 58% (GPT-5 Mini) to 92% (Opus 4.6); build 100% for all | [wu2026looks, p. 7] | [V] | Benchmark; 29 apps x 6 models = 174 single runs | moderate | |
| RQ 2 | Visual fidelity not saturated: point-wise means 1.60–2.99/5; Borda 0.13–0.91 | [wu2026looks, pp. 6–7] | [V] | Benchmark; 29 apps x 6 models = 174 single runs | moderate | |
| RQ 2 | Navigation failures: click has no effect 48.4%, target unreachable 32.8%, route error 10.9%, blank page 7.8% (n = 64, Wilson CIs) | [wu2026looks, p. 8] | [V] | Benchmark; 29 apps x 6 models = 174 single runs | weak | |
| RQ 2 | Interaction fidelity measured by interaction-part CLIP/SSIM/Text/Position, Implement Rate (webdriver), and human Usability Rate | [xiao2025benchmarking, p. 4] | [V] | Benchmark; 127 pages / 374 interactions x 6 MLLMs x 3 prompts | moderate | |
| RQ 2 | Best human Usability Rate 81% (GPT-4o Mark); Claude 66–77%; Qwen-3B 17–19% (number of evaluated items not reported, [xiao2025benchmarking, p. 6]) | [xiao2025benchmarking, p. 6] | [V] | Benchmark; 127 pages / 374 interactions x 6 MLLMs x 3 prompts | moderate | |
| RQ 2 | Interactive parts scored lower than full pages for | [xiao2025benchmarking, p. 6] | [V] | Benchmark; 127 pages / | moderate | |

| | | | | | | |
|---|---|---|---|---|---|---|
| | all six models | | | 374 interactions x 6 MLLMs x 3 prompts | | |
| RQ 2 | Ten failure types; missing element, no interaction, wrong function yield <10% usability | [xiao2025benchmarking, pp. 7–8] | [V] | Benchmark; 127 pages / 374 interactions x 6 MLLMs x 3 prompts | moderate | |
| RQ 2 | Behavioural properties can be recovered from a running Android app without source or specs (runtime observation plus manifest activity names and a static string whitelist): 912 of 985 LLM-generated properties judged valid (92.6%) | [xiong2026propgen, p. 4; xiong2026propgen, p. 5; xiong2026propgen, p. 8; xiong2026propgen, p. 9] | [V] | Tool + evaluation; 12 Android apps, single run | moderate | |
| RQ 2 | An LLM given only the app description produced 240 properties, none valid (hallucinated widgets); runtime evidence is required | [xiong2026propgen, p. 9] | [V] | Tool + evaluation; 12 Android apps, single run | moderate | |
| RQ 2 | Generated properties found 25 previously unknown functional bugs in 12 apps; prior tools rediscovered 3 of them, but themselves found 13 unknown bugs, 10 of which PropGen missed; authors call the approaches complementary (A) | [xiong2026propgen, p. 9; xiong2026propgen, p. 10] | [V] | Tool + evaluation; 12 Android apps, single run | moderate | |
| RQ 2 | No model fully rebuilds any of 200 programs; the best reaches ≥95% of tests on only 3% of tasks. | [yang2026programbench, p. 6] | [V] | Benchmark; 200 programs x 9 models, 1 run each | weak | preprint; analyst: preprint, one run per model-task |
| RQ 2 | Mean behavioural test pass rates range from 15.9% to 52.0% across | [yang2026programbench, p. 31] | [V] | Benchmark; 200 programs x 9 models, 1 | weak | preprint |

| | | | | | | |
|---|---|---|---|---|---|---|
| | models and fall with program size/complexity (e.g. 73.9% Easy to 24.7% Hard for Opus 4.6). | | | run each | | |
| RQ 2 | Best agent setting (SWE-agent + Claude 4 Sonnet) passes upstream human-written functional tests on 57.0% of 186 repository-level feature tasks; the 12 settings range 12.9–57.0% FuncPass | [zhao2025vibe, p. 7] | [V] | Benchmark; 186 tasks x 12 agent settings, single pass@1 | weak | preprint: downgraded from moderate |
| RQ 2 | Functional success drops as the reference patch spans more files (Claude 4 Sonnet: 66.9% single-file vs 30.3% for >2 files) | [zhao2025vibe, p. 14] | [V] | Benchmark; 186 tasks x 12 agent settings, single pass@1 | weak | preprint: downgraded from moderate |
| RQ 2 | Navigation fidelity via PTG Coverage Rate (manual comparison with DroidBot UTG): DeclarUI 96.8% vs Claude-3.5 43.3% vs GPT-4o 1.4% (React Native, 50 apps) | [zhou2025declarui, p. 7; zhou2025declarui, p. 9] | [V] | Technique + baselines; 50 apps (250 pages); user study 5 devs | moderate | |
| RQ 2 | Compilation success 98% (DeclarUI Claude-3.5) vs 74–76% for raw MLLMs; CLIP 0.88 vs 0.65–0.66; SSIM 0.68 vs 0.41–0.44 | [zhou2025declarui, p. 9] | [V] | Technique + baselines; 50 apps (250 pages); user study 5 devs | moderate | |
| RQ 2 | Without PTG + refinement, PCR falls to 23.1% | [zhou2025declarui, p. 10] | [V] | Technique + baselines; 50 apps (250 pages); user study 5 devs | weak | |
| RQ 3 | Agentic PRs from low-experience users were accepted less often (31% lower; e.g. docs 93.06% vs 75.39%) and drew 4.52x more review comments than those from high-experience users | [asdaque2026novice, p. 3] | [V] | MSR mining; 22,953 agentic PRs / 1,719 users | moderate | |

| | | | | | | |
|---|---|---|---|---|---|---|
| RQ 3 | Low-experience users' agentic PRs were larger (2.15x commits, 1.47x files per PR) and took 5.16x longer from creation to merge | [asdaque2026novice, p. 2; asdaque2026novice, p. 3] | [V] | MSR mining; 22,953 agentic PRs / 1,719 users | moderate | |
| RQ 3 | Authors attribute friction to AI code that ignores build/CI constraints and lacks system context (from 15 inspected PRs) | [asdaque2026novice, p. 3; asdaque2026novice, p. 4] | [A] | MSR mining; 22,953 agentic PRs / 1,719 users | weak | |
| RQ 3 | Recreations are much smaller and more concentrated than references: 89.4% smaller (median LOC ratio 16.9%), 92.3% fewer files, 83.8% larger largest-file share (four models analysed: Claude Opus 5, GPT-6 Astra, GLM-5.3, Qwen3.8-Max-0902; desktop and Android only; coverage varies across model-platform cells) | [bai2026recreationworld, p. 12; bai2026recreationworld, p. 15; bai2026recreationworld, p. 16] | [V] | Benchmark; 250 apps x 10 models, 5 platforms, rollouts not reported (probably 1) | weak | Covers 4 of 10 models; code-size figures from desktop + Android only; preprint: downgraded from moderate |
| RQ 3 | More generated code does not mean higher fidelity among the four analysed models (highest-LOC model is not the best; best model has lowest LOC on 4/5 platforms; coverage varies across cells) | [bai2026recreationworld, p. 12; bai2026recreationworld, p. 15; bai2026recreationworld, p. 31] | [V] | Benchmark; 250 apps x 10 models, 5 platforms, rollouts not reported (probably 1) | weak | 4 of 10 models only; preprint |
| RQ 3 | Agents frequently switch UI framework (e.g. 75% of Electron/Tauri/Wails references rebuilt natively; four analysed models, Web excluded); authors frame this as architectural substitution, not quality | [bai2026recreationworld, p. 15; bai2026recreationworld, p. 16] | [V] | Benchmark; 250 apps x 10 models, 5 platforms, rollouts not reported (probably 1) | weak | 4 of 10 models only; preprint |
| RQ 3 | Developers evolving AI-co-developed code showed no significant difference in completion | [borg2026echoes, p. 33] | [V] | Preregistered 2-phase experiment; RCT n=75 | moderate | |

| | | | | | | |
|---|---|---|---|---|---|---|
| | time (median 136 vs 173 min, p=0.56, Cliff's δ=−0.079) | | | (Phase 1 n=76) | | |
| RQ 3 | No significant difference in CodeHealth after evolution (8.49 vs 8.44, p=0.24, d=0.28); Bayesian +0.10 CH for code from habitual AI users (P=98%) | [borg2026echoes, p. 38; borg2026echoes, p. 39] | [V] | Preregistered 2-phase experiment; RCT n=75 (Phase 1 n=76) | moderate | |
| RQ 3 | No significant difference in test coverage (median 0.70 both, p=0.49) or perceived productivity (d=−0.21, p=0.37) | [borg2026echoes, p. 35; borg2026echoes, p. 42] | [V] | Preregistered 2-phase experiment; RCT n=75 (Phase 1 n=76) | moderate | |
| RQ 3 | Higher Task 1 Java skill (skill 3 vs 1): +0.16 CH in Task 2, P(Δ>0) > 99%, CrI [+0.07, +0.26] | [borg2026echoes, p. 40] | [V] | Preregistered 2-phase experiment; RCT n=75 (Phase 1 n=76) | moderate | |
| RQ 3 | Authors conclude the Task 1 developer's Java proficiency mattered more for Task 2 outcomes than AI use (different contrast from the +0.10 habitual-AI subgroup, [borg2026echoes, p. 39]) | [borg2026echoes, p. 55] | [A] | Preregistered 2-phase experiment; RCT n=75 (Phase 1 n=76) | moderate | |
| RQ 3 | AI-assisted Task 1 code (observational) introduced more Code Duplication (23% vs 11%) and Complex Conditionals (13% vs 5%), fewer PMD complexity violations, and much larger code volume in outliers (up to 10,788 lines) | [borg2026echoes, p. 30; borg2026echoes, p. 46; borg2026echoes, p. 47] | [V] | Preregistered 2-phase experiment; RCT n=75 (Phase 1 n=76) | weak | |
| RQ 3 | AI assistance cut Task 1 median completion time by 30.7% (p=0.0029, δ=−0.42; observational) | [borg2026echoes, p. 43] | [V] | Preregistered 2-phase experiment; RCT n=75 (Phase 1 n=76) | moderate | |

| | | | | | | |
|---|---|---|---|---|---|---|
| RQ 3 | Across 104 sources, the debt types most often linked to LLM-assisted code are code (14 formal / 56 grey), design (2 / 30) and documentation (0 / 17) debt | [ehsani2026faster, p. 16] | [V] | Multivocal review; 31 formal + 73 grey sources | moderate | analyst: mention counts, mostly grey |
| RQ 3 | Six LLM-specific debts are proposed: governance, prompt, fast-integration, data, ethical, provenance. Governance (19) and fast-integration (13) appear only in grey sources | [ehsani2026faster, pp. 16–17] | [V] | Multivocal review; 31 formal + 73 grey sources | weak | analyst: practitioner commentary, no formal support for two of six |
| RQ 3 | DO NOT COUNT SEPARATELY. Secondary claim reported from [he2026speed] (ref [90]): the review restates its +30% warnings / +41% complexity without the primary study's estimator-sensitivity and codebase-size caveats [he2026speed, p. 8; he2026speed, p. 16]. Cite the primary study, not the review, for this effect | [ehsani2026faster, p. 18] | [A] | Multivocal review; 31 formal + 73 grey sources | not counted | Not counted as independent evidence: same effect as [he2026speed, p. 7] |
| RQ 3 | Practitioners mostly perceive vibe-coded output as "fast but flawed" (78/114, 68%) or fragile/error-prone (22/114, 19%); only 3 units (3%) call it high quality | [fawzy2026vibe, p. 8] | [V] | Grey-literature review; 101 sources, 518 coded units | weak | |
| RQ 3 | Authors conclude vibe coding trades reliability and maintainability for speed; based on perceptions, no code measured | [fawzy2026vibe, p. 1; fawzy2026vibe, p. 9] | [A] | Grey-literature review; 101 sources, 518 coded units | weak | |
| RQ 3 | Cursor adoption raises SonarQube static-analysis warnings by +30.26% (p<0.001) | [he2026speed, p. 7] | [V] | Staggered DiD + GMM; 806 Cursor repos vs | moderate | Estimator-sensitive: negative and n.s. |

| | | | | | | |
|---|---|---|---|---|---|---|
| | relative to matched controls (Borusyak DiD) | | | matched controls | | under Callaway–Sant'Anna [he2026speed, p. 16]; GMM attributes the rise to more code [he2026speed, p. 8] |
| RQ 3 | Cursor adoption raises total cognitive complexity by +41.64% (p<0.001), persisting 6 months; still significant (0.086, p<0.01) after controlling for codebase size in GMM | [he2026speed, p. 7; he2026speed, p. 8] | **[V]** | Staggered DiD + GMM; 806 Cursor repos vs matched controls | moderate | Estimator-sensitive: −3.80% n.s. under Callaway–Sant'Anna [he2026speed, p. 16]; text also cites 25%/25.1% (inconsistent; table value used) |
| RQ 3 | Duplicated line density shows no significant change after adoption (+7.03%, n.s.) | [he2026speed, p. 7] | **[V]** | Staggered DiD + GMM; 806 Cursor repos vs matched controls | moderate | |
| RQ 3 | With codebase size controlled, Cursor has no significant direct effect on warnings (GMM); the warning rise runs mainly through more code | [he2026speed, p. 8] | **[V]** | Staggered DiD + GMM; 806 Cursor repos vs matched controls | moderate | |
| RQ 3 | Higher complexity and warnings predict lower next-month lines added (−64.5% / −50.3% per 100% increase) | [he2026speed, p. 8] | **[V]** | Staggered DiD + GMM; 806 Cursor repos vs matched controls | moderate | |
| RQ 3 | Velocity gain is transient: lines added +281.3% month 1, +48.4% month | [he2026speed, p. 8] | **[V]** | Staggered DiD + GMM; 806 Cursor | moderate | |

| | | | | | | |
|---|---|---|---|---|---|---|
| | 2, then n.s. | | | repos vs matched controls | | |
| RQ 3 | Agentic code had higher McCabe complexity (mean 11.67 vs about 6-6.6 for other workflows) and lower baseline branch coverage (65.3% vs 79-84% for human workflows), adding helpers, logging and validation | [mock2026vibe, p. 24; mock2026vibe, p. 27; mock2026vibe, p. 41; mock2026vibe, p. 42] | **[V]** | Static-group experiment; 16 professionals + 15 automated runs, 2 tiny tasks | weak | |
| RQ 3 | Authors argue agentic solutions are architecturally more complex and "potentially more difficult to maintain"; maintainability not directly measured | [mock2026vibe, p. 35] | **[A]** | Static-group experiment; 16 professionals + 15 automated runs, 2 tiny tasks | weak | |
| RQ 3 | GPT-4o solutions (zero-shot, few-shot, fine-tuned) had far lower odds of SonarQube-flagged bugs than APPS human solutions at all difficulty levels (OR 0.00–0.019, all $p<0.001$; the 0.00 implies a zero cell, so the minimum is not a real estimate) | [molison2025llm, p. 8] | **[V]** | Comparative static analysis; GPT-4o vs APPS humans (N per group not reported) | weak | SonarQube rule-based 'bugs', not functional failures; analyst: single model and study; N per group not reported; few-shot prompt contains baseline solutions |
| RQ 3 | Code-smell (maintainability) differences were mixed: zero-shot had 3.57× the odds of a smell vs human on introductory tasks, while fine-tuned had fewer smells; several comparisons were not | [molison2025llm, p. 8] | **[V]** | Comparative static analysis; GPT-4o vs APPS humans (N per group not reported) | weak | |

| | | | | | | |
|---|---|---|---|---|---|---|
| | significant | | | | | |
| RQ 3 | SonarQube-estimated fix effort was lower for LLM code on introductory/interview tasks but not on competition tasks (human 6.95 min vs few-shot 7.33, fine-tuned 7.60) | [molison2025llm, p. 7] | [V] | Comparative static analysis; GPT-4o vs APPS humans (N per group not reported) | weak | |
| RQ 3 | On competition-level problems LLM outputs carried more critical-severity issues than human code | [molison2025llm, p. 6] | [A] | Comparative static analysis; GPT-4o vs APPS humans (N per group not reported) | weak | |
| RQ 3 | Per KLOC, Lovable apps had 22.83 SonarQube issues vs 9.21 (v0) and 9.56 (Replit), and 220.77 vs about 34 min/KLOC remediation effort; mostly MINOR code smells driven by deprecated-API rule S1874 (204 occurrences) | [mota2026comparing, p. 3; mota2026comparing, p. 5] | [V] | Exploratory static analysis; 9 apps (3 per platform) | weak | preprint |
| RQ 3 | Replit apps were the largest (48,411 NCLOC for 3 apps) with about 60% duplicated lines and 46% MAJOR-severity issues | [mota2026comparing, p. 4; mota2026comparing, p. 5; mota2026comparing, p. 6] | [V] | Exploratory static analysis; 9 apps (3 per platform) | weak | preprint |
| RQ 3 | Authors suggest vibe-coding tools may systematically use outdated APIs because LLMs over-represent historical API patterns | [mota2026comparing, p. 5; mota2026comparing, p. 6] | [A] | Exploratory static analysis; 9 apps (3 per platform) | weak | preprint |
| RQ 3 | Reference-normalised non-functional scores: resource usage is the weakest dimension (31.25% standard / 28.92% thinking), followed by maintainability | [pan2026functional, p. 6] | [V] | Benchmark; 38 Python repos x 16 LLMs x 3 runs | weak | M = 0.5 means parity with the reference MI, so ~33% |

| | | | | | | |
|---|---|---|---|---|---|---|
| | (MI lower bound; 33.45% / 31.38%); security averages 87.40% / 85.32% | | | | | implies lower MI than the reference; partial repositories can score well on static dimensions [I]; preprint; analyst: single MI proxy; parity with reference scores only 0.5 |
| RQ 3 | On GPT-5.2, self-repair, environment repair and planning each lowered the aggregate non-functional score (46.1–49.9% vs 57.1% baseline) without raising functional correctness. | [pan2026functional, p. 8] | [V] | Benchmark; 38 Python repos x 16 LLMs x 3 runs | weak | preprint; analyst: one model, one-shot strategies |
| RQ 3 | No code quality or maintainability measure; only rendered visual similarity | [si2025benchmarking, pp. 3–4] | [V] | Benchmark; 484 (+80 HARD) pages; human eval 100 pages | moderate | |
| RQ 3 | Code readability, security and visual design quality of the generated apps are not measured | [tran2026vibe, p. 9] | [A] | Benchmark; 50 test apps x 16 models, 1 generation each | strong | |
| RQ 3 | Maintainability proxies (LoC, LoC/file, reuse, dead components, colour consistency): GPT-5 668 LoC, 7.8% dead vs Opus 1,441 LoC, 19.9% dead | [wu2026looks, p. 4; wu2026looks, p. 7] | [V] | Benchmark; 29 apps x 6 models = 174 single runs | moderate | |
| RQ 3 | Visual-fidelity leader is not maintainability leader; authors call them | [wu2026looks, pp. 6–8] | [A] | Benchmark; 29 apps x 6 models = 174 | weak | |

| | | | | | | |
|---|---|---|---|---|---|---|
| | uncorrelated (no coefficient computed) | | | single runs | | |
| RQ 3 | No code quality or maintainability measured; code compared to original only via BLEU 0.12–0.14 for contamination | [xiao2025benchmarking, p. 5] | [V] | Benchmark; 127 pages / 374 interactions x 6 MLLMs x 3 prompts | moderate | |
| RQ 3 | Rebuilt codebases passing ≥75% of tests are shorter (median 1,173 vs 3,068 lines), use fewer files (3 vs 15), are shallower, and have fewer, longer functions than the human originals. | [yang2026programbench, p. 9] | [V] | Benchmark; 200 programs x 9 models, 1 run each | weak | preprint; analyst: descriptive, selected subset, language-confounded |
| RQ 3 | 79.3% of the functionally correct solutions of the best setting fail the security tests; SecPass is 3.8–12.9% across settings | [zhao2025vibe, p. 6] | [V] | Benchmark; 186 tasks x 12 agent settings, single pass@1 | weak | preprint: downgraded from moderate |
| RQ 3 | Functional and security skill diverge: the functionally weakest model (Gemini 2.5 Pro) has the highest secure ratio on commonly solved tasks (30.4% vs 18.3% for Claude 4 Sonnet); exception: Gemini 3 Pro best on SWE-agent (36.4%); the commonly solved subset is unreported and at most ~24–37 tasks per framework ([zhao2025vibe, p. 7]) | [zhao2025vibe, p. 6] | [V] | Benchmark; 186 tasks x 12 agent settings, single pass@1 | weak | Rests on the unreported solved-by-all subset; treat as weak; preprint |
| RQ 3 | Simple security prompting (self-selection / oracle CWE) raises SecPass by 2.7–3.3 pp but lowers FuncPass by 1.6–7.0 pp (single setting, single run) | [zhao2025vibe, p. 8] | [V] | Benchmark; 186 tasks x 12 agent settings, single pass@1 | weak | preprint |
| RQ 3 | Developer ratings (5 devs × 12 apps): readability | [zhou2025declarui, p. 10] | [V] | Technique + baselines; 50 | weak | |

| | | | | | | |
|---|---|---|---|---|---|---|
| | 4.62 vs 3.75, maintainability 4.55 vs 3.37, Mann-Whitney p < 1e-11 | | | apps (250 pages); user study 5 devs | | |
| RQ 3 | Code quality/maintainability measured only by Likert ratings, code availability and modification time; no static metrics | [zhou2025declarui, p. 7; zhou2025declarui, p. 9] | [V] | Technique + baselines; 50 apps (250 pages); user study 5 devs | moderate | |
| RQ 3 | In 90 algorithmic tasks, larger LLMs produced more Long Method smells than humans (Qwen-480B 11, Gemini 5, human 1, zero-shot); humans alone produced Temporal Field (4) | [zhu2026understanding, p. 7] | [V] | Exploratory static analysis; 90 problems x 5 LLMs; ~20 MetaGPT apps | weak | |
| RQ 3 | Few-shot prompting with coding standards did not reduce Long Method smells (Qwen-480B 11→13, Gemini 5→8) | [zhu2026understanding, p. 6; zhu2026understanding, p. 7] | [V] | Exploratory static analysis; 90 problems x 5 LLMs; ~20 MetaGPT apps | weak | |
| RQ 3 | MetaGPT-generated apps mainly show Too Many Branches, High Response for a Class, Potential Improper API Usage, Scattered Functionality and Unstable Dependency smells | [zhu2026understanding, p. 7; zhu2026understanding, p. 8] | [V] | Exploratory static analysis; 90 problems x 5 LLMs; ~20 MetaGPT apps | weak | |
| RQ 3 | In MetaGPT apps, architectural smell counts correlate with total LoC ($\rho$=0.94, $p<0.001$); more detailed requirement stages had no significant effect ($p>0.8$) | [zhu2026understanding, p. 8; zhu2026understanding, p. 9] | [V] | Exploratory static analysis; 90 problems x 5 LLMs; ~20 MetaGPT apps | weak | |
| RQ 4 | Experience = total commits / GitHub account age; quartile split Q1–2 vs Q3–4; group differences reported as ratios of means without effect sizes or CIs | [asdaque2026novice, p. 2; asdaque2026novice, p. 3] | [V] | MSR mining; 22,953 agentic PRs / 1,719 users | moderate | |

| | | | | | | |
|---|---|---|---|---|---|---|
| RQ 4 | Authors state the commit-based experience proxy "conflates activity with skill" | [asdaque2026novice, p. 4] | **[A]** | MSR mining; 22,953 agentic PRs / 1,719 users | moderate | |
| RQ 4 | Unit of analysis for the tests is unclear ("on mean values of each metric across PR categories", [asdaque2026novice, p. 2]); if PR-level, observations are clustered by user (based on 22,953 PRs / 1,719 users, [asdaque2026novice, p. 2]) | [asdaque2026novice, p. 2] | **[I]** | MSR mining; 22,953 agentic PRs / 1,719 users | weak | |
| RQ 4 | Fidelity measured by reference-validated, human-reviewed hidden tests in a programmatic and a VLM channel; 94.2% of cases check interaction outcomes | [bai2026recreationworld, p. 9] | **[V]** | Benchmark; 250 apps x 10 models, 5 platforms, rollouts not reported (probably 1) | weak | preprint: downgraded from moderate |
| RQ 4 | No CIs or repeated runs for main scores (probably one rollout per task); judge accuracy and reviewer agreement not reported; model and scaffold confounded. Our appraisal of absences, based on Tables 3, 8, 11 and the evaluation text | [bai2026recreationworld, p. 8; bai2026recreationworld, p. 9; bai2026recreationworld, p. 11; bai2026recreationworld, p. 28; bai2026recreationworld, p. 30] | **[I]** | Benchmark; 250 apps x 10 models, 5 platforms, rollouts not reported (probably 1) | weak | preprint: downgraded from moderate |
| RQ 4 | Agents attempt evaluation shortcuts (four analysed models) | [bai2026recreationworld, p. 13] | **[V]** | Benchmark; 250 apps x 10 models, 5 platforms, rollouts not reported (probably 1) | weak | preprint |
| RQ 4 | A transformed replay evaded the scrape detector and scored 54.82% Prog ([bai2026recreationworld, p. 47]); detected replay is capped at 0.10 | [bai2026recreationworld, p. 38; bai2026recreationworld, p. 46; bai2026recreationworld, p. 47] | **[V]** | Benchmark; 250 apps x 10 models, 5 platforms, rollouts not reported (probably 1) | weak | preprint |

| | | | | | | |
|---|---|---|---|---|---|---|
| | ([bai2026recreationworld, p. 38]) | | | | | |
| RQ 4 | Maintainability operationalised as hand-off evolution by a second developer (time) plus customised CodeScene CodeHealth, JaCoCo coverage and SPACE-based perceived productivity | [borg2026echoes, p. 13] | [V] | Preregistered 2-phase experiment; RCT n=75 (Phase 1 n=76) | moderate | |
| RQ 4 | RCT underpowered (75 of 128 planned; 80% power only for d≈0.66); authors read null results as evidence against large effects only | [borg2026echoes, p. 19; borg2026echoes, p. 61] | [V] | Preregistered 2-phase experiment; RCT n=75 (Phase 1 n=76) | moderate | |
| RQ 4 | Functional fidelity measured by agent replay of a ground-truth action sequence plus MLLM-judged assertions; visual fidelity by OCR + DINO + MLLM rubric | [chen2025iwr, p. 7; chen2025iwr, p. 20] | [V] | Benchmark; 113 video tasks x 28 LVLMs, single run | weak | preprint |
| RQ 4 | Metric weights tuned on 60 human-rated instances (5 raters); Spearman rho 0.57 (VFS) and 0.65 (Final); no held-out check is described (I) | [chen2025iwr, p. 17] | [V] | Benchmark; 113 video tasks x 28 LVLMs, single run | weak | preprint |
| RQ 4 | Agent-judge errors in 3 of 100 manually observed runs | [chen2025iwr, p. 9] | [V] | Benchmark; 113 video tasks x 28 LVLMs, single run | weak | preprint |
| RQ 4 | VFS and IFS use different denominators (reached checkpoints vs all actions), so their gap is not a like-for-like comparison | [chen2025iwr, p. 7] | [I] | Benchmark; 113 video tasks x 28 LVLMs, single run | weak | preprint |
| RQ 4 | No repeated runs, confidence intervals or significance tests for model comparisons were found (our check of the | [chen2025iwr, p. 8; chen2025iwr, p. 9] | [I] | Benchmark; 113 video tasks x 28 LVLMs, single run | weak | preprint |

| | | | | | | |
|---|---|---|---|---|---|---|
| | results and tables) | | | | | |
| RQ 4 | No standardised benchmarks or datasets exist to evaluate TD or maintainability of LLM-generated code; benchmarks focus on binary correctness | [ehsani2026faster, pp. 23–24] | [A] | Multivocal review; 31 formal + 73 grey sources | moderate | analyst: consistent across both literatures; review search to Jan 2026 |
| RQ 4 | SonarQube and general linters are the main measurement tools, but they are not designed for LLM-specific debt; candidate LLM-specific metrics (PSC, AUP/DUR, data-smell strength/density) are early and not standardised | [ehsani2026faster, p. 22; ehsani2026faster, pp. 24–25; ehsani2026faster, p. 28] | [A] | Multivocal review; 31 formal + 73 grey sources | moderate | |
| RQ 4 | Commercial LLM-aware quality tools are rarely independently evaluated | [ehsani2026faster, p. 28] | [A] | Multivocal review; 31 formal + 73 grey sources | weak | |
| RQ 4 | Evidence is counts of coded quotes from 101 self-selected grey-literature sources, coded mainly by one author, no agreement statistic reported | [fawzy2026vibe, p. 5; fawzy2026vibe, p. 10] | [V] | Grey-literature review; 101 sources, 518 coded units | moderate | |
| RQ 4 | Quality results are estimator-sensitive: Callaway–Sant'Anna gives negative, n.s. estimates for warnings (−10.49%) and complexity (−3.80%) | [he2026speed, p. 16] | [V] | Staggered DiD + GMM; 806 Cursor repos vs matched controls | moderate | |
| RQ 4 | Project-level quality measured with SonarQube warnings, cognitive complexity and duplicated-line density; treatment proxied by committed .cursorrules files (intent-to-treat) | [he2026speed, p. 5; he2026speed, p. 7] | [V] | Staggered DiD + GMM; 806 Cursor repos vs matched controls | moderate | |
| RQ | Fidelity oracle: | [kim2026programdistill, | [V] | Benchmark; | weak | preprint: |

| | | | | | | |
|---|---|---|---|---|---|---|
| 4 | deterministic browser-trace replay with expected signals, admitted only if replayable on the intact app, failing after masking, and passing with the gold patch. | p. 4; kim2026programdistill, p. 5; kim2026programdistill, p. 6] | | 300 partial tasks x 9 models; 12 apps x 3 models full | | downgraded from moderate; analyst: design fact |
| RQ 4 | Agents' own validation (syntax checks, request success, partial text checks) missed behavioural mismatches with the reference. | [kim2026programdistill, p. 12] | **[A]** | Benchmark; 300 partial tasks x 9 models; 12 apps x 3 models full | weak | preprint |
| RQ 4 | Tasks are built and validated without human intervention by one LLM (GPT-5.6 Sol), which is also evaluated; authors note quality depends on the construction model. | [kim2026programdistill, p. 1; kim2026programdistill, p. 2; kim2026programdistill, p. 15] | **[A]** | Benchmark; 300 partial tasks x 9 models; 12 apps x 3 models full | weak | preprint |
| RQ 4 | Functional oracle = Gherkin/Behave tests with Selenium, derived from the original app's code via LLM agents plus human review, passing 100% on the original projects; no LLM judge | [liu2026benchmarking, p. 5] | **[V]** | Benchmark; 46 web apps x 36 settings x 3 runs | moderate | |
| RQ 4 | Test-quality audit by 10 experts on 100 sampled tests gives Fleiss' kappa 0.81; a manual-only annotation pilot gave kappa 0.23 | [liu2026benchmarking, p. 14] | **[V]** | Benchmark; 46 web apps x 36 settings x 3 runs | moderate | |
| RQ 4 | Human vs automatic evaluation agreement Cohen's kappa 0.74 (human–human 0.86), but sample and unit not described | [liu2026benchmarking, p. 22] | **[V]** | Benchmark; 46 web apps x 36 settings x 3 runs | weak | |
| RQ 4 | Functional fidelity is judged by a WebVoyager UI agent (Qwen2.5-VL-32B) on GPT-4o-drafted, human-refined atomic test | [lu2025webgen, p. 6] | **[V]** | Benchmark; 101 instructions / 647 tests x 16 settings, single run | moderate | |

| | | | | | | |
|---|---|---|---|---|---|---|
| | cases, with YES/PARTIAL/NO and half credit for PARTIAL | | | | | |
| RQ 4 | UI-agent labels match adjudicated human labels on 86.1–94.4% of test cases (3 Bolt.diy settings); inter-human agreement and chance-corrected agreement not reported | [lu2025webgen, p. 9] | [V] | Benchmark; 101 instructions / 647 tests x 16 settings, single run | moderate | |
| RQ 4 | No error bars or significance tests; single run per setting | [lu2025webgen, p. 40] | [V] | Benchmark; 101 instructions / 647 tests x 16 settings, single run | strong | |
| RQ 4 | GPT-4o appearance scores give the same ranking as o3, Claude-3.5-Sonnet and human grading on 3 settings | [lu2025webgen, p. 23] | [V] | Benchmark; 101 instructions / 647 tests x 16 settings, single run | weak | |
| RQ 4 | Evaluation used spec-derived category-partition oracle tests, MCC, coverage, LCS/LED similarity, iterations and time, with Kruskal-Wallis, one-tailed Mann-Whitney, Cliff's delta and Holm-Bonferroni | [mock2026vibe, p. 10; mock2026vibe, p. 11; mock2026vibe, p. 14] | [V] | Static-group experiment; 16 professionals + 15 automated runs, 2 tiny tasks | moderate | |
| RQ 4 | Sample: 9 solo vs 7 collaborative professionals, 3 fully-automated and 12 agentic runs; 2 small Python tasks; time self-reported for humans but measured for automated runs | [mock2026vibe, p. 12; mock2026vibe, p. 13; mock2026vibe, p. 16; mock2026vibe, p. 42] | [V] | Static-group experiment; 16 professionals + 15 automated runs, 2 tiny tasks | moderate | |
| RQ 4 | Comparison used SonarQube default profile, odds ratios, and Mann-Whitney U with Cliff's delta, without CIs or | [molison2025llm, p. 5; molison2025llm, p. 8] | [V] | Comparative static analysis; GPT-4o vs APPS | moderate | |

| | | | | | | |
|---|---|---|---|---|---|---|
| | multiple-comparison correction | | | humans (N per group not reported) | | |
| RQ 4 | The few-shot prompt included the human reference solutions of the same question, so few-shot vs human is not an independent comparison | [molison2025llm, p. 4] | [I] | Comparative static analysis; GPT-4o vs APPS humans (N per group not reported) | moderate | analyst: directly from method description |
| RQ 4 | Only 3 projects per tool (free-tier limits); one Lovable project differed strongly (59 vs 164-167 issues), code-smell SD 62.1 vs 7.0 and 20.3 | [mota2026comparing, p. 3; mota2026comparing, p. 6] | [V] | Exploratory static analysis; 9 apps (3 per platform) | weak | preprint |
| RQ 4 | Quality measured only by SonarQube static analysis, which the authors note is an auxiliary indicator; no functional testing of the generated apps | [mota2026comparing, p. 2; mota2026comparing, p. 6] | [V] | Exploratory static analysis; 9 apps (3 per platform) | weak | preprint: downgraded from moderate |
| RQ 4 | Tests are kept only if they pass on the pinned reference repository; functional suites caught 76/76 injected defects. | [pan2026functional, p. 2; pan2026functional, p. 5] | [V] | Benchmark; 38 Python repos x 16 LLMs x 3 runs | weak | preprint; analyst: only 2 defects/task, method not described |
| RQ 4 | Functional scores had zero variance across 5 reruns; the NF ranking is robust to weighting choices (Spearman ≥0.90) except rank-based aggregation (0.64). | [pan2026functional, p. 7; pan2026functional, p. 10] | [V] | Benchmark; 38 Python repos x 16 LLMs x 3 runs | weak | preprint |
| RQ 4 | Authors cannot rule out pretraining exposure to the public reference repositories. | [pan2026functional, p. 2] | [A] | Benchmark; 38 Python repos x 16 LLMs x 3 runs | weak | preprint |
| RQ 4 | Giving the reference's module/package surface | [pan2026functional, p. 4] | [V] | Benchmark; 38 Python | weak | preprint; analyst: |

| | | | | | | |
|---|---|---|---|---|---|---|
| | raised one task's functional score from 0.17 to 0.67, so interface hints strongly affect measured fidelity. | | | repos x 16 LLMs x 3 runs | | single example |
| RQ 4 | Metrics predict human Win/Lose with 79.9% accuracy (logistic regression, 50-page test half); Text coefficient negative | [si2025benchmarking, p. 7] | [V] | Benchmark; 484 (+80 HARD) pages; human eval 100 pages | moderate | |
| RQ 4 | Simulated vs human win rates: Pearson r = 0.975, Kendall τ = 0.931 over 10 systems; not out-of-sample (model fit on 50 of the same 100 annotated pages, applied to all 484 incl. them) | [si2025benchmarking, p. 7; si2025benchmarking, p. 17] | [V] | Benchmark; 484 (+80 HARD) pages; human eval 100 pages | weak | Not out-of-sample |
| RQ 4 | Human agreement low-moderate: Fleiss' κ 0.46 / 0.32 / 0.26 | [si2025benchmarking, p. 16] | [V] | Benchmark; 484 (+80 HARD) pages; human eval 100 pages | moderate | |
| RQ 4 | No significance tests or variance across runs for model comparisons | [si2025benchmarking, pp. 5–6] | [I] | Benchmark; 484 (+80 HARD) pages; human eval 100 pages | moderate | |
| RQ 4 | Functional fidelity is scored by an LLM browser agent (Claude Sonnet 4.5) running pre-authored NL workflows; workflow passes at ≥90% substeps | [tran2026vibe, p. 4] | [V] | Benchmark; 50 test apps x 16 models, 1 generation each | moderate | |
| RQ 4 | Human–human substep agreement is 88.6–93.6%; judge–human agreement ranges 36.1% (GPT-5.2) to 86.4% (Claude Sonnet 4.5) on 18 apps / 1,401 substeps; raw agreement only, no kappa | [tran2026vibe, p. 9] | [V] | Benchmark; 50 test apps x 16 models, 1 generation each | moderate | |

| | | | | | | |
|---|---|---|---|---|---|---|
| RQ 4 | Generation variance dominates evaluator variance (73.5–92.5% of variance) on a 6-app x 3-model x 5-run panel | [tran2026vibe, p. 8] | [V] | Benchmark; 50 test apps x 16 models, 1 generation each | weak | |
| RQ 4 | Anchor-referenced list-wise VLM judge vs human ranking on 55 scenarios: Kendall τ = 0.716, top-1 83.6%; cross-judge τ = 0.867 | [wu2026looks, p. 13] | [V] | Benchmark; 29 apps x 6 models = 174 single runs | moderate | |
| RQ 4 | Pair-wise VLM judging shows 23.3% position bias; point-wise shows ties-in-the-middle (3-scenario smoke test) | [wu2026looks, pp. 12–13] | [V] | Benchmark; 29 apps x 6 models = 174 single runs | weak | |
| RQ 4 | Single trial per app–model cell; no significance tests between models | [wu2026looks, p. 6; wu2026looks, p. 9] | [V] | Benchmark; 29 apps x 6 models = 174 single runs | moderate | |
| RQ 4 | Build success saturates when a verify-repair loop is present and stops discriminating models | [wu2026looks, p. 12] | [A] | Benchmark; 29 apps x 6 models = 174 single runs | moderate | |
| RQ 4 | Automatic metrics declared "consistent" with human evaluation; no correlation or inter-annotator agreement reported | [xiao2025benchmarking, p. 6] | [V] | Benchmark; 127 pages / 374 interactions x 6 MLLMs x 3 prompts | weak | |
| RQ 4 | Widget Similarity/Match (GUIPilot) unreliable for web pages (WS 0.02–0.05) | [xiao2025benchmarking, pp. 5–6] | [V] | Benchmark; 127 pages / 374 interactions x 6 MLLMs x 3 prompts | moderate | |
| RQ 4 | Claimed FAP gain on all models contradicted by Table VI for Qwen-3B/7B | [xiao2025benchmarking, pp. 8–9] | [V] | Benchmark; 127 pages / 374 interactions x 6 MLLMs x 3 prompts | moderate | |
| RQ 4 | Saliency, modality sub-studies on 10 pages; user study 2 vs 2 participants | [xiao2025benchmarking, pp. 9–11] | [V] | Benchmark; 127 pages / 374 interactions x | weak | |

| | | | | | | |
|---|---|---|---|---|---|---|
| | | | | 6 MLLMs x 3 prompts | | |
| RQ 4 | Validity judged by 2 independent non-author annotators (kappa 0.91 / 0.82 / 0.81); no ground-truth spec exists for apps | [xiong2026propgen, p. 7] | [V] | Tool + evaluation; 12 Android apps, single run | moderate | |
| RQ 4 | Bug counts across functional-testing tools are oracle-dependent and not directly comparable; scope analysis used instead | [xiong2026propgen, p. 8; xiong2026propgen, p. 10] | [V] | Tool + evaluation; 12 Android apps, single run | moderate | |
| RQ 4 | No repeated runs reported (single run stated for RQ4 baselines) at temperature 1.0; no variance or statistical tests found (our check) | [xiong2026propgen, p. 7; xiong2026propgen, p. 8] | [I] | Tool + evaluation; 12 Android apps, single run | moderate | |
| RQ 4 | Agent-generated behavioural tests reach 79.7% mean line coverage vs 56.8% for native suites; against dedicated behavioural suites, 62.2% vs 66.1%. | [yang2026programbench, p. 7; yang2026programbench, p. 8] | [V] | Benchmark; 200 programs x 9 models, 1 run each | weak | preprint |
| RQ 4 | An assertion-quality linter plus gold/dummy filtering cut trivially passable tests from 18.5% to 3.7% (dummy pass rate). | [yang2026programbench, p. 8] | [V] | Benchmark; 200 programs x 9 models, 1 run each | weak | preprint |
| RQ 4 | With internet access, 20–36% of tasks for the three stronger models were flagged as cheating by a 9-judge majority vote (mostly source lookup); judges disagreed on 40–57% of these tasks (pooled κ = 0.57), so the authors block internet access by default. | [yang2026programbench, p. 6; yang2026programbench, p. 17] | [V] | Benchmark; 200 programs x 9 models, 1 run each | weak | preprint |
| RQ 4 | Authors state that the test-pass percentage does not reliably correlate | [yang2026programbench, p. 5] | [A] | Benchmark; 200 programs x 9 models, 1 | weak | preprint |

| | | | | | | |
|---|---|---|---|---|---|---|
| | with the share of working functionality and is only valid for relative comparison. | | | run each | | |
| RQ 4 | Functional and security oracles are human-written upstream tests, validated by execution on masked, vulnerable and fixed commits; no LLM judge is used | [zhao2025vibe, p. 5] | [V] | Benchmark; 186 tasks x 12 agent settings, single pass@1 | weak | preprint: downgraded from moderate |
| RQ 4 | Five experts reviewed security tests: 6.5% were coupled to the reference implementation, 44.0% might miss edge-case secure alternatives; tests were rewritten; no inter-annotator agreement reported | [zhao2025vibe, p. 19] | [V] | Benchmark; 186 tasks x 12 agent settings, single pass@1 | weak | preprint: downgraded from moderate |
| RQ 4 | Results are single-run pass@1 with no confidence intervals or significance tests | [zhao2025vibe, p. 6] | [V] | Benchmark; 186 tasks x 12 agent settings, single pass@1 | n/a (design fact) | |
| RQ 4 | CLIP/SSIM acknowledged not to capture human-perceived similarity; no correlation with human ratings reported | [zhou2025declarui, p. 12] | [A] | Technique + baselines; 50 apps (250 pages); user study 5 devs | moderate | |
| RQ 4 | Ablation CSR values (98%, 80%) impossible for 13 apps; user-study test ignores rating dependence | [zhou2025declarui, pp. 9–10] | [I] | Technique + baselines; 50 apps (250 pages); user study 5 devs | moderate | |
| RQ 4 | Smell counts are not normalised by size, Exp I has no statistical tests, and functional correctness is not measured despite claims about it | [zhu2026understanding, p. 1; zhu2026understanding, p. 7; zhu2026understanding, p. 8] | [I] | Exploratory static analysis; 90 problems x 5 LLMs; ~20 MetaGPT apps | weak | |
| RQ 5 | Authors call for industrial and longitudinal studies of | [asdaque2026novice, p. 4] | [A] | MSR mining; 22,953 | weak | |

| | | | | | | |
|---|---|---|---|---|---|---|
| | how AI's role changes with developer experience | | | agentic PRs / 1,719 users | | |
| RQ 5 | Fidelity is measured in detail, but maintainability of rebuilt code is assessed only through size and structure proxies. Our appraisal; **[A]** authors themselves say the framework data show "architectural substitution rather than implementation quality" ([bai2026recreationworld, p. 16]) | [bai2026recreationworld, p. 15; bai2026recreationworld, p. 16] | **[I]** | Benchmark; 250 apps x 10 models, 5 platforms, rollouts not reported (probably 1) | weak | preprint: downgraded from moderate |
| RQ 5 | Authors call for studies of third-generation agents, AI-assisted second developers, security, code bloat and cognitive debt, and longitudinal designs | [borg2026echoes, p. 62; borg2026echoes, p. 63] | **[A]** | Preregistered 2-phase experiment; RCT n=75 (Phase 1 n=76) | moderate | |
| RQ 5 | Benchmark scores behaviour and appearance only; single-file output means code quality/maintainability is not assessed (our appraisal of the metric definitions and prompt) | [chen2025iwr, p. 7; chen2025iwr, p. 20] | **[I]** | Benchmark; 113 video tasks x 28 LVLMs, single run | weak | preprint |
| RQ 5 | Authors call for longitudinal studies of debt accumulation, LLM-specific metrics (incl. reproducibility across runs) and community benchmarks | [ehsani2026faster, pp. 25–26; ehsani2026faster, p. 29] | **[A]** | Multivocal review; 31 formal + 73 grey sources | moderate | analyst: review-level gap statement |
| RQ 5 | Authors call for empirical evidence on whether vibe coding yields defect/vulnerability patterns distinct from AI-assisted and human coding, and on which review practices work | [fawzy2026vibe, p. 10] | **[A]** | Grey-literature review; 101 sources, 518 coded units | moderate | |
| RQ 5 | Authors call for enterprise replications, other | [he2026speed, p. 11; | **[A]** | Staggered DiD + GMM; | moderate | |

| | | | | | | |
|---|---|---|---|---|---|---|
| | tools/languages, and research on why LLMs generate overly complex code | he2026speed, p. 16] | | 806 Cursor repos vs matched controls | | |
| RQ 5 | Visual fidelity and screenshot-based observation are not evaluated; authors name them, plus desktop/mobile apps, as future work. Code quality of rebuilds is not measured (our observation). | [kim2026programdistill, p. 14] | **[A]** | Benchmark; 300 partial tasks x 9 models; 12 apps x 3 models full | weak | preprint |
| RQ 5 | Tests depend on data-testid anchors taken from the original source and supplied in the prompt, so a black-box rebuild could fail on naming rather than behaviour | [liu2026benchmarking, p. 16] | **[I]** | Benchmark; 46 web apps x 36 settings x 3 runs | weak | |
| RQ 5 | Authors call for other LLMs, more languages and metrics, RAG and cost–quality trade-off studies | [molison2025llm, p. 10] | **[A]** | Comparative static analysis; GPT-4o vs APPS humans (N per group not reported) | weak | |
| RQ 5 | Authors call for more runs per tool, premium tiers and additional static analysers before definitive conclusions | [mota2026comparing, p. 6] | **[A]** | Exploratory static analysis; 9 apps (3 per platform) | weak | preprint |
| RQ 5 | Authors propose quality-aware generation and multi-turn/agentic evaluation under the same benchmark as next steps. | [pan2026functional, p. 5; pan2026functional, p. 10] | **[A]** | Benchmark; 38 Python repos x 16 LLMs x 3 runs | weak | preprint |
| RQ 5 | Authors call for dynamic webpages evaluated on interactive functions beyond visual similarity | [si2025benchmarking, p. 10] | **[A]** | Benchmark; 484 (+80 HARD) pages; human eval 100 pages | moderate | |

| | | | | | | |
|---|---|---|---|---|---|---|
| RQ 5 | Authors state passing functional tests does not imply maintainable, secure or documented code | [tran2026vibe, p. 9] | [A] | Benchmark; 50 test apps x 16 models, 1 generation each | moderate | |
| RQ 5 | Output is a web proxy, not native; agent also receives human-reviewed navigation description | [wu2026looks, p. 3; wu2026looks, p. 9] | [V] | Benchmark; 29 apps x 6 models = 174 single runs | moderate | |
| RQ 5 | Back-end-dependent interactions (login, search) not evaluable; future work | [xiao2025benchmarking, p. 11] | [A] | Benchmark; 127 pages / 374 interactions x 6 MLLMs x 3 prompts | moderate | |
| RQ 5 | Imprecise LLM properties are common (127 of 985 caused false positives) but 92.9% could be refined, mainly via pre/postconditions | [xiong2026propgen, p. 9] | [V] | Tool + evaluation; 12 Android apps, single run | moderate | |
| RQ 5 | Authors list non-functional properties (speed, memory) as untested, and multi-agent and human-guided reconstruction as future work. | [yang2026programbench, p. 12] | [A] | Benchmark; 200 programs x 9 models, 1 run each | weak | preprint |
| RQ 5 | Authors call for property-based/adversarial tests and static analysis to complement test-based security evaluation | [zhao2025vibe, p. 24] | [A] | Benchmark; 186 tasks x 12 agent settings, single pass@1 | weak | preprint |
| RQ 5 | No back-end/business logic, image assets ignored; sample 50 apps limited by token cost | [zhou2025declarui, p. 2; zhou2025declarui, p. 12] | [A] | Technique + baselines; 50 apps (250 pages); user study 5 devs | moderate | |
| RQ 5 | Authors call for semantic-cohesion metrics, longitudinal evolution benchmarks and architecture-aware/refactoring agents | [zhu2026understanding, p. 10] | [A] | Exploratory static analysis; 90 problems x 5 LLMs; ~20 MetaGPT apps | weak | |

# Contradictions

<!-- Findings that disagree: which papers, what differs (data, method, population, metric), likely explanation (INTERPRETATION). -->

Each entry records both sides with page refs, then compares data, population, method, metric and time. Explanations are labelled **[I]**. We do not pick a side.

## C1. Does AI assistance harm maintainability? he2026speed vs borg2026echoes

- **Side A [V]**: Cursor adoption in 806 OSS repositories raised total SonarQube static-analysis warnings by +30.26% and total cognitive complexity by +41.64% relative to matched controls (Borusyak DiD) [he2026speed, p. 7]. The complexity effect stays significant after controlling for codebase size in the GMM (0.086, p<0.01) [he2026speed, p. 8].
- **Side B [V]**: When a second developer evolved AI-co-developed code without AI, there was no significant difference in completion time [borg2026echoes, p. 33], CodeHealth (8.49 vs 8.44) [borg2026echoes, p. 38] or test coverage [borg2026echoes, p. 35]. The only "significant" maintainability effect, a Bayesian +0.10 CodeHealth for code from habitual AI users, favours AI [borg2026echoes, p. 39].
- **Differences**:
- Data and population: real-world OSS repositories with ≥10 stars [he2026speed, p. 4] vs one 2 KLoC Java/Spring Boot task done by 151 volunteer professionals [borg2026echoes, p. 19; borg2026echoes, p. 64].
- Tool generation: an agentic IDE (Cursor Composer/agent) [he2026speed, p. 2] vs late-2024 assistants (Copilot, ChatGPT, Cursor) [borg2026echoes, p. 3; borg2026echoes, p. 60].
- Metric: repository-total warnings and cognitive complexity, which scale with code size [he2026speed, p. 5] vs a LoC-weighted file-level CodeHealth average plus a second developer's evolution time [borg2026echoes, p. 13].
- Time: monthly panel of up to about 20 months [he2026speed, p. 5] vs a single hand-off task.
- Robustness: he2026speed's quality effects turn negative and non-significant under the Callaway–Sant'Anna estimator (warnings −10.49%, complexity −3.80%) [he2026speed, p. 16]. borg2026echoes is underpowered (75 of 128 planned participants; 80% power only for d≈0.66) [borg2026echoes, p. 19].
- **Partial agreement [V]**: borg2026echoes' observational Phase 1 also found more duplication (23% vs 11%), more complex conditionals and very large code-volume outliers in AI-assisted code [borg2026echoes, p. 30; borg2026echoes, p. 46]. With size controlled, he2026speed finds no direct Cursor effect on warnings [he2026speed, p. 8].
- **Possible explanation [I]**: the two results need not conflict. Both fit "AI adds more code, and project-level totals rise with it, while per-file health and the short-term effort of the next developer are not clearly worse". The he2026speed effect depends on the estimator, and borg2026echoes can rule out only large effects. Neither study settles the question.

## C2. Fewer or more static-analysis issues in LLM code? molison2025llm vs he2026speed

- **Side A [V]**: GPT-4o solutions had far lower odds of SonarQube-flagged "bugs" than APPS human solutions (OR 0.00–0.019) [molison2025llm, p. 8], and lower SonarQube fix effort on introductory and interview tasks [molison2025llm, p. 7].
- **Side B [V]**: Cursor adoption raised repository SonarQube warnings by +30.26% [he2026speed, p. 7].
- **Differences**:
- Unit: per-solution odds of an issue in single-file stdin/stdout programs [molison2025llm, p. 5] vs monthly repository totals [he2026speed, p. 5].
- Baseline: competitive-programming submissions written for correctness, not maintainability [molison2025llm, p. 9] vs matched non-adopting repositories, which may themselves use other AI tools [he2026speed, p. 7].
- Metric: the SonarQube "bug" category vs all reliability, maintainability and security warnings.
- Leakage: molison2025llm's few-shot prompt contains the human reference solutions [molison2025llm, p. 4].
- Time: GPT-4o in 2024 vs Cursor agents in 2024–2025.
- **Possible explanation [I]**: the results are not comparable. Both use SonarQube, but the units, baselines and code types differ, so neither one refutes the other. molison2025llm's smell results are themselves mixed: zero-shot code had 3.57x the odds of a smell on introductory tasks [molison2025llm, p. 8].

## C3. Which instrument measures the quality of AI code? zhu2026understanding vs SonarQube-based studies

- **Side A [A]**: zhu2026understanding chose PyExamine to separate structural and architectural smells from the "superficial linting issues" counted by SonarQube [zhu2026understanding, p. 2; zhu2026understanding, p. 4]. It found more Long Method smells in larger LLMs' code than in human code [zhu2026understanding, p. 7].
- **Side B [V]**: SonarQube-based studies report fewer issues in LLM code [molison2025llm, p. 8], a platform-dependent issue density (22.83 vs about 9 issues/KLOC) [mota2026comparing, p. 5], or more warnings with adoption [he2026speed, p. 7].
- **Context [A]**: technical-debt tools disagree on what counts as real debt [ehsani2026faster, p. 5], and SonarQube and general linters were not designed for LLM-specific debt [ehsani2026faster, p. 22]. mota2026comparing calls SonarQube an auxiliary indicator only [mota2026comparing, p. 6]. borg2026echoes argues that CodeHealth outperforms the SonarQube Maintainability Rating; note that its first author works for CodeScene, the vendor of CodeHealth [borg2026echoes, p. 4; borg2026echoes, p. 75].
- **Differences**: the tools differ (PyExamine, SonarQube, CodeHealth, MI), and so do the task types (algorithmic snippets, generated web apps, OSS repositories). zhu2026understanding's Exp I counts are small, unnormalised and untested [zhu2026understanding, p. 7].

- **Possible explanation [I]**: the direction of "LLM vs human code quality" may depend on the measuring instrument. The field has no validated, size-normalised maintainability measure for AI-generated code.

## C4. Does iterative feedback or self-repair help? pan2026functional vs tran2026vibe / zhou2025declarui / wu2026looks

- **Side A [V]**: On GPT-5.2, three one-shot strategies all lowered the aggregate non-functional score without raising functional correctness: self-repair, environment repair and planning (NF 46.1–49.9% vs 57.1% at baseline) [pan2026functional, p. 8].
- **Side B**:
- **[V]** Browser self-testing during generation correlates with accuracy across 16 models (r = 0.72) [tran2026vibe, p. 7].
- **[V]** Removing DeclarUI's page-transition graph plus refinement loop cuts navigation coverage to 23.1% [zhou2025declarui, p. 10]. Its compile success is 98% vs 74–76% for raw MLLMs [zhou2025declarui, p. 9].
- **[V]**/**[A]** A tsc + vite verify-repair loop gives 100% build success for all six models [wu2026looks, p. 6]. The authors attribute this to the loop [wu2026looks, p. 12].
- **Further evidence [A]**: agents' own validation missed behavioural mismatches with the reference [kim2026programdistill, p. 12].
- **Differences**:
- Feedback type: generic one-shot repair of a whole Python repository [pan2026functional, p. 8], targeted compile and navigation checks [zhou2025declarui, p. 5], and build checks [wu2026looks, p. 6].
- Design: a single-model intervention (pan2026functional) vs an observational correlation across models, where the system prompt itself demands browser testing [tran2026vibe, p. 16].
- Outcome: non-functional quality (pan2026functional) vs compilation, navigation or build (zhou2025declarui, wu2026looks).
- Circularity: zhou2025declarui scores navigation against the same graph that guides its refinement [zhou2025declarui, p. 7].
- **Possible explanation [I]**: feedback improves the property it checks (compilation, build, navigation edges). Self-testing goes along with stronger models. No study shows that generic repair improves both behavioural fidelity and code quality, and the only study that measures quality after repair finds harm (single model, single run).

## C5. Do rebuilds "look right but behave wrong"? chen2025iwr / lu2025webgen / xiao2025benchmarking vs bai2026recreationworld / wu2026looks

- **Side A [V]**:
- Interactive functionality is below visual fidelity for all 28 models (GPT-5 IFS 24.39 vs VFS 64.25) [chen2025iwr, p. 8].

- Design-validation tests are passed more often than functional tests (e.g. 44.3% vs 21.1%) [lu2025webgen, p. 9].
- Interactive regions score lower than full pages for all six models [xiao2025benchmarking, p. 6].
- **Side B [V]**:
- In bai2026recreationworld the visual (VLM) channel is below the programmatic channel for 9 of 10 models [bai2026recreationworld, p. 11], although static structure beats actions within the programmatic tests [bai2026recreationworld, p. 16].
- In wu2026looks navigation passes 58–92% [wu2026looks, p. 7], while visual fidelity stays at or below 2.99/5 [wu2026looks, p. 6].
- **Differences**:
- chen2025iwr's VFS is averaged only over reached checkpoints, so part of its IFS<VFS gap comes from how the metrics are built [chen2025iwr, p. 7].
- Visual scores come from different instruments: OCR+DINO+MLLM rubric, VLM assertions, Borda ranks and CLIP.
- Functional scores also differ: action replay, accessibility tests and navigation tests.
- **Possible explanation [I]**: bai2026recreationworld and chen2025iwr are only *directionally consistent* on one point: static structure is reproduced more easily than interaction logic. The broader claim "visual fidelity systematically exceeds functional fidelity" is not supported, because the visual and functional scales are not commensurable across or within papers.

## C6. Multi-agent / agentic frameworks: MetaGPT fails in liu2026benchmarking but leads in mock2026vibe

- **Side A [V]**: MetaGPT nearly always fails on E2EDev (0–5.4% Req.Acc) [liu2026benchmarking, p. 6]. Multi-agent frameworks "do not consistently improve" over a plain LLM (by our count, 9 of 24 cells higher **[I]**) [liu2026benchmarking, p. 6].
- **Side B [V]**: Agentic MetaGPT X runs had the highest test pass rate (mean 94.58%) [mock2026vibe, p. 25; mock2026vibe, p. 41]. zhu2026understanding generated complete multi-file apps with MetaGPT, but only after adding explicit modularity prompts [zhu2026understanding, p. 5].
- **Differences**:
- Task size: a two-function string task [mock2026vibe, p. 12] vs 46 multi-requirement web apps.
- Backbone: GPT-5 and Claude Sonnet 4 [mock2026vibe, p. 17] vs older or smaller backbones such as GPT-4o and Qwen2.5 [liu2026benchmarking, p. 5].
- Product: MetaGPT X web interface vs the open-source framework.
- Evaluation: 12 runs pooled vs 46 projects x 3 runs.
- **Possible explanation [I]**: framework value depends on the backbone and the task scale. mock2026vibe's agentic advantage is confounded with the newer model generation.

## C7. Security of generated code: pan2026functional vs zhao2025vibe

- **Side A [V]**: Reference-normalised security averages 87.40% and is the strongest non-functional dimension [pan2026functional, p. 6].
- **Side B [V]**: 79.3% of the functionally correct solutions of the best agent fail targeted security tests, and SecPass is only 3.8–12.9% [zhao2025vibe, p. 6].
- **Differences**: pan2026functional counts high-risk static findings relative to the reference, and the score can reward incomplete code. zhao2025vibe runs 3.5 targeted security tests per task for a known historical CWE [zhao2025vibe, p. 5]. The tasks also differ: whole-repository generation vs security-sensitive feature patches.
- **Possible explanation [I]**: the two instruments measure different things, a static absence of flagged patterns vs exploit-relevant behaviour. "Secure" cannot be compared across them.

## C8. More code or less code? Rebuilds are smaller, but assisted development adds code

- **Side A [V]**: Reconstructions are much smaller than their references. In bai2026recreationworld they are 89.4% smaller with 92.3% fewer files (4 of 10 models; desktop and Android only) [bai2026recreationworld, p. 15]. In yang2026programbench they are shorter (median 1,173 vs 3,068 lines) and use fewer files [yang2026programbench, p. 9].
- **Side B [V]**:
- Cursor adoption raises lines added by +281.3% in month 1 [he2026speed, p. 8].
- AI-assisted Task 1 code produced outliers of up to 10,788 lines [borg2026echoes, p. 30].
- In MetaGPT apps, architectural smell counts track code size ($\rho = 0.94$) [zhu2026understanding, p. 8].
- The visual-fidelity leader writes 1,441 LoC vs 668 for GPT-5 [wu2026looks, p. 7].
- **Differences**: rebuilds from scratch scored against a behavioural oracle of a mature original, vs incremental additions to existing projects or apps generated from a spec.
- **Possible explanation [I]**: reconstructions under-implement the original, which is consistent with low full-pass rates. Assisted development adds volume. Size therefore carries opposite signals in the two settings, and size-driven quality metrics (smell counts, totals) must be normalised before comparison.

## C9. Apparent disagreements that come from different constructs (do not pool)

- **"Vibe coding" means different things [V]**:
- fawzy2026vibe: natural-language prompting plus re-prompting with minimal review [fawzy2026vibe, p. 1].
- asdaque2026novice: a supervised agent workflow, operationalised as opening agentic PRs [asdaque2026novice, p. 1]. Its authors warn that the findings may not hold under a broader definition [asdaque2026novice, p. 4].
- mock2026vibe: human-written TDD tests plus scripted prompts that participants could not modify [mock2026vibe, p. 15].

- he2026speed: Cursor configuration files committed to a repository (intent-to-treat) [he2026speed, p. 4].
- ehsani2026faster: a driver of fast-integration debt in mostly grey sources [ehsani2026faster, p. 17].
- **[I]** These are different interventions. Their results should not be pooled as "effects of vibe coding".
- **Fidelity scores are not comparable [V]**:
- 58.06% average of programmatic and VLM channels [bai2026recreationworld, p. 11].
- 36.35 Final Score (0.7 IFS + 0.3 VFS) [chen2025iwr, p. 8].
- 49.15% cumulative workflows [kim2026programdistill, p. 11].
- 53.75% Req.Acc with data-testid anchors [liu2026benchmarking, p. 6].
- 27.8% UI-agent accuracy with half credit for PARTIAL [lu2025webgen, p. 8].
- 61.77% workflows passing at ≥90% of substeps [tran2026vibe, p. 6].
- 0% resolved / 3% almost [yang2026programbench, p. 6].
- **[I]** Each figure uses a different oracle, input, application type and model generation. Do not rank across them.

## C10. Same model family builds or judges the benchmark and is evaluated (possible bias; untested)

- **Overlap and the same family wins [V]**:
- yang2026programbench: Claude Sonnet 4.5 wrote the tests [yang2026programbench, p. 3], and Claude models top the leaderboard [yang2026programbench, p. 31].
- tran2026vibe: GPT-5 drafted the specs [tran2026vibe, p. 3], and GPT-5.3-Codex ranks first [tran2026vibe, p. 6]. The canonical judge is Claude Sonnet 4.5 [tran2026vibe, p. 4].
- kim2026programdistill: GPT-5.6 Sol builds the tasks [kim2026programdistill, p. 2], GPT-6 Astra ranks first [kim2026programdistill, p. 8], and GPT-6 Astra also classifies the failures [kim2026programdistill, p. 12].
- zhao2025vibe: Claude 4 Sonnet runs the environment-building agent [zhao2025vibe, p. 17], and SWE-agent constructs the tasks [zhao2025vibe, p. 4]. The best setting is SWE-agent + Claude 4 Sonnet [zhao2025vibe, p. 7].
- **Overlap without a visible advantage [V]**:
- liu2026benchmarking: GPT-4o built the annotations and is evaluated [liu2026benchmarking, p. 13]; the best backbone is Claude-Haiku.
- lu2025webgen: GPT-4o wrote the test cases and scored low [lu2025webgen, p. 8].
- bai2026recreationworld: the Qwen3.7-Plus judge is the lowest-scoring model (average 9.15) [bai2026recreationworld, p. 11].
- chen2025iwr: the Gemini-2.5-Pro judge ranks mid-table [chen2025iwr, p. 7].
- wu2026looks: the judge is Gemini 2.5 Pro, Opus leads [wu2026looks, p. 13], and the authors report no self-preference **[A]**.
- **[I]** No paper tests construction-model or judge-model self-preference directly. The pattern is a validity threat, not a demonstrated bias.

## Agreements / converging evidence

- **G-A1. Whole-application functional fidelity is far from complete [V]**. Every benchmark agrees:
- Full behavioural equivalence: 2.8% of apps [bai2026recreationworld, p. 11] and 0 of 200 programs [yang2026programbench, p. 6].
- Best workflow-level or requirement-level pass rates: 49–62% [kim2026programdistill, p. 11; liu2026benchmarking, p. 6; tran2026vibe, p. 6].
- Lower still in some benchmarks: IFS <25 [chen2025iwr, p. 8], <45% [pan2026functional, p. 5], 27.8% [lu2025webgen, p. 8].
- Strength [I]: **moderate**. The direction is consistent across 8 independent benchmarks, but 5 of them are preprints and most use single runs.
- **G-A2. Behaviour and interaction are harder than static structure [V]**:
- Structure beats actions [bai2026recreationworld, p. 16].
- Static tasks score higher than interactive ones [chen2025iwr, p. 16].
- Interactive parts score lower than full pages [xiao2025benchmarking, p. 6].
- UI plus logic is harder than logic alone [kim2026programdistill, p. 38].
- "Click has no effect" is the largest navigation failure category (48.4%) [wu2026looks, p. 8].
- Strength [I]: **moderate**, with the measurement caveat in C5.
- **G-A3. Fidelity falls as dependency depth, size or complexity rise [V]**: [kim2026programdistill, p. 8; yang2026programbench, p. 31; tran2026vibe, p. 6; zhao2025vibe, p. 14]. Strength [I]: **moderate**.
- **G-A4. Functional correctness and other quality dimensions diverge [V]**:
- Best static quality but lowest Pass@1 [molison2025llm, p. 8].
- Correct but insecure [zhao2025vibe, p. 6].
- Equal functional scores but very different non-functional scores [pan2026functional, p. 5].
- The visual leader is not the maintainability leader [wu2026looks, p. 7].
- Authors state that passing functional tests does not imply maintainable code **[A]** [tran2026vibe, p. 9].
- Strength [I]: **moderate**. The pattern is consistent, but each study measures a different second dimension.
- **G-A5. Rebuilt code is structurally simpler and more monolithic than the original [V]**: [bai2026recreationworld, p. 15; yang2026programbench, p. 9]. Strength [I]: **weak**. It rests on 2 preprints, selected subsets and size proxies only.
- **G-A6. Code quality and maintainability are rarely measured in fidelity benchmarks**:
- Not measured at all [V]: [si2025benchmarking, p. 3; xiao2025benchmarking, p. 5; kim2026programdistill, p. 14].
- Explicitly left out [A]: [tran2026vibe, p. 9].
- Measured only by proxies [V]: size and structure [bai2026recreationworld, p. 15; yang2026programbench, p. 9], LoC/reuse proxies [wu2026looks, p. 4], a single MI index [pan2026functional, p. 3], Likert ratings [zhou2025declarui, p. 7].
- Strength [I]: **strong** as a description of the field (verified absences across 12+ papers).
- **G-A7. LLM judges must be validated per judge model [V]**:

- Judge–human agreement ranges from 36.1% to 86.4% depending on the evaluator model [tran2026vibe, p. 9].
- 86–94% raw agreement on 3 settings [lu2025webgen, p. 9].
- $\tau = 0.716$ on 55 author-labelled scenarios [wu2026looks, p. 13].
- $\rho = 0.65$, tuned in-sample [chen2025iwr, p. 17].
- Low human inter-annotator $\kappa$ (0.26–0.46) [si2025benchmarking, p. 16].
- Strength **[I]**: **moderate**.
- **G-A8. Practitioners perceive vibe-coded output as fast but flawed [V]**: [fawzy2026vibe, p. 8]. This converges with LLM-linked code and design debt in mostly grey sources [ehsani2026faster, p. 16] and with higher review burden for novice agent users [asdaque2026novice, p. 3]. Strength **[I]**: **weak**. It rests on perception and proxy evidence, not code measurement.